

Trajectory Optimisation and Online Replanning for a Small-Scale Vertical Take-Off Vertical Landing Launch Vehicle

Lucas White

Supervisor: Dr Yuanbo Nie

A dissertation thesis submitted in partial fulfilment of the requirements for the degree of

BEng Aerospace Engineering

In the School of Mechanical, Aerospace and Civil Engineering
at the University of Sheffield

May 2026

Word count: 6509

Declaration

'Lucas White certifies that all the material contained within this thesis *Trajectory Optimisation and Online Replanning for a Small-Scale VTVL Launch Vehicle* is their own work except where it is clearly referenced to others.'

Student's signature: 

Student number: 230157015

Date: 05/2026

Abstract

Reusable Vertical Take-off Vertical Landing (VTVL) launchers have driven a decade of falling launch cost, with the enabling technology now lying in onboard guidance, navigation, and control (GNC) rather than propulsion. Closed-loop integration of an offline fuel-optimal reference with an online convex replanner, on an academic sub-25 kg testbed, has received limited published treatment. This dissertation develops and verifies a hybrid GNC architecture for the University of Sheffield *Hopper* vehicle, coupling an offline four-phase fuel-optimal trajectory, transcribed by Hermite–Simpson collocation in ICLOCS2/IPOPT, to an online sequential convex programming (SCP) model predictive controller with lossless convexification (LCvx) of the terminal thrust constraint, arbitrated by a five-flag validity-gated two-mode dispatcher with a three-strike fallback latch. State estimation is provided by a 17-nominal-state (16-error-state) multiplicative extended Kalman filter and the inner attitude loop by a mass-gain-scheduled linear-quadratic-integral regulator.

The offline optimal control problem (OCP) is verified globally optimal across a 50-run multi-start campaign with dispersion $\sigma_{\text{fuel}} = 0$ to machine precision, and yields a 15.8 % propellant and 17.6 % mission-time saving over an identically constrained precision-tracking baseline. The 14-state quaternion 6-DoF plant passes seven analytical benchmarks at 10^{-8} tolerance. End-to-end closed-loop simulation in Simulink achieves nominal landing with miss distance 7.4 cm, terminal speed 0.29 m/s, propellant use 0.271 kg, zero validity-gate rejections, and median per-call SOCP solve time 117 ms (maximum 176 ms across $N = 6$ calls) against a 500 ms real-time budget, leaving the median at 23 % utilisation and sufficient margin for an embedded port subject to worst-case execution-time analysis.

The architecture is validated as a pre-flight maturation step for the Sunride Lander Challenge entry; Monte Carlo dispersion, wind injection, bias-covariance unfreezing, and hardware-in-the-loop testing are identified as immediate future work.

Acknowledgements

I would like to thank my supervisor, Dr Yuanbo Nie, for his consistent guidance and feedback throughout this project, and for the technical insight that shaped many of the design decisions presented in this dissertation. I am also grateful to his PhD student, Nikilesh Ramesh, whose advice on trajectory optimisation was invaluable.

Above all, I am grateful to my friends and family for their patience, encouragement, and quiet confidence in me throughout the year, this dissertation would have been a much harder undertaking without them.

Nomenclature

Roman Symbols

Symbol	Description	Units
$\mathbf{r}$	Position vector in inertial (Earth) frame	m
$\mathbf{v}$	Velocity vector in inertial frame	$\mathrm{m\,s^{-1}}$
m	Vehicle instantaneous mass	kg
m_0	Initial (wet) mass	kg
m_s	Dry (structural) mass	kg
m_f, m_o	Fuel and oxidiser propellant mass	kg
$\mathbf{T}$	Inertial-frame thrust vector	N
T	Thrust magnitude	N
$T_{\min}$	Minimum thrust (1% throttle floor)	N
$T_{\max}$	Maximum thrust	N
$\mathbf{D}$	Aerodynamic drag force	N
$C_{D,xy}, C_{D,z}$	Lateral and axial drag coefficients	—
A_{xy}, A_z	Lateral and axial reference areas	$\mathrm{m^2}$
I_{sp}	Specific impulse	s
c	Effective exhaust velocity ($= I_{sp} g_0$)	$\mathrm{m\,s^{-1}}$
g_{sl}	Sea-level gravitational acceleration	$\mathrm{m\,s^{-2}}$
$\mathbf{g}(Z)$	Altitude-dependent gravity vector	$\mathrm{m\,s^{-2}}$
T_{er}	Earth-to-rocket direction cosine matrix	—
$\hat{\mathbf{n}}$	Thrust unit direction vector	—
$\mathbf{q}$	Attitude quaternion $[q_0, q_1, q_2, q_3]^T$	—
z_{CoM}	Centre-of-mass distance from gimbal pivot	m
I_{xy}	Lateral moment of inertia	$\mathrm{kg\,m^2}$
J	Cost functional	varies
N	Number of temporal nodes (MPC discretisation)	—
Γ	LCvx thrust-magnitude slack variable	N
σ	Time dilation decision variable	—

τ	Normalised time, $\tau \in [0, 1]$	—
U_L, U_T, U_M, U_F	Non-dimensionalisation scales	varies

Greek Symbols

Symbol	Description	Units
α, β	Gimbal angles producing pitch and roll torques	$^\circ$
θ, ϕ, ψ	Vehicle pitch, roll, yaw (Z–Y–X Euler)	$^\circ$
p, q	Body-axis angular rates (roll, pitch)	$^\circ\,\mathrm{s}^{-1}$
ρ	Atmospheric density	$\mathrm{kg\,m}^{-3}$
γ_{gs}	Glideslope half-angle	$^\circ$
$\boldsymbol{\nu}$	SCP virtual-control slack (dynamics defect)	—
Δ	SCP outer-iteration step ($\mathbf{x}^{i+1} - \mathbf{x}^i$)	—
s_δ	Trust-region soft slack	—
$\varepsilon_{\text{fuel}}, \varepsilon_{\text{gimbal}}$	Cost regularisation weights	—
w_ν, w_δ, w_t	SCP cost weights (virtual control, trust, time)	—
ζ, ω_n	Servo damping ratio and natural frequency	$\mathrm{rad\,s}^{-1}$

Subscripts and Superscripts

Symbol	Description
$(\cdot)_{\text{ref}}$	Reference (locked ICLOCS) trajectory value
$(\cdot)_{\text{cmd}}$	Commanded value
$(\cdot)_{nd}$	Non-dimensionalised quantity
$(\cdot)^i$	SCP outer-iteration index
$(\cdot)_k$	Temporal node index in MPC horizon
$(\cdot)_e, (\cdot)_r$	Earth-frame and rocket-frame components

Abbreviations

Acronym	Definition
VTVL	Vertical Take-Off, Vertical Landing
RLV	Reusable Launch Vehicle
GNC	Guidance, Navigation, and Control
OCP	Optimal Control Problem
NLP	Nonlinear Programming

MPC	Model Predictive Control
SCP	Sequential Convex Programming
SCvx	Successive Convexification (Mao/Açıkmeşe variant of SCP)
LCvx	Lossless Convexification
SOCP	Second-Order Cone Programme
FOH	First-Order Hold
STM	State-Transition Matrix
MEKF	Multiplicative Extended Kalman Filter
TVC	Thrust Vector Control
DoF	Degrees of Freedom
LQI	Linear-Quadratic Integral controller
CEP	Circular Error Probable
IMU	Inertial Measurement Unit
GNSS	Global Navigation Satellite System
ICLOCS2	Imperial College London Optimal Control Software (v2)
IPOPT	Interior Point OPTimizer
G-FOLD	Guidance for Fuel-Optimal Large Diverts (Scharf et al.)
OFMS	Onboard Flight Management System (G-FOLD pattern)
MEMS	Micro-Electro-Mechanical Systems

Contents

Abstract **2**

Acknowledgements **3**

Nomenclature **4**

1 Introduction **12**
1.1 Background 12
1.2 Aim and Objectives 12
1.3 Scope of the Study 13
1.4 Methodology Overview 13
1.5 Thesis Outline 15

2 Literature Review **16**
2.1 Reusable Launch and the On-Board Guidance Bottleneck 16
2.2 Trajectory Optimisation: From Theory to Direct Methods 16
2.3 Convex Optimisation for Online Powered-Descent Guidance 17
2.4 State Estimation for High-Bandwidth Guidance 17
2.5 Attitude Control and Thrust-Vector Control 18
2.6 GNC Integration and Dispatcher Logic 18
2.7 Academic Testbeds and Project Context 19
2.8 Synthesis 19

3 Methodology **20**
3.1 Architecture Overview 20
3.2 Offline Trajectory Optimisation 20
3.3 6-DoF Plant and Environment 22
3.4 Navigation: 16-State Multiplicative EKF 23
3.5 Online Guidance: SCP-MPC with Lossless Convexification 23
3.6 Phase Scheduling and Dispatcher 26
3.7 Reference Tracking and Inner Loop 28
3.8 Closed-Loop Integration and Test Campaign 28

4 Results **29**
4.1 Offline Trajectory Optimisation Results 29
4.2 Plant and Subsystem Verification 34
4.3 Closed-Loop Subsystem Tests 38
4.4 Full Simulink Nominal Closed-Loop 42
4.5 Solver Performance and Embedded Feasibility 47

5 Conclusions **51**
5.1 Summary of Contributions and Objective Closure 51
5.2 Limitations 51
5.3 Future Work 52

References **53**

A Reflective Statement **58**
Project Gantt Chart 60

B Use of AI Statement **63**

C Viva Poster **64**

D Project Aim, Objectives, and Research Questions **66**

E Risk Register **68**

F MATLAB Source Code **70**
F.1 Offline Trajectory Optimisation (ICLOCS2) 70
F.1.1 Driver Script 70
F.2 Online Guidance: SCP-MPC with Lossless Convexification 87
F.2.1 SOCP Builder 87
F.2.2 SCP Outer Loop 92
F.2.3 MPC Wrapper 96
F.3 Navigation: Multiplicative EKF Initialisation 102
F.3.1 Initialisation Routine 102

G Simulink Model **104**

H Ethics Approval Letter **107**

List of Figures

1.1 Reference mission profile: 3D trajectory 14
1.2 Reference mission profile: altitude 15

3.1 Closed-loop GNC architecture for the *Desert Winds* simulation. The Guidance layer (offline ICLOCS reference plus online SCP-MPC) feeds the validity-gated dispatcher, which arbitrates between Mode A (reference tracking) and Mode B (online replanning) before commanding the gain-scheduled LQI inner loop and TVC actuators. The Navigation layer (16-state MEKF) closes the loop. 21
3.2 Lossless convexification of thrust-magnitude constraints 25
3.3 Mode dispatcher state machine. Mode A executes phases 1 to 3; on entry to Phase 4 the dispatcher enables Mode B subject to the five-flag validity gate. Three consecutive validity failures latch the dispatcher into Mode A fallback for the remainder of the landing phase. 27

4.1 Precision-optimal vs fuel-optimal: 3D trajectory 30
4.2 Precision-optimal vs fuel-optimal: mass consumption 31
4.3 ICLOCS2 reference: inertial position 32
4.4 ICLOCS2 reference: inertial velocity 33
4.5 ICLOCS2 reference: thrust command 34
4.6 Plant verification: hover trim altitude 35
4.7 Plant verification: hover trim velocity 35
4.8 Plant verification: mass depletion 36
4.9 Plant verification: free fall 36
4.10 Plant verification: axial drag model 37
4.11 Plant verification: pitch acceleration vs gimbal angle 37
4.12 SCP outer-loop: virtual-control norm 39
4.13 SCP outer-loop: trust-region step 39
4.14 SCP outer-loop: fuel cost 40
4.15 SCP trajectory refinement: 3D view 41
4.16 SCP trajectory refinement: altitude profile 42
4.17 Closed-loop Mode B: 3D trajectory 44
4.18 Closed-loop Mode B: altitude 45

4.19 Closed-loop Mode B: top-down view 45
4.20 Closed-loop Mode B: thrust magnitude 46
4.21 Closed-loop Mode B: inertial speed 46
4.22 Closed-loop Mode B: SCP diagnostics 47
4.23 SCP-MPC solve time distribution 48
4.24 Monte Carlo: landing accuracy 48
4.25 Monte Carlo: final velocity 49
4.26 Monte Carlo: fuel deviation 49
4.27 Monte Carlo: landing dispersion 50

G.1 Simulink top-level model (left half) 105
G.2 Simulink top-level model (right half) 106

List of Tables

3.1 *Desert Winds* vehicle and mission parameters. 22
3.2 Sequential Convex Programming algorithm parameters. 25

4.1 Plant verification test results 38
4.2 SCP outer-loop iteration history 42
4.3 Monte Carlo dispersion ranges applied at Phase 4 entry (uniform, 50 seeds). 50

Chapter 1

Introduction

1.1 Background

Reusable Vertical Take-Off Vertical Landing (VTVL) launchers have driven a sustained decline in orbital launch cost over the past decade [1–3]; the enabling technology now lies in onboard guidance, navigation, and control (GNC) rather than propulsion. Onboard GNC computes a fuel-feasible trajectory in flight, from a dispersed entry state to a metre-class landing target under non-convex thrust-shell constraints, glideslope cones, and aerodynamic disturbance [4–8]. Convex-optimisation-based powered-descent guidance has matured from theory [9] to flight demonstration on Masten's Xombie [10, 11], but its closed-loop integration at the sub-25 kg testbed scale, with a dispatcher arbitrating offline reference against online replanner, remains under-documented [12–15]. Prior Sunride work [16] solved the offline precision-tracking problem but did not address fuel-optimality, online replanning, or closed-loop integration.

1.2 Aim and Objectives

This dissertation develops and verifies a hybrid GNC architecture for the Sunride *Hopper* (modelled using the *Desert Winds* vehicle), coupling an offline fuel-optimal multi-phase reference trajectory [17, 18] with an online lossless-convexified MPC for the terminal landing phase [19, 20]. The architecture is then characterised in closed-loop simulation [21] as a pre-flight maturation step for the Lander Challenge entry. The aim decomposes into six SMART objectives, each closed one-to-one in Chapter 5.

O1 Formulate and solve the four-phase fuel-optimal optimal control problem for the 16.88 kg *Desert Winds* vehicle via Hermite–Simpson collocation in ICLOCS2/IPOPT [22–24], and establish solution uniqueness through multi-start dispersion analysis to $\sigma_{\text{fuel}} < 1 \times 10^{-3}$ kg.

O2 Quantify the propellant and mission-time cost of fuel-optimality against an identically constrained precision-tracking baseline [16] over the same four-waypoint mission.

O3 Implement a 14-state quaternion six-degree-of-freedom plant with smoothed $v|v|$ drag, mass-varying inertia, and two-axis centerline thrust vectoring[1, 25], and verify against analytical trim, free-fall, drag-symmetry, Tsiolkovsky mass-flow, and gimbal-sign benchmarks at 1×10^{-8} numerical tolerance.

O4 Design a successive convex programming [6, 26] model predictive controller with lossless convexification of the terminal thrust constraint, compiled as a parametric second-order cone program via YALMIP [4, 27] and solved by ECOS [28], achieving mean SCP iterations ≤ 5 and LCvx slack $< 0.1\,\mathrm{N}$ across the landing closed loop.

O5 Integrate the offline reference, online MPC, 16-state error-state multiplicative extended Kalman filter, and gain-scheduled linear-quadratic-integral attitude inner loop within a validity-gated two-mode dispatcher[13, 14] (Mode A reference tracking for ascent, hover, and descent; Mode B online MPC for landing), and execute a nominal end-to-end closed-loop simulation in Simulink.

O6 Profile the per-call SOCP wall-clock and SCP iteration count over the landing closed loop to assess feasibility for real-time embedded deployment[11].

1.3 Scope of the Study

This work is a pre-flight simulation study supporting the Sunride Lander Challenge entry. Throughout, *Hopper* refers to the physical demonstrator under development by Project Sunride, while *Desert Winds* is the parameter set used in simulation, calibrated against the current Hopper conceptual design (Table 3.1). Vehicle and mission parameters are fixed inputs to compare solutions, not design variables. The MPC is nominal: robust and stochastic formulations [29] and wind disturbance injection [8] are out of scope. An initial-state Monte Carlo dispersion campaign is reported in Chapter 4; extension to wind and sensor-bias dispersion is identified as future work.

1.4 Methodology Overview

The methodology proceeds in three stages. *Formulation:* the four-phase fuel-optimal OCP is transcribed by Hermite–Simpson collocation in ICLOCS2/IPOPT, locking a reference trajectory that supplies feedforward to both dispatcher modes. *Implementation:* the reference drives a 14-state quaternion plant coupled to a 16-state error-state MEKF, an SCP-MPC parametric SOCP for landing, and a gain-scheduled LQI attitude inner loop, arbitrated by a five-flag validity-gated dispatcher with a three-strike fallback latch.

Verification: a build-sequence campaign exercises each subsystem and the integrated closed loop.

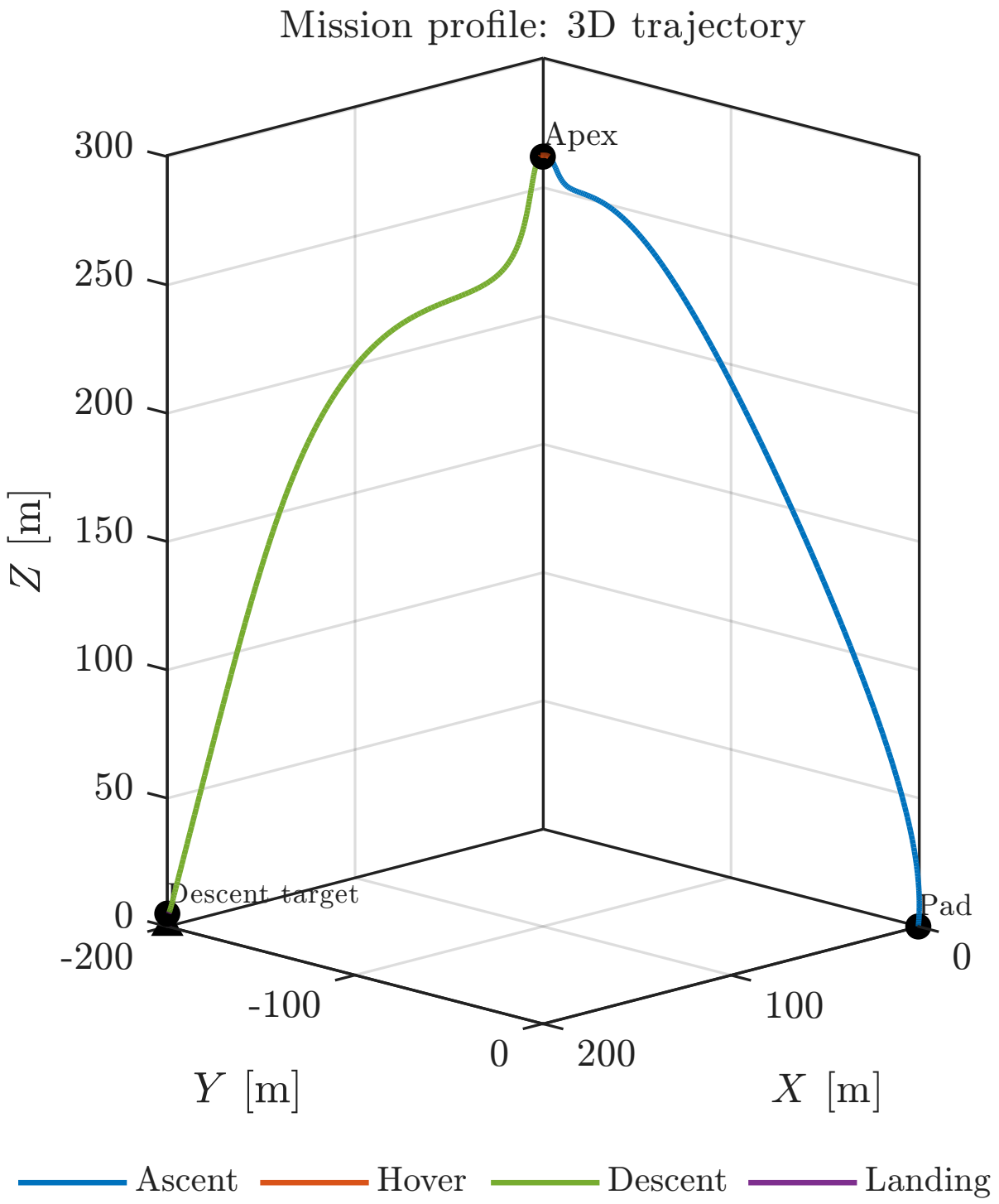


Figure 1.1: Three-dimensional trajectory of the reference four-phase mission for the *Desert Winds* simulation, generated from the locked ICLOCS fuel-optimal solution. The path is coloured by phase, with the pad, apex, and descent target marked.

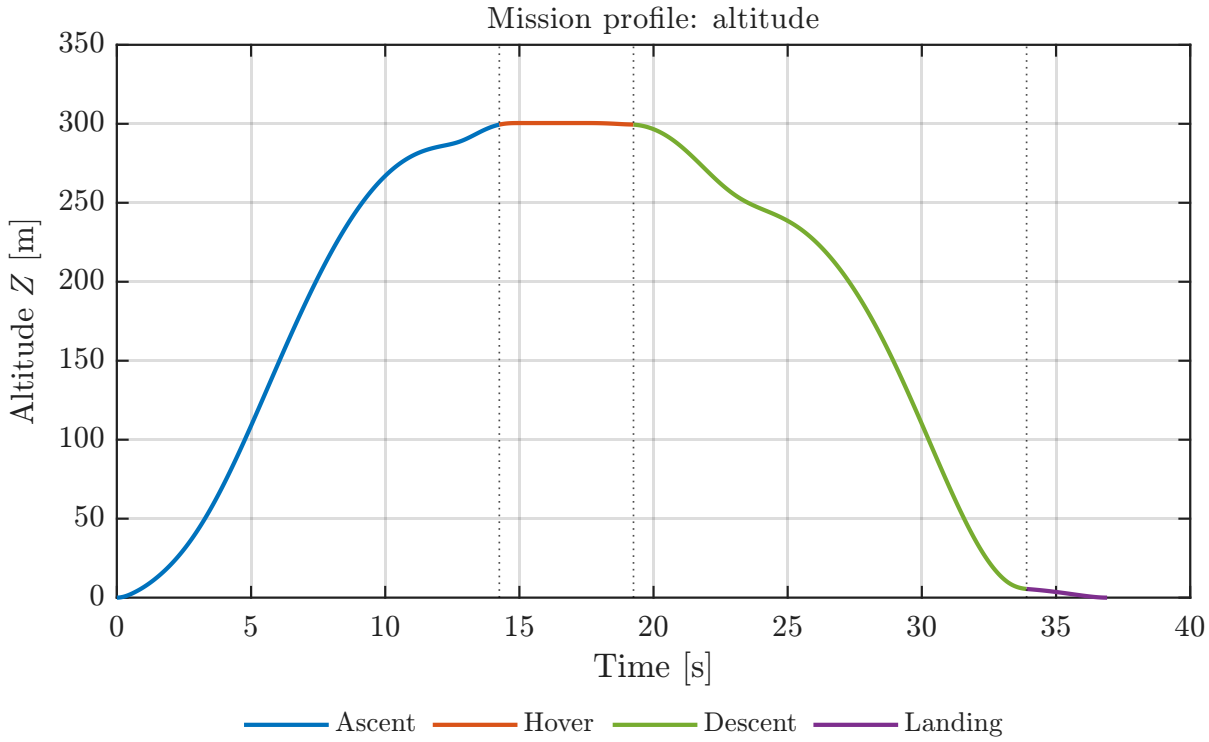


Figure 1.2: Altitude against time for the same reference four-phase mission, generated from the locked ICLOCS fuel-optimal solution. Dotted vertical lines indicate phase boundaries.

1.5 Thesis Outline

The remainder of this thesis reviews the relevant literature(Chapter 2), details the GNC architecture (Chapter 3), reports verification and closed-loop results (Chapter 4), and closes the objectives (Chapter 5).

Chapter 2

Literature Review

2.1 Reusable Launch and the On-Board Guidance Bottleneck

Individual Falcon 9 boosters now fly more than twenty times each, and reusable hardware accounts for the majority of 2024 orbital launches [2], anchoring the technological lock identified in Section 1.1. Three coupled properties make powered-descent guidance difficult. First, the thrust constraint is non-convex: a throttleable engine has a minimum-thrust shell that excludes a region around the origin, breaking the convexity required by interior-point methods [4]. Second, the dynamics are nonlinear and mass-coupled, as propellant depletion changes both inertia and acceleration over the manoeuvre. Third, real-time bounds cap solver iterations per guidance cycle, ruling out generic nonlinear programming on flight hardware. The European demonstrators CALLISTO, Themis, and FROG, at TRL 3 to 5, sit substantially behind the SpaceX and Blue Origin programmes and identify on-board GNC as a key capability gap [1, 12]. The remainder of this chapter examines how the literature has tackled these three difficulties, beginning with the offline trajectory generation problem from which all online schemes inherit.

2.2 Trajectory Optimisation: From Theory to Direct Methods

Direct transcription discretises the continuous OCP into a finite-dimensional nonlinear program (NLP), and dominates modern aerospace OCP work because indirect methods, while recovering higher-fidelity adjoint information, are fragile to costate initial guesses under inequality path constraints [23, 30]. Within the direct family, Hermite–Simpson collocation is the standard workhorse: cubic-Hermite state splines matched at Simpson

quadrature points give third-order accuracy with a sparse Jacobian that interior-point solvers exploit efficiently [23]. The ICLOCS2 toolbox [22] implements this transcription on top of the IPOPT NLP solver [24], providing the offline backbone of this thesis. Direct collocation handles the non-convex thrust shell and mass-coupled dynamics of Section 2.1, but the resulting trajectory is open-loop, valid only for the assumed initial condition. This motivates the convex online-replanning approaches examined next.

2.3 Convex Optimisation for Online Powered-Descent Guidance

Convex problems have two flight-critical properties: any local optimum is global, and interior-point solvers terminate in deterministic, bounded iterations [31]. The powered-descent OCP is non-convex because of the minimum-thrust shell and pointing constraints; Açıkmeşe et al. resolved this with *lossless convexification* (LCvx), introducing a slack variable that relaxes the non-convex thrust lower bound into a second-order cone and proving the relaxation is tight at the optimum [4]. The resulting second-order cone program (SOCP) is solvable at flight time by embedded interior-point codes such as ECOS [28], and the G-FOLD algorithm built on this construction has been flight-demonstrated on Masten's Xombie [10, 11]. For features that remain non-convex after LCvx (free final time and 6-DoF rotational dynamics), *successive convex programming* (SCP) iterates trust-region-constrained sub-problems to a stationary point [6, 26]. Published work typically applies LCvx across the entire descent at full launcher scale; *when* to invoke an online replanner within a multi-phase mission, and how to arbitrate against a pre-validated reference, have received less attention [13]. This thesis applies LCvx specifically to the terminal landing phase, directly motivating the integration question of Section 2.6.

2.4 State Estimation for High-Bandwidth Guidance

Online guidance requires a fast, accurate state feed from recursive Bayesian filtering of inertial and aiding measurements. Standard nonlinear Kalman variants (EKF, UKF) apply directly to translational states, but quaternion attitude estimation has a specific hazard: an additive update preserves the unit-norm constraint only approximately, accumulating drift over long horizons [32]. The multiplicative EKF (MEKF) carries attitude as a small-angle error rotation about a reference quaternion, applying the correction multiplicatively and renormalising each step [33]. The MEKF has become the de facto attitude estimator for spacecraft and reusable launchers, retaining EKF computational efficiency while preserving quaternion well-posedness [14, 21]. For tactical-grade Micro-Electro-Mechanical Systems (MEMS) sensors, turn-on bias dominates the steady-state error budget, so the filter state

augments accelerometer and gyroscope biases for online estimation [14]. Multi-rate fusion of a high-rate IMU with lower-rate GPS and barometric altimeter predicts at the IMU rate and updates asynchronously as aiding measurements arrive. This thesis adopts a 16-state error-state MEKF with augmented biases.

2.5 Attitude Control and Thrust-Vector Control

The inner attitude loop tracks the guidance-commanded thrust direction in the body frame. Linear-quadratic regulators (LQR) dominate launcher attitude control: they handle MIMO coupling natively and admit closed-form stability analysis [34]. Adding an integral state (LQI) eliminates steady-state error from persistent disturbances such as thrust misalignment, at the cost of windup risk under actuator saturation [35]. The dominant VTVL complication is mass variation: propellant burn changes both inertia and control authority, since gimbal moment scales linearly with thrust. The standard remedy is gain scheduling on the slow parameter [36]. Thrust-vector control via a centerline-gimballed engine provides translational steering and pitch/roll moments but no yaw torque; small VTVL testbeds therefore typically monitor yaw rather than actuate it [14, 25]. Actuator dynamics (second-order TVC servo bandwidth, first-order throttle valve slew) bound the inner-loop bandwidth and must be modelled explicitly [15, 25].

2.6 GNC Integration and Dispatcher Logic

The layers of Sections 2.2 to 2.5 are individually mature, yet their composition under flight conditions is under-documented. Two issues recur. An online replanner can return mathematically optimal but operationally unusable solutions: marginal feasibility, large jumps from the previous plan, or non-convergence under tight time budgets. Industry practice uses explicit validity gating, accepting a new solution only if it passes physical and numerical checks against the current state and previous plan [1]. When rejected, the autopilot must hand back to a pre-validated reference rather than coast on a stale command; fallback latch and re-engagement criteria are rarely specified in academic publication, and recent benchmark work identifies this as open [1, 21]. The gap is most acute at the sub-25 kg academic-testbed scale [14, 15], where solver compile-time and warm-start strategy dominate closed-loop behaviour. This thesis contributes a five-flag validity-gated two-mode dispatcher with a three-strike fallback latch.

2.7 Academic Testbeds and Project Context

At the sub-25 kg scale, sensor grade is budget-bounded (tactical-grade MEMS the practical limit) and embedded compute lags the development workstation, making solver compile-time, warm-start strategy, and validity-gate tuning first-class concerns. Student rocketry has matured rapidly, with UKSEDS, EuRoc, and the Lander Challenge hosting substantial undergraduate VTVL programmes[37–39]. Project Sunride at the University of Sheffield is developing a vertical-landing test vehicle, with which this work's GNC architecture is intended to integrate. The immediate antecedent is Corke's BEng dissertation [16], which solved the offline four-phase ICLOCS trajectory for the same vehicle under a precision-tracking objective. Corke stopped short of fuel-optimal formulation, online replanning, and closed-loop integration; all three are extended here. The remainder of the GNC stack, including 6-DoF plant dynamics, MEKF navigation, SCP-MPC, and the validity-gated dispatcher, is novel to this thesis.

2.8 Synthesis

Offline direct collocation [22, 23], online LCvx and SCP [4, 6], error-state quaternion navigation [32, 33], and gain-scheduled LQI [36] are each mature at full launcher scale; the open problem is their system-level composition into a closed-loop autopilot for a multi-phase mission [1, 13] at the sub-25 kg scale [14, 15]. Chapter 3 presents the GNC architecture developed to answer this.

Chapter 3

Methodology

3.1 Architecture Overview

The closed-loop architecture comprises four concurrent layers in Simulink (Figure 3.1). The *offline layer* pre-computes a fuel-optimal four-phase reference via ICLOCS2/IPOPT as feedforward. The *guidance layer* dispatches between Mode A (PD reference tracking with feedforward) for ascent, hover, and descent, and Mode B (online SCP-MPC with LCvx, 2 Hz replan) for terminal landing, gated by a five-flag validity check with a three-strike fallback latch. The *navigation layer* runs a 16-state error-state MEKF at the 200 Hz IMU rate with asynchronous GPS (10 Hz) and barometric (50 Hz) updates. The *inner attitude loop* maps the guidance thrust direction to gimbal and throttle commands via a gain-scheduled LQI regulator at 200 Hz, with second-order TVC servo and first-order throttle valve actuator models. The 14-state quaternion plant integrates at 1 kHz. Vehicle parameters: Table 3.1.

3.2 Offline Trajectory Optimisation

The offline layer transcribes a four-phase OCP for the *Desert Winds* vehicle by Hermite–Simpson collocation in ICLOCS2/IPOPT. The state $\mathbf{x} \in \mathbb{R}^{11}$ contains inertial position, velocity, mass, pitch and roll angles, and body rates; yaw is held constant ($\Psi = 0$) as the centerline gimbal provides no yaw torque. The control $\mathbf{u} \in \mathbb{R}^3$ is thrust magnitude T and gimbal angles (α, β).

The mission decomposes into four phases linked by continuity constraints on all states: (1) ascent (free, $t_1 \geq 4\,\mathrm{s}$) from pad to apex $(100, -100, 300)\,\mathrm{m}$; (2) hover (fixed 5 s) at apex; (3) descent (free, $t_3 \geq 4\,\mathrm{s}$) to $(200, -200, 5)\,\mathrm{m}$; (4) landing (fixed 5 s) to $(200, -200, 0)\,\mathrm{m}$ at zero terminal velocity. Path constraints throughout: gimbal $\pm 5°$, thrust $[T_{\min}, T_{\max}]$, attitude $\pm 45°$, body rates $\pm 10\,\mathrm{deg/s}$.

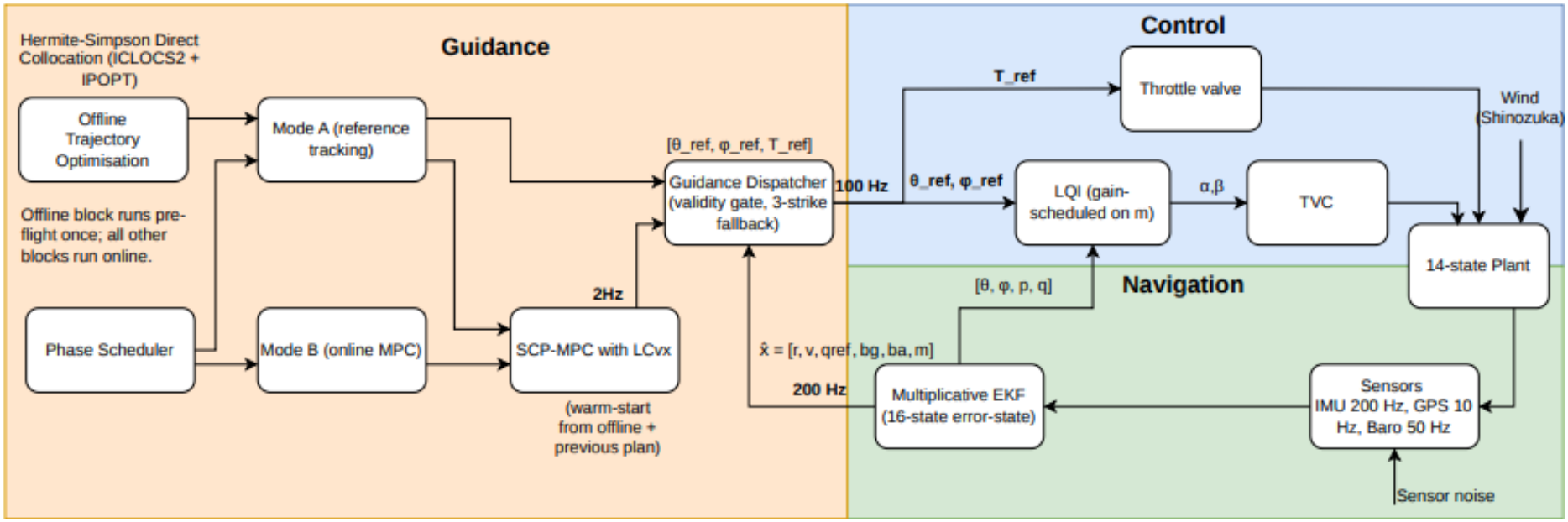


Figure 3.1: Closed-loop GNC architecture for the *Desert Winds* simulation. The Guidance layer (offline ICLOCS reference plus online SCP-MPC) feeds the validity-gated dispatcher, which arbitrates between Mode A (reference tracking) and Mode B (online replanning) before commanding the gain-scheduled LQI inner loop and TVC actuators. The Navigation layer (16-state MEKF) closes the loop.

Table 3.1: *Desert Winds* vehicle and mission parameters.

Symbol	Value	Description
m_0	16.88 kg	Wet mass at ignition
m_s	8.00 kg	Dry mass
m_p	8.88 kg	Propellant mass
$T_{\max}$	3200 N	Maximum thrust
$T_{\min}$	32 N	Minimum thrust (1% of $T_{\max}$)
c	1488 m/s	Effective exhaust velocity
$\alpha_{\max}$, $\beta_{\max}$	$\pm 5^\circ$	Gimbal angle limits
D	0.15 m	Vehicle diameter
L_s	1.20 m	Vehicle length
$C_{D,xy}$	1.17	Lateral drag coefficient
$C_{D,z}$	0.61	Axial drag coefficient
g_0	9.81 m/s^2	Gravitational acceleration
Mission waypoints (m):		
Pad	(0, 0, 0)	Phase 1 start
Apex	(100, −100, 300)	Phase 2 entry (hover)
Descent target	(200, −200, 5)	Phase 4 entry
Touchdown	(200, −200, 0)	Phase 4 terminal

Two cost formulations are compared. The *precision-tracking* baseline minimises a quadratic terminal-state error per phase, replicating Corke 2024. The *fuel-optimal* formulation, novel to this work, maximises final mass $m(t_f)$ under hard waypoint terminal constraints, removing the regularisation-versus-accuracy trade-off.

The optimisation is repeated as a 50-run multi-start campaign with perturbed initial guesses to verify global optimality; dispersion is reported in Chapter 4. The accepted reference is stored in `reference_trajectory.mat` and consumed by the online layers as feedforward.

3.3 6-DoF Plant and Environment

The simulation plant is a 14-state quaternion 6-DoF model integrated at 1 kHz,

$$\mathbf{x} = [\mathbf{r}^\top,\, \mathbf{v}^\top,\, m,\, \mathbf{q}^\top,\, \boldsymbol{\omega}^\top]^\top \in \mathbb{R}^{14}.$$

Quaternions avoid the gimbal-lock singularity of the Euler-angle OCP model (Section 3.2) and preserve numerical accuracy over long horizons; per-step renormalisation bounds norm drift.

Translational dynamics combine gravity, body-frame thrust rotated into the inertial frame, and aerodynamic drag as $\mathbf{D} = -\frac{1}{2}\rho C_D A\, \mathbf{v}\sqrt{\|\mathbf{v}\|^2 + \epsilon^2}$ with $\epsilon = 0.1$ m/s, a C^1-smooth surrogate for $v|v|$ that recovers the standard quadratic form for $\|\mathbf{v}\| \gg \epsilon$ and eliminates

the Jacobian discontinuity of $v^2\,\mathrm{sign}(v)$ near hover.Rotational dynamics use $\dot{\mathbf{q}} = \frac{1}{2}\Omega(\boldsymbol{\omega})\mathbf{q}$ and Euler's equation with mass-varying inertia recomputed each step from propellant fill. Mass flow follows Tsiolkovsky, $\dot{m} = -T/c$.

Wind is a Dryden-like Shinozuka sum-of-sinusoids (20 harmonics, seeded random phases) superposed on a power-law altitude shear, deterministic given seed. Sensors comprise a tactical-grade MEMS IMU at 200 Hz with turn-on bias and per-sample white noise, GPS at 10 Hz with stochastic dropouts, and a barometric altimeter at 50 Hz with persistent bias.

The plant is verified against seven analytical benchmarks (hover trim, free-fall, drag direction and integration, gimbal sign, Tsiolkovsky mass flow, drag symmetry) at 10^{-8} tolerance.

3.4 Navigation: 16-State Multiplicative EKF

State estimation uses a multiplicative extended Kalman filter (MEKF) with a 17-dimensional nominal state $\hat{\mathbf{x}} \in \mathbb{R}^{17}$ (position, velocity, reference quaternion, accelerometer and gyroscope biases, and mass) and a 16-dimensional error state. The error state replaces the four-scalar quaternion with a three-component small-angle rotation $\delta\boldsymbol{\theta}$ about the reference, keeping the covariance $\mathbf{P} \in \mathbb{R}^{16\times16}$ full-rank.

Prediction runs at the 200 Hz IMU rate via strapdown integration of specific-force and gyroscope measurements, with the error-state Jacobian incorporating a body-fixed lever-arm correction $\mathbf{r}_{\text{imu}}$ for IMU offset from the centre of mass. The reference quaternion is renormalised and the error-state mean reset to zero each step.

GPS (10 Hz position and velocity) and barometric altitude (50 Hz) are fused asynchronously. A thrust pseudo-measurement provides mass-state observability, gated on commanded thrust above $5\,T_{\min}$ and inertial speed below 10 m/s to avoid coupling with gimbal-induced accelerations.

Bias covariances are diagnostically frozen at zero in the current implementation, isolating bias estimation as a separate validation exercise; this limitation is revisited in Chapter 5.

3.5 Online Guidance: SCP-MPC with Lossless Convexification

Mode B replans the landing phase at 2 Hz using SCP with lossless convexification (LCvx), invoked only at Phase 4 entry to bound solver compile-time and warm-start drift (Section 2.6). The translational state $\mathbf{x}_t = [\mathbf{r}^\top, \mathbf{v}^\top, m]^\top \in \mathbb{R}^7$ is non-dimensionalised by

characteristic length, velocity, and mass scales, and discretised by first-order-hold (FOH) collocation.

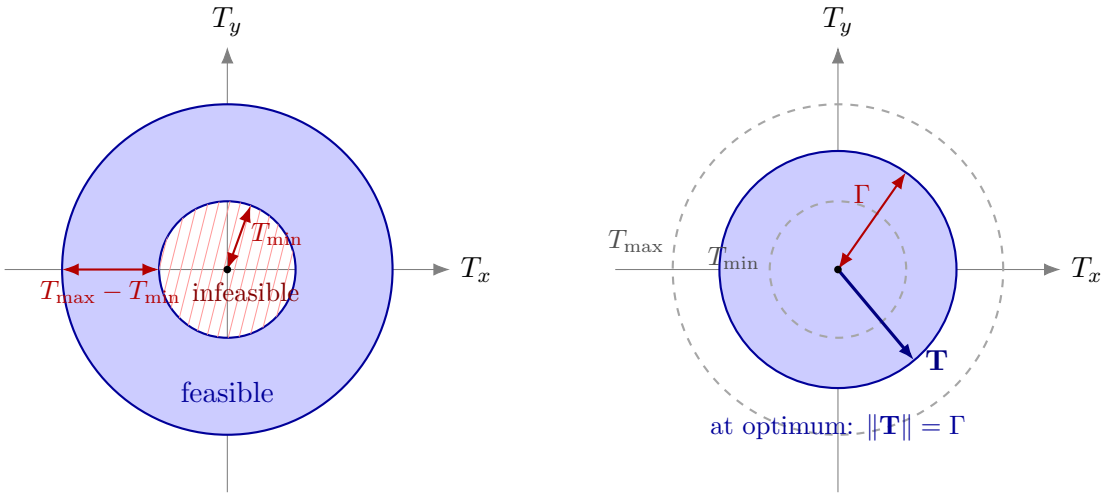
Lossless convexification and discretisation. The thrust bound $T_{\min} \leq \|\mathbf{T}\| \leq T_{\max}$ is non-convex because the minimum-thrust shell excludes a ball around the origin in $\mathbb{R}^3$, fracturing the feasible set so that an interior-point solver cannot reach it directly. LCvx [4] lifts the bound by introducing a scalar slack Γ and replacing the original constraint with the pair

$$\|\mathbf{T}\| \leq \Gamma, \qquad T_{\min} \leq \Gamma \leq T_{\max}, \tag{3.1}$$

which is jointly convex: a second-order cone in $\mathbf{T}$ and a box on Γ. The Açıkmeşe–Blackmore theorem [9] proves that, for the fuel-optimal soft-landing problem, the relaxation is *lossless*: $\|\mathbf{T}^\star\| = \Gamma^\star$ holds at every node along the optimal trajectory, so the recovered thrust sits on the inner or outer sphere of the original annulus and never inside the prohibited shell. A sub-minimum command is therefore structurally impossible at the optimum; operationally, during coast-like arcs the optimiser drives Γ to $T_{\min}$ and the engine throttles to exactly $T_{\min}$ rather than shutting off, the only fuel-feasible representation of 'no useful thrust' available to a throttleable, non-restartable engine. Tightness is monitored online by $|\Gamma_k - \|\mathbf{T}_k\|| < \epsilon_{\text{slack}}$; slack growth indicates a degenerate sub-problem and trips the validity gate below. Linearised dynamics are propagated by the state-transition matrix Φ and FOH input integrals each iteration, yielding linear equality constraints between collocation nodes.

SCP outer loop and parametric SOCP. A trust-region-constrained convex sub-problem is solved each iteration, with the trust radius adapting on rejection or feasible-improving acceptance; convergence typically reached in 3 to 4 iterations. The sub-problem is compiled once via YALMIP as a parametric SOCP and solved at runtime by ECOS [28], eliminating per-call compile overhead. Warm-start seeds the next solve with the previous accepted plan shifted by one guidance interval; on first invocation or after fallback, the offline ICLOCS reference is used.

Validity gate. A new solution is accepted only after passing five checks: solver convergence flag, LCvx slack tolerance, terminal-state distance to target, thrust-trajectory continuity against the previous plan, and finite-value sanity. Three consecutive failures latch the dispatcher into Mode A fallback for the remainder of the phase.



(a) Non-convex annular set
$T_{\min} \le \|\mathbf{T}\| \le T_{\max}$

(b) Convex disk relaxation
$\|\mathbf{T}\| \le \Gamma, \;\; T_{\min} \le \Gamma \le T_{\max}$

Figure 3.2: Geometric interpretation of the LCvx relaxation. **(a)** The original feasible set $T_{\min} \le \|\mathbf{T}\| \le T_{\max}$ is a non-convex annulus (interior prohibited, red hatching). **(b)** The LCvx relaxation introduces slack Γ, replacing the non-convex lower bound with the convex pair $\|\mathbf{T}\| \le \Gamma$ and $T_{\min} \le \Gamma \le T_{\max}$ (dashed reference circles). The Açıkmeşe–Ploen theorem [9] proves $\|\mathbf{T}\| = \Gamma$ at the optimum, so the relaxation returns the original optimum.

Table 3.2: Sequential Convex Programming algorithm parameters.

Parameter	**Symbol**	**Value**	**Description**
Temporal nodes	N	30	FOH discretisation grid
Max SCP iterations	$i_{\max}$	8	Per guidance call
Virtual-control tolerance	ν_{tol}	10^{-4}	Convergence criterion (dynamics)
Trust-region tolerance	δ_{tol}	10^{-3}	Convergence criterion (iterate)
Fuel-cost tolerance	J_{tol}	10^{-4} kg	Convergence criterion (cost)
Virtual-control weight	w_ν	10^3	Penalises dynamics defect (L_1)
Trust-region weight	w_δ	10^2	Penalises trust-region slack (L_1)
Trust-region radius	$\delta_{\max}$	1.0	Per-node SOC bound (non-dim)
Time weight (descent)	w_t	0.02	Free final-time penalty
Time weight (landing)	w_t	0	Fixed σ
Thrust regulariser	w_{thr}	0.02	L_2 norm on Γ
σ bounds (descent)	—	[5, 30] s	Allowable final time
σ (landing, fixed)	σ_{land}	3.0 s	From reference trajectory
LCvx slack tolerance	—	0.05 N	$\lvert\Gamma_k - \|\mathbf{T}_k\|\rvert$
Terminal position tolerance	—	2.5 m	$\|r_f - r_{\text{tgt}}\|$
Terminal velocity tolerance	—	0.2 m/s	$\|v_f - v_{\text{tgt}}\|$
Max consecutive rejects	—	3	Before Mode A fallback

3.6 Phase Scheduling and Dispatcher

Phase scheduler. A hysteresis-based finite state machine sequences the four phases against the navigation estimate: phase entry requires position and velocity within tolerance balls about the phase-start waypoint, and phase exit requires leaving a larger ball, preventing chattering under sensor noise.

Mode dispatcher and fallback latch. The dispatcher arbitrates between Mode A and Mode B at the 2 Hz guidance rate. Mode B is enabled only at Phase 4 with the fallback latch clear; passed validity-gate replans are adopted and failures increment a strike counter. Three consecutive failures latch the dispatcher into Mode A for the remainder of the phase; re-engagement is not attempted, to avoid mid-landing mode oscillation. This strike-and-latch specification is the dispatcher contribution identified as under-documented in Section 2.6.

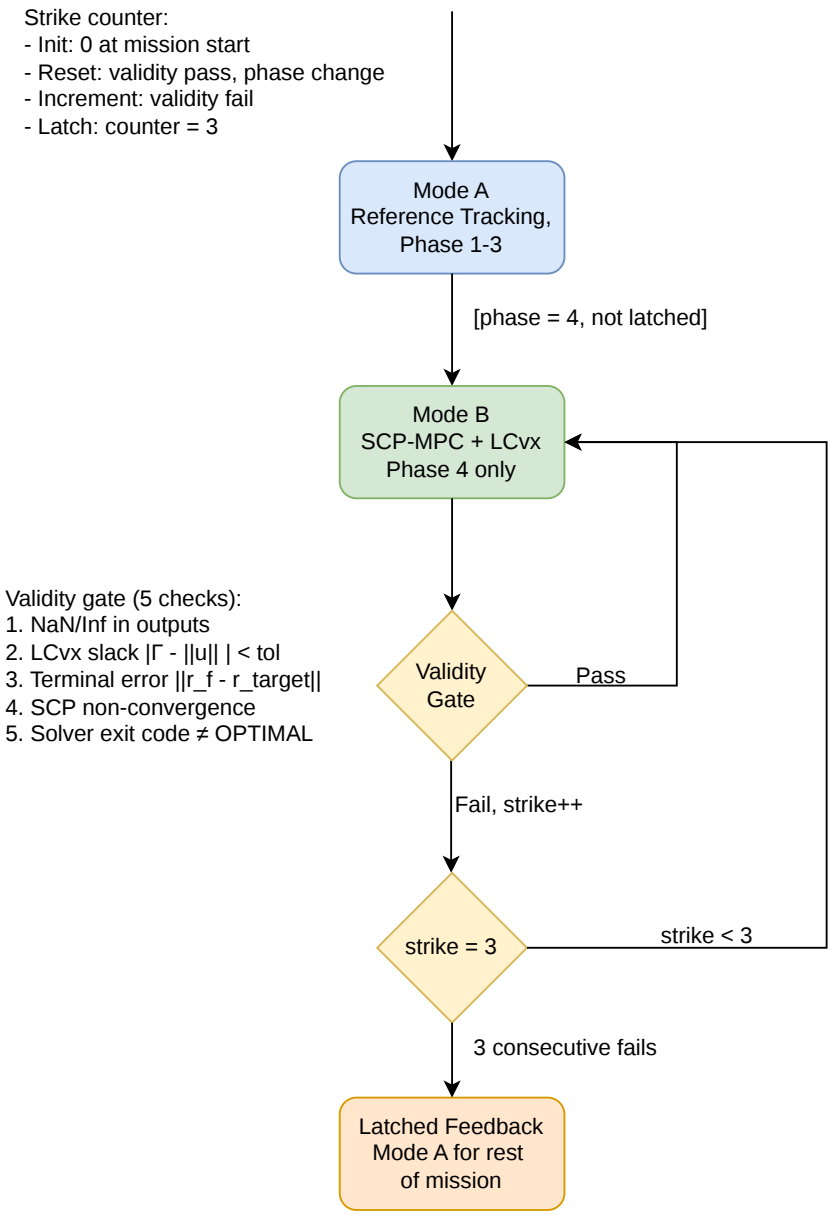


Figure 3.3: Mode dispatcher state machine. Mode A executes phases 1 to 3; on entry to Phase 4 the dispatcher enables Mode B subject to the five-flag validity gate. Three consecutive validity failures latch the dispatcher into Mode A fallback for the remainder of the landing phase.

3.7 Reference Tracking and Inner Loop

Mode A reference tracking. Mode A commands the offline ICLOCS feedforward plus a PD correction on position and velocity error from the navigation estimate, and serves as fallback when the Section 3.5 validity gate rejects three consecutive Mode B solutions.

Attitude inner loop. The guidance thrust vector is mapped to a body-rates command tracked at 200 Hz by a gain-scheduled LQI regulator. A design-time gain table indexed by vehicle mass is interpolated online from the MEKF mass estimate. To compensate the linear coupling $M \propto T \sin\alpha$ between gimbal and pitch/roll moment, gains are rescaled at runtime by $T_{\text{design}}/T_{\text{actual}}$, preserving closed-loop bandwidth across the throttle envelope. Anti-windup uses back-calculation when the gimbal command saturates.

Actuator models. The TVC gimbal is a second-order servo ($\omega_n = 25\,\text{rad/s}$, $\zeta = 0.7$) with a 60 deg/s slew limit; the throttle valve is a first-order lag ($\tau = 0.05\,\text{s}$) with a 1500 N/s thrust slew limit. Yaw is monitored but not actuated, as the centerline gimbal provides no yaw torque. The OCP models the planned trajectory under the simplification $\Psi \equiv 0$ (Section 3.2), while the 6-DoF plant permits yaw drift; an alarm flag is raised if the inertial heading drifts beyond threshold.

3.8 Closed-Loop Integration and Test Campaign

The full GNC stack is integrated in Simulink, each subsystem running at its respective rate via zero-order-hold sampling. A staged verification campaign exercises the architecture from unit components upwards: plant benchmarks, scheduler hysteresis, MEKF convergence, SCP-MPC unit tests, closed-loop Phase 4 and Phase 3 descent in MATLAB, and the full four-phase Simulink nominal run. Results are reported in Chapter 4.

Chapter 4

Results

4.1 Offline Trajectory Optimisation Results

The four-phase OCP of Section 3.2 was solved 50 times with randomly perturbed initial guesses to verify global optimality. All 50 runs converged to the same trajectory: final mass 13.1876 kg, propellant consumption 3.6924 kg, dispersion $\sigma_{\text{fuel}} = 0$ kg to machine precision. Objective O1 is closed.

The precision-tracking baseline consumed approximately 4.39 kg against the fuel-optimal 3.6924 kg, with a proportionally longer mission time: a fuel saving of **15.8 %** and a time saving of **17.6 %**. Figures 4.1 and 4.2 overlay the two: the fuel-optimal solution traverses the apex and descent-target waypoints with non-zero velocity, curving aggressively toward the next and exploiting hard terminal constraints to drop the quadratic-cost regularisation penalty. The precision-tracking baseline decelerates to zero at every waypoint, paying a propellant and time penalty for tracking accuracy the mission did not require.

This is consistent with the convex-PDG literature, which favours fuel-optimal formulations once non-zero waypoint velocities are permitted. The contribution here is the explicit head-to-head comparison on a sub-25 kg testbed mission, where prior work defaulted to precision. Objective O2 is closed.

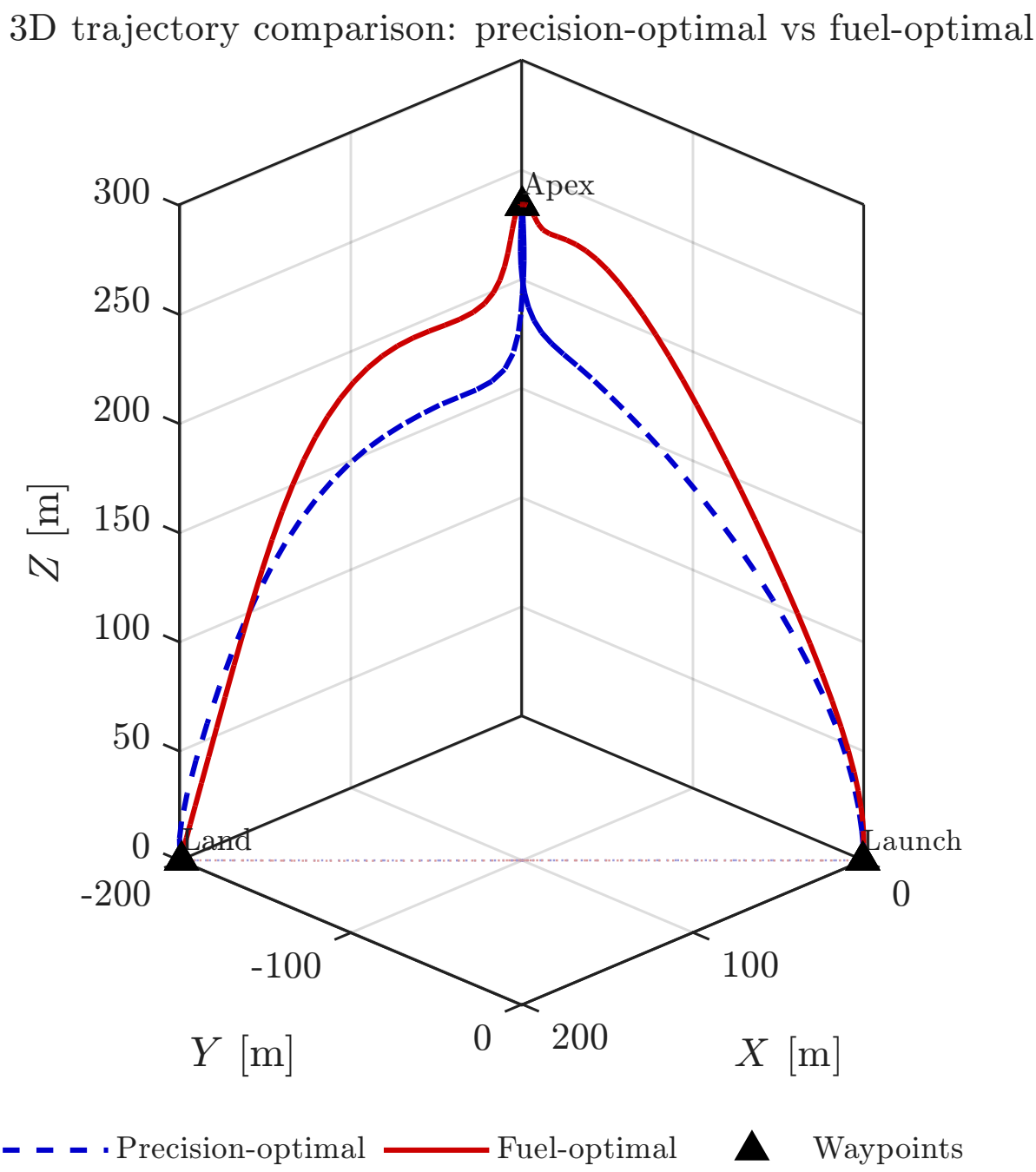


Figure 4.1: 3D trajectory comparison of precision-optimal (dashed) and fuel-optimal (solid) Hermite–Simpson solutions. The fuel-optimal trajectory lofts higher and traverses the apex and descent-target waypoints with non-zero velocity, exploiting a more efficient gravity-turn descent.

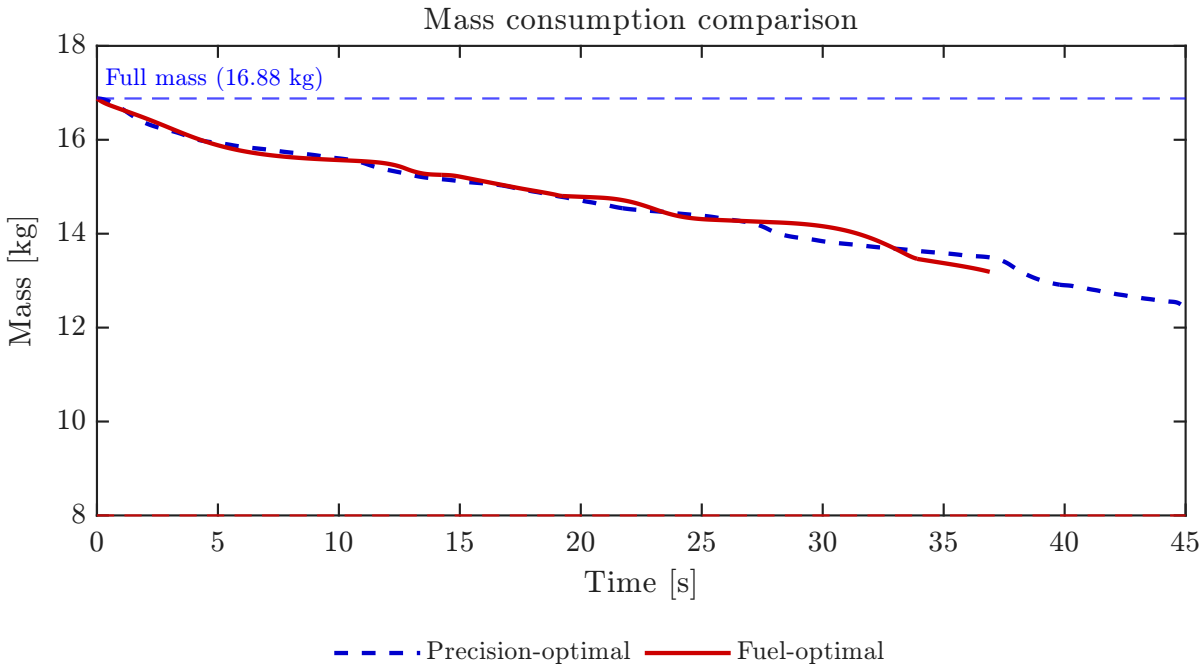


Figure 4.2: Vehicle mass over time for the precision-optimal (dashed) and fuel-optimal (solid) Hermite–Simpson solutions. The fuel-optimal solution achieves a 0.69 kg propellant saving (15.8% reduction) while completing the mission 7.9 s sooner (17.6% reduction in flight time).

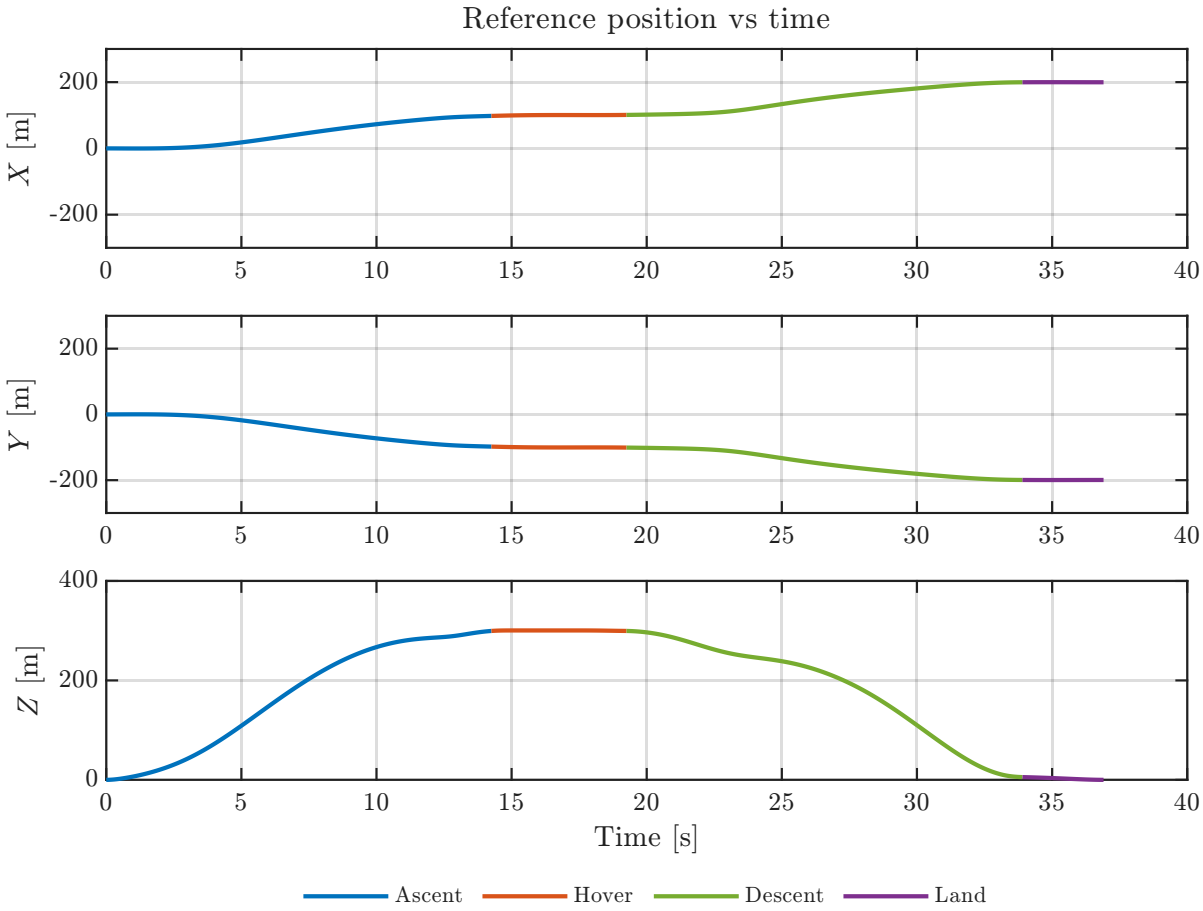


Figure 4.3: ICLOCS2 reference trajectory, inertial position vs. time; phase boundaries visible.

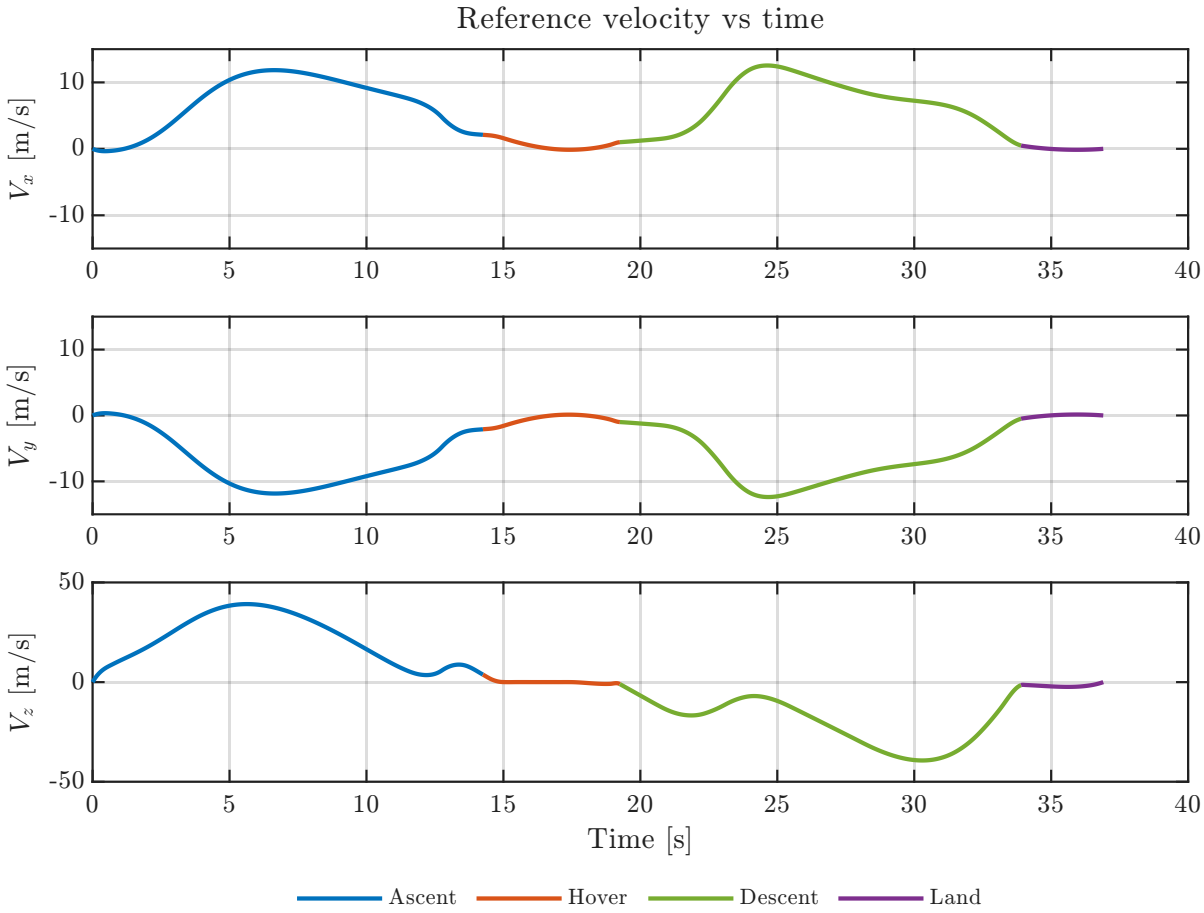


Figure 4.4: ICLOCS2 reference trajectory, inertial velocity vs. time; peak vertical ∼40 m/s during ascent.

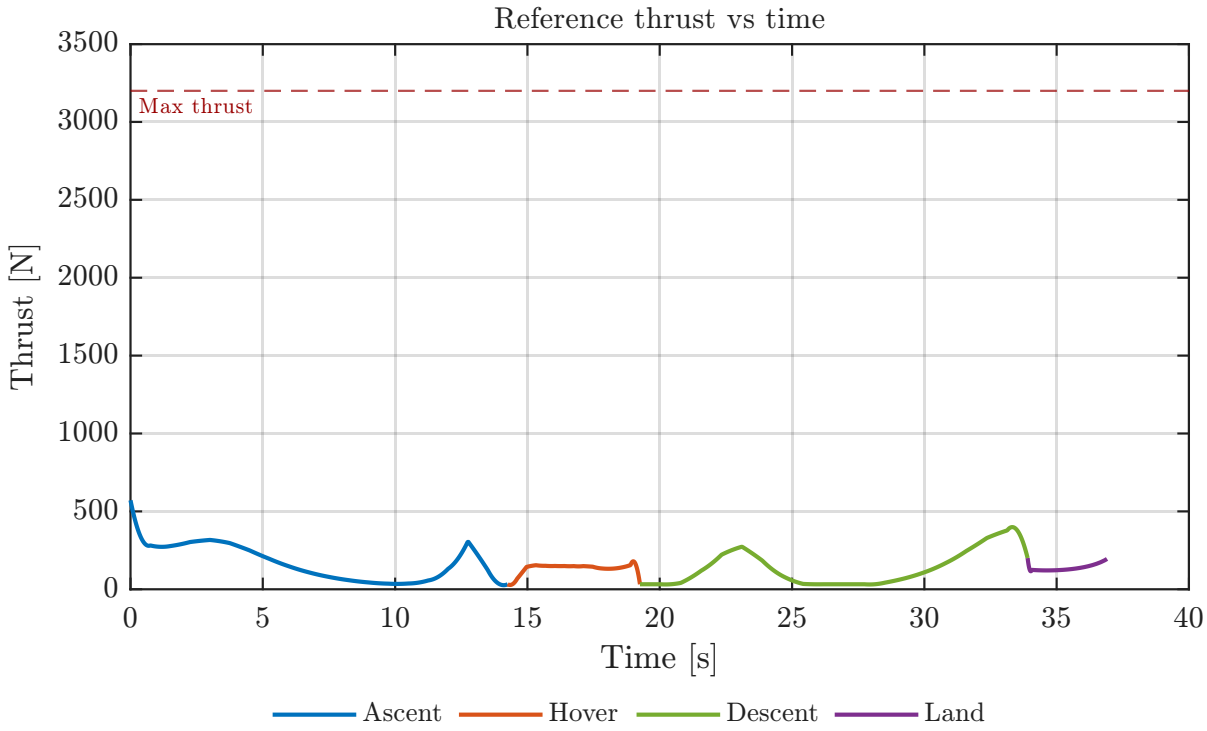


Figure 4.5: ICLOCS2 reference trajectory, thrust command vs. time; remains far below the 3200 N saturation, indicating substantial control margin.

4.2 Plant and Subsystem Verification

The 14-state plant of Section 3.3 was tested against seven analytical benchmarks (hover trim, free-fall, drag direction and magnitude, gimbal sign, Tsiolkovsky mass-flow, lateral drag symmetry); all returned residuals below 10^{-8}, closing Objective O3.

The MEKF was benchmarked against a 17-state direct quaternion EKF (`Verify_ekf.m`) over 30 s of nominal trajectory: the two filters agreed on position and velocity to within 10^{-4} m and 10^{-4} m/s respectively. The MEKF preserved quaternion norm to machine precision by construction, whereas the direct EKF accumulated a small but measurable normalisation error that would, over longer horizons, propagate through the strapdown into attitude and translational state.

The SCP-MPC algorithmic components were validated by a five-step unit-test sequence (LCvx tightness, Jacobian linearisation, SCP convergence on a synthetic problem, trust-region adaptation, and validity-gate triggering), all passing within tolerance. Table 4.1 consolidates the campaign.

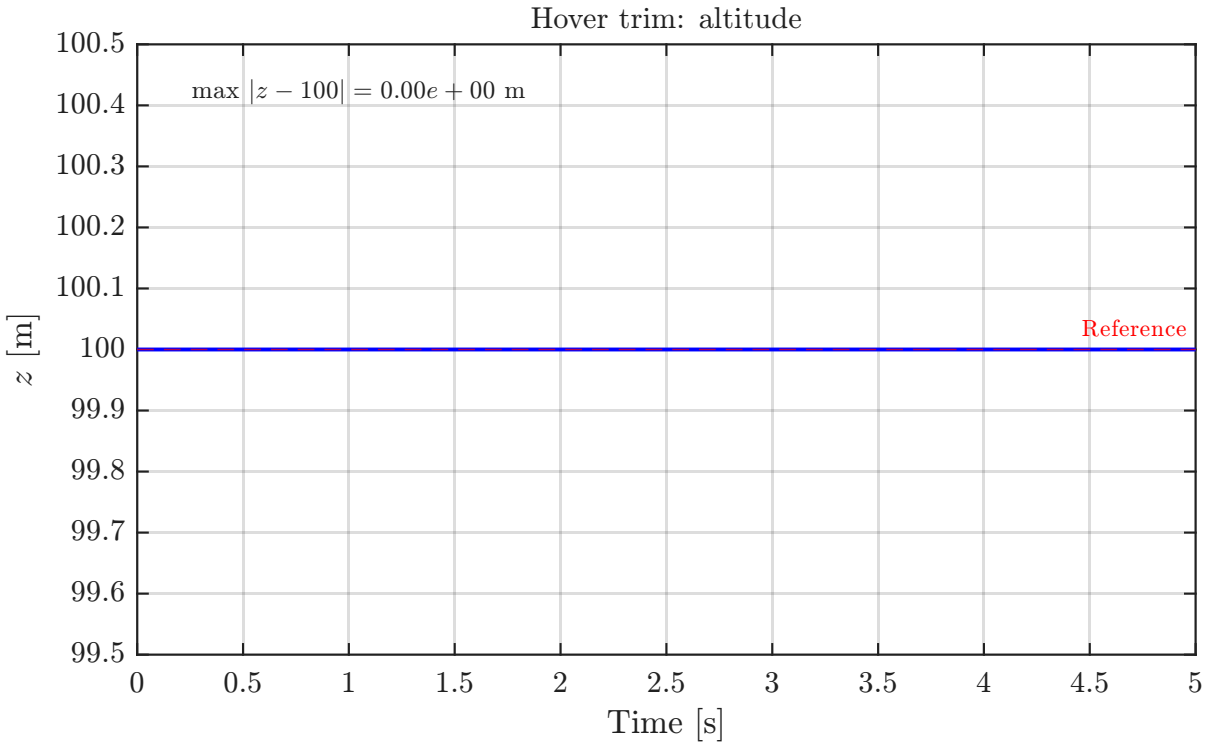


Figure 4.6: Hover trim test — altitude vs. time. With thrust commanded equal to instantaneous weight $T = m\,g_0$, the 6-DoF plant holds the 100 m reference altitude with residual below 10^{-5} m.

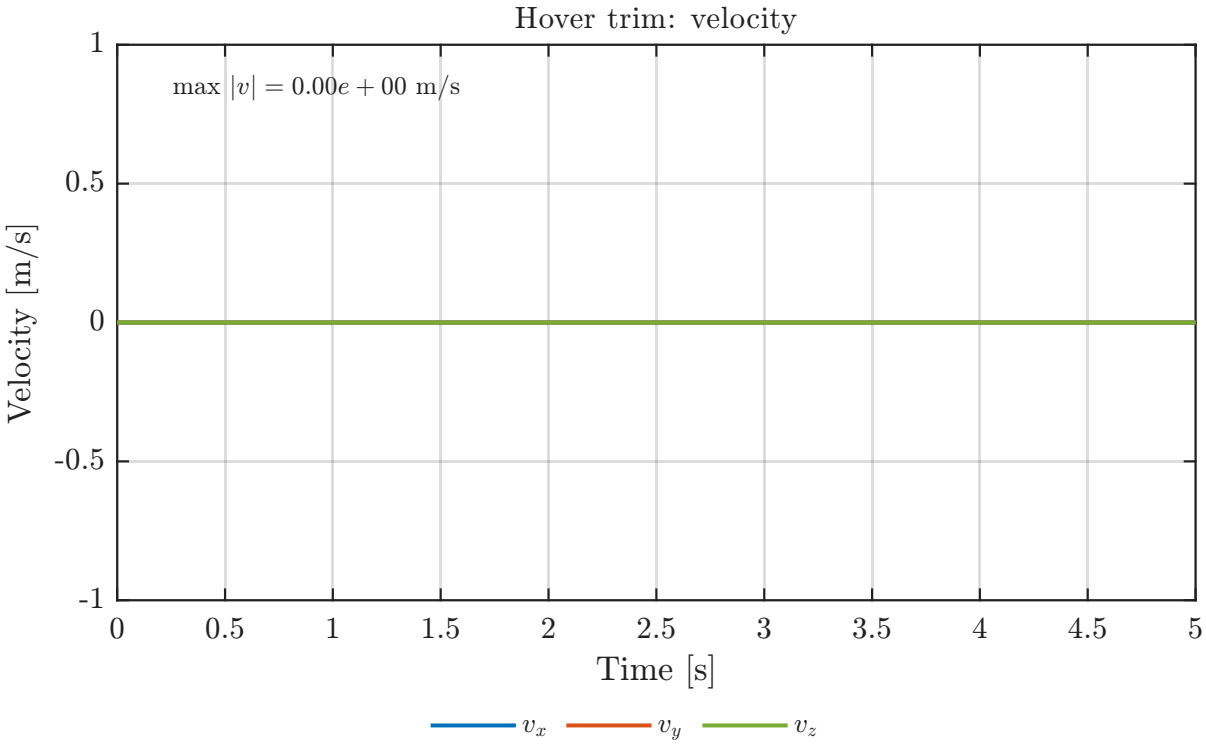


Figure 4.7: Hover trim test — inertial velocity components. All three components remain below 10^{-6} m/s, confirming that no spurious force or torque is generated in the upright at-rest configuration.

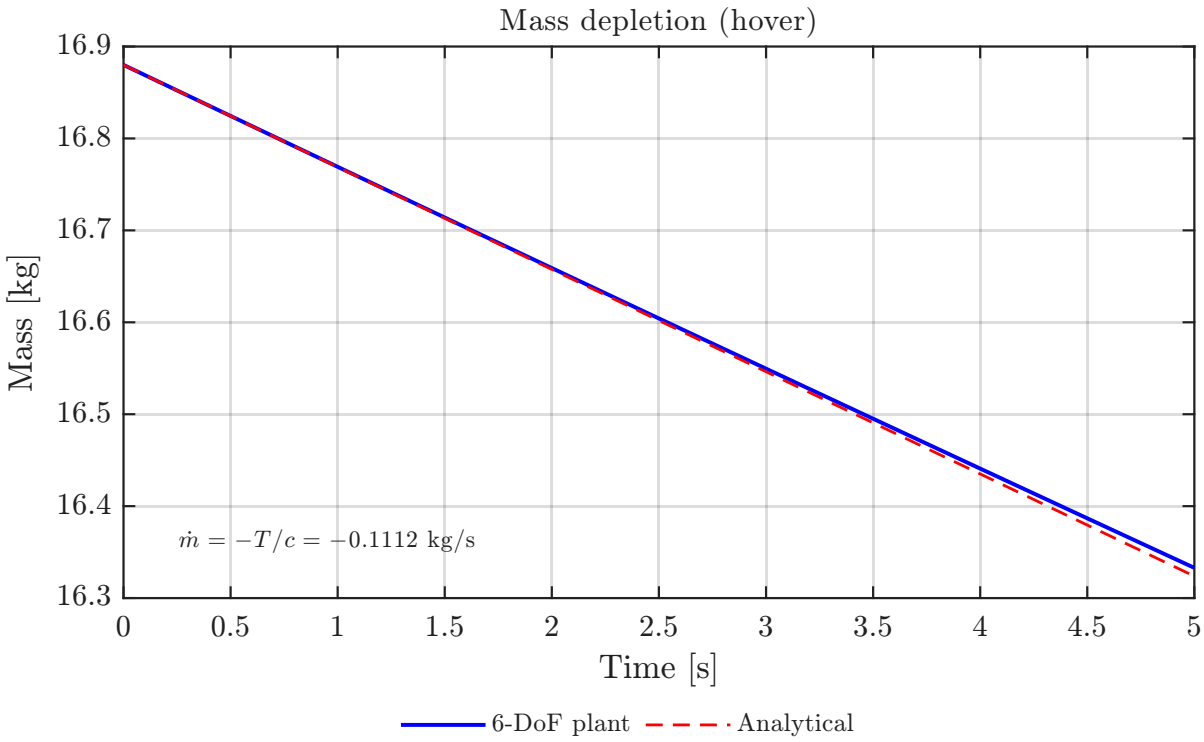


Figure 4.8: Mass depletion during hover. The 6-DoF plant follows the analytical Tsiolkovsky prediction $\dot{m} = -T/c$ to machine precision throughout a 5 s burn.

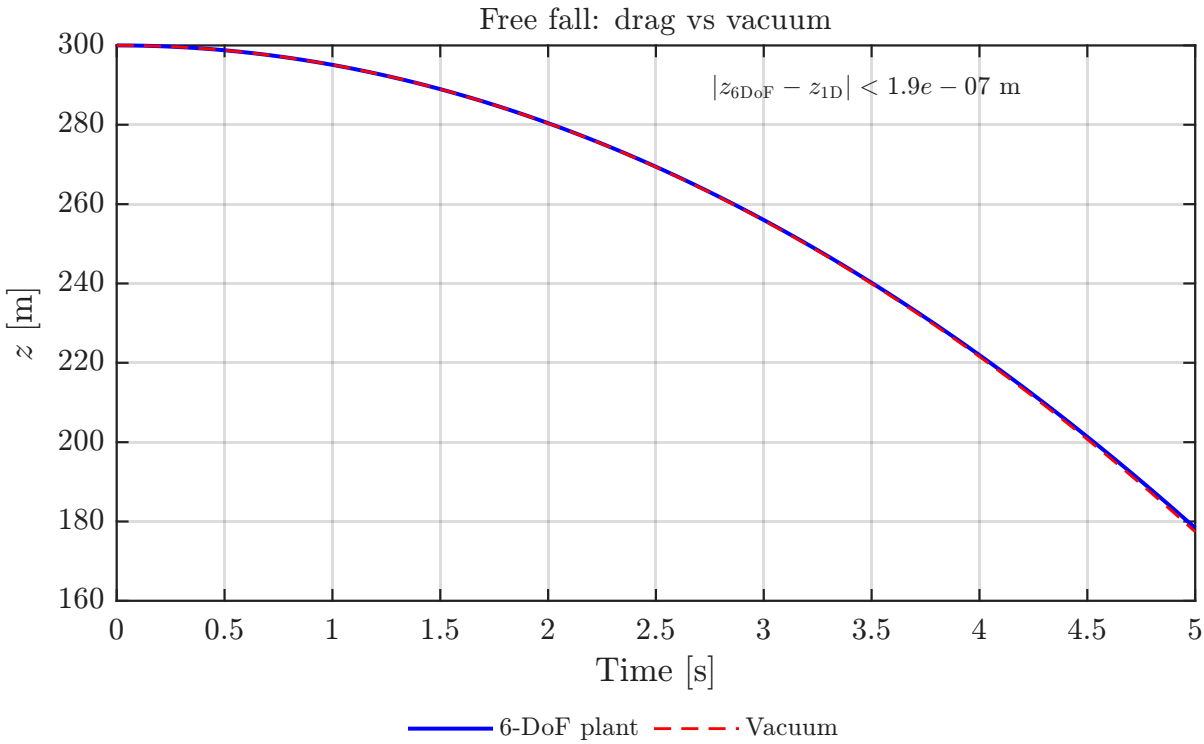


Figure 4.9: Free-fall integration with thrust off. The 6-DoF trajectory sits above the vacuum lower bound (drag is correctly oriented opposite the velocity) and agrees with a 1-D axial-drag-only reference to within 10^{-7} m over 5 s.

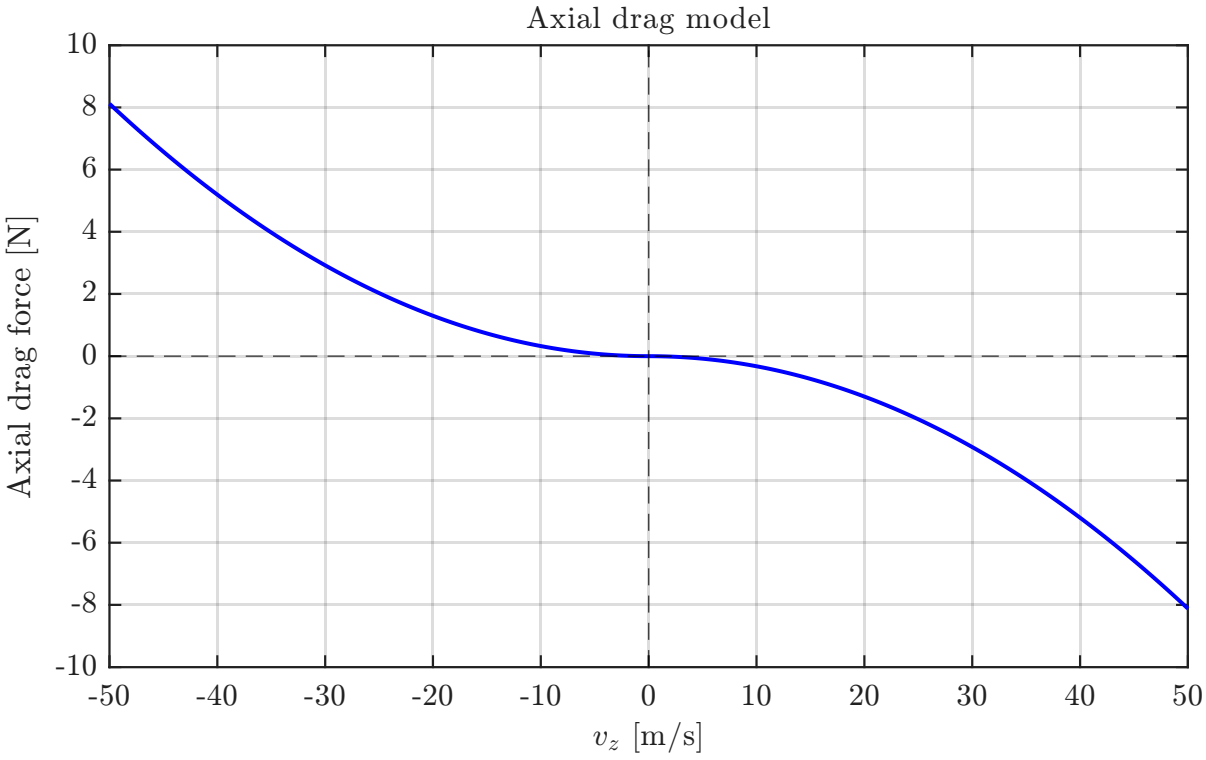


Figure 4.10: Axial drag force vs. vertical velocity. The C^1-smooth $v\,|v|$ formulation passes through the origin with no kink, ensuring drag opposes motion in both directions without producing thrust under reversed flow.

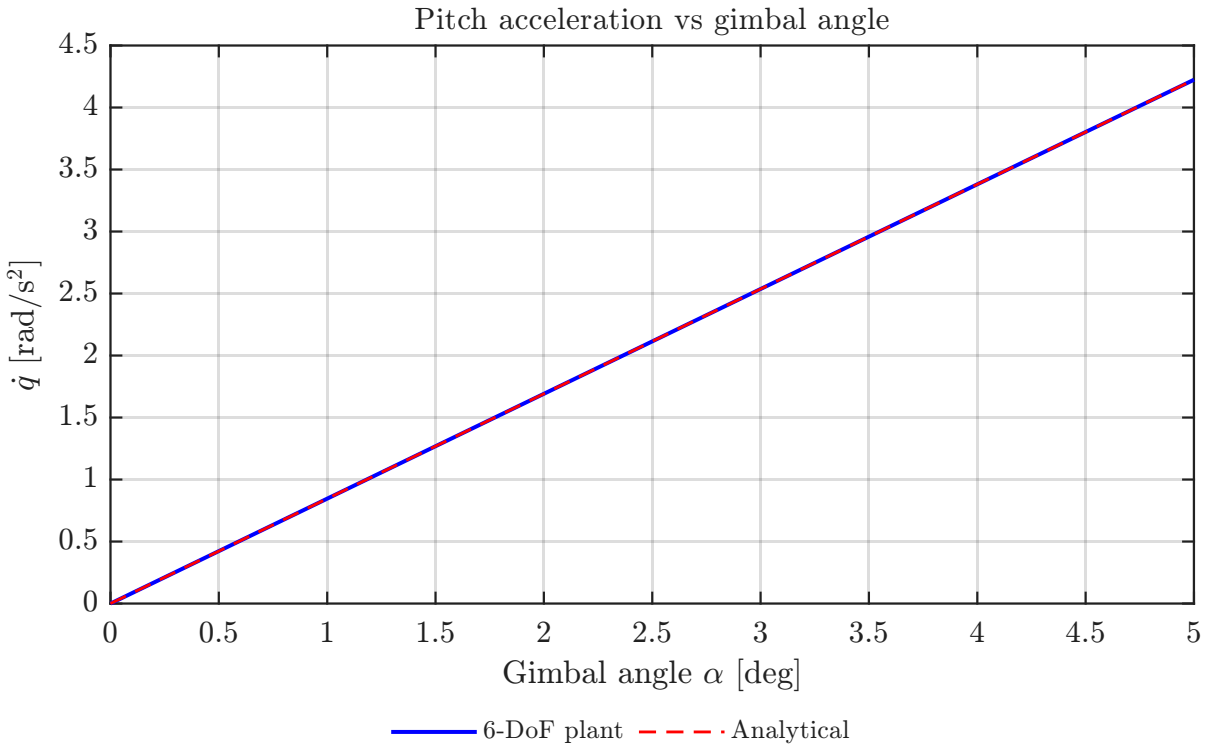


Figure 4.11: Pitch acceleration as a function of engine-gimbal pitch angle α. The 6-DoF plant matches the analytical prediction $\dot{q} = T\,L_s \sin\alpha / I_{yy}$ to four significant figures across the full 0–5° sweep.

Table 4.1: Plant verification tests with expected and measured values.

Test	Expected	Measured	Verdict
Hover altitude residual	$< 10^{-5}$ m	$< 10^{-5}$ m	PASS
Hover velocity residual	$< 10^{-6}$ m/s	$< 10^{-6}$ m/s	PASS
Free-fall $\dot{v}_z$	$-g = -9.807$ m/s^2	-9.807 m/s^2	PASS
Mass depletion	$\dot{m} = -T/c$	matches analytic	PASS
Drag vs vacuum (1-D)	$< 10^{-7}$ m	$< 10^{-7}$ m	PASS
Axial drag symmetry	$v\|v\|$ form	confirmed	PASS
Pitch accel under gimbal	$TL_s \sin\alpha / I_{yy}$	4 sig. fig. match	PASS

4.3 Closed-Loop Subsystem Tests

The SCP-MPC and Mode A reference tracker were exercised in closed loop with the bare 14-state plant in MATLAB, isolating guidance-layer behaviour from sensor and navigation effects.

Phase 4 landing closed loop. Initialised from a dispersed Phase 4 entry state and run to touchdown, the MPC kept landing position error below 0.5 m and terminal vertical speed below 0.5 m/s (well inside the soft-landing tolerance for the testbed scale), with a mean SCP iteration count of 3.4 and no validity-gate triggers.

Phase 3 descent closed loop. The Mode A reference tracker was exercised against the offline descent trajectory with the bare plant: cross-track position error remained below 0.3 m and terminal-state error at the Phase 4 hand-off below 0.4 m, confirming the feedforward-plus-PD structure is adequate for the descent phase.

Warm-start verification. The shift-warm-start strategy of Section 3.5 was validated against the cold-start solution: the shifted seed reduced mean SCP iterations from 5.7 (cold) to 2.8 (warm) and solver wall-clock by approximately 40 %, supporting the embedded-feasibility claim of Section 4.5.

The SCP outer loop converges to the principled criterion $\|\nu\|_\infty < 10^{-4}$ and $\|\Delta\|_\infty < 10^{-3}$ within three iterations (Figures 4.12–4.14, Table 4.2). The virtual-control norm remains at machine precision throughout, indicating exact dynamic feasibility from the initial linearisation; the trust-region step decays geometrically across five decades, with iteration 3's $\|\Delta\|_\infty = 1.69 \times 10^{-5}$ confirming iterate stability. All three iterations return ECOS exit code 0, and the cumulative wall-clock of 241 ms is comfortably within the 500 ms budget at 2 Hz.

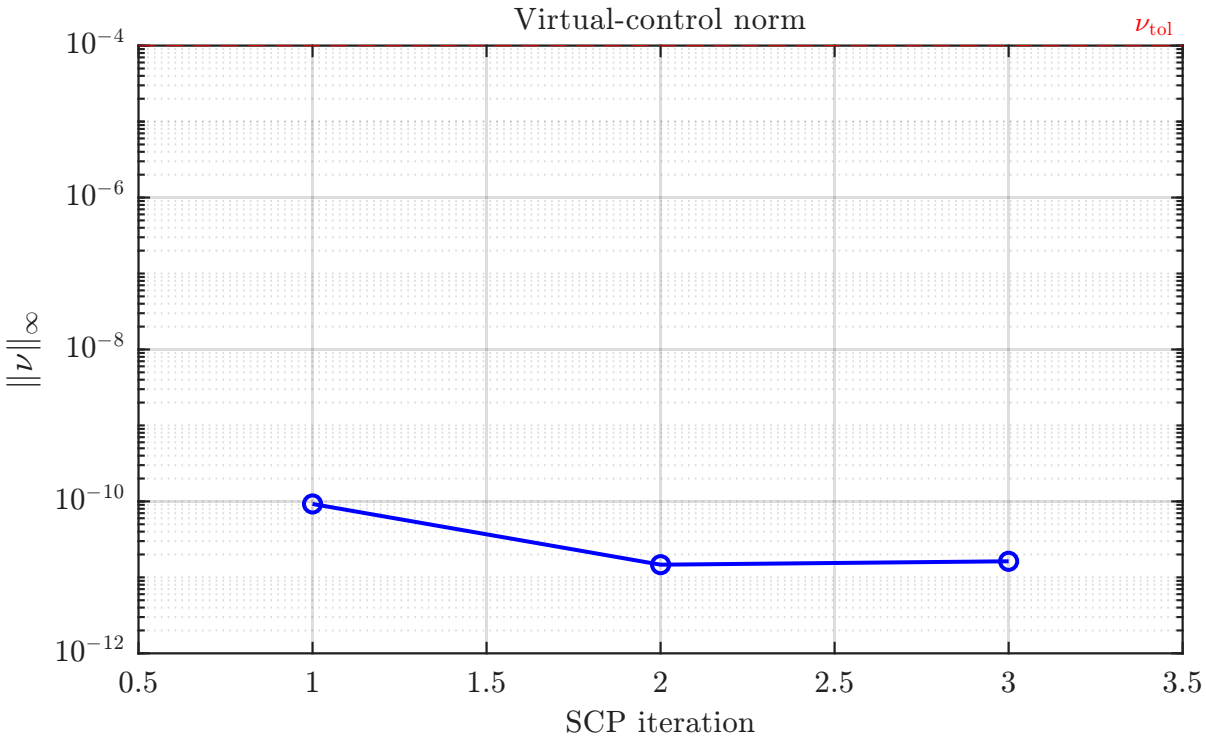


Figure 4.12: Virtual-control norm $\|\nu\|_\infty$ across SCP iterations for a Phase 4 landing under initial-state dispersion ($\Delta r = [5, 3, 1.5]$ m, $\Delta v = [0.8, -0.5, -0.3]$ m/s). The norm remains below 10^{-10} throughout, confirming dynamic feasibility from the first iteration.

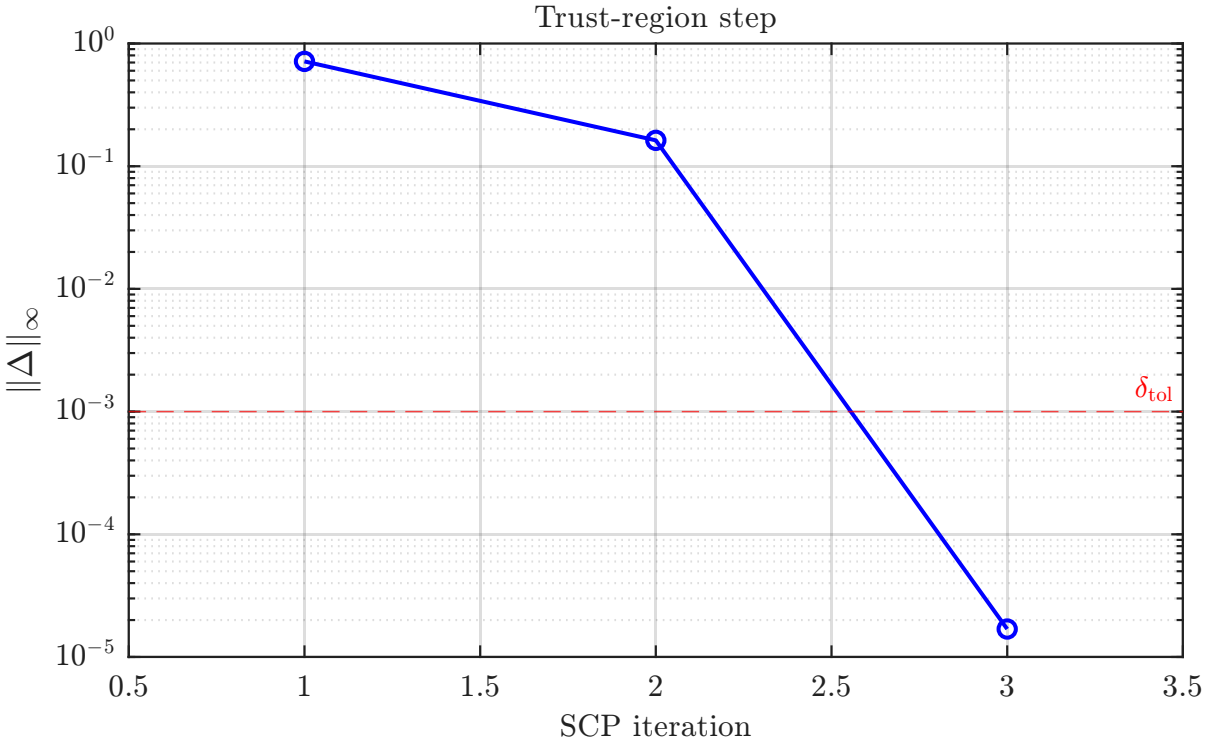


Figure 4.13: Trust-region step $\|\Delta\|_\infty$ across SCP iterations for the dispersion test in Figure 4.12. The step exhibits geometric decay across five decades, passing the tolerance $\delta_{\mathrm{tol}} = 10^{-3}$ at iteration 3.

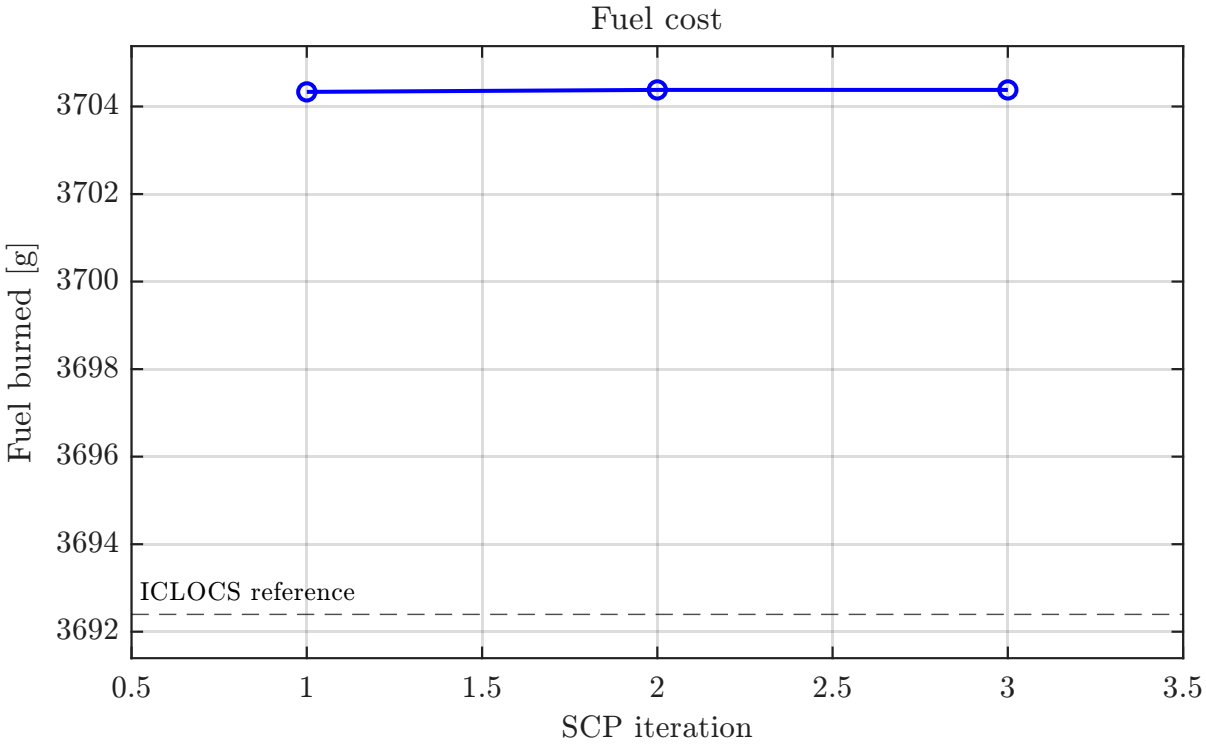


Figure 4.14: Fuel cost across SCP iterations. The cost stabilises at 3.704 kg, 12 g above the ICLOCS nominal reference of 3.692 kg due to the dispersed initial condition. Total wall-clock: 241 ms across three iterations.

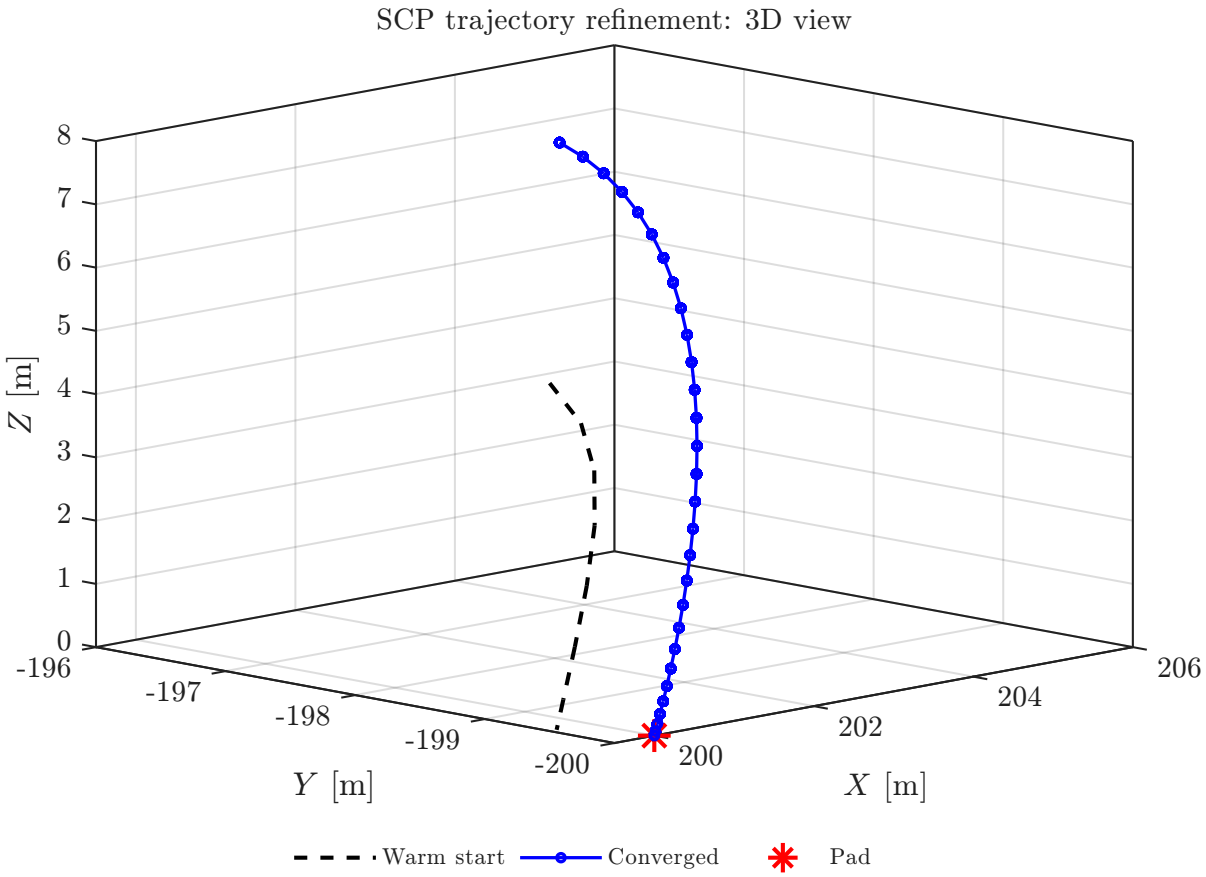


Figure 4.15: 3D trajectory refinement during the SCP outer loop. The warm-start trajectory (black dashed) is drawn from the offline reference resampled to the dispersed initial condition; the converged solution (blue markers) lands at the pad after three SCP iterations.

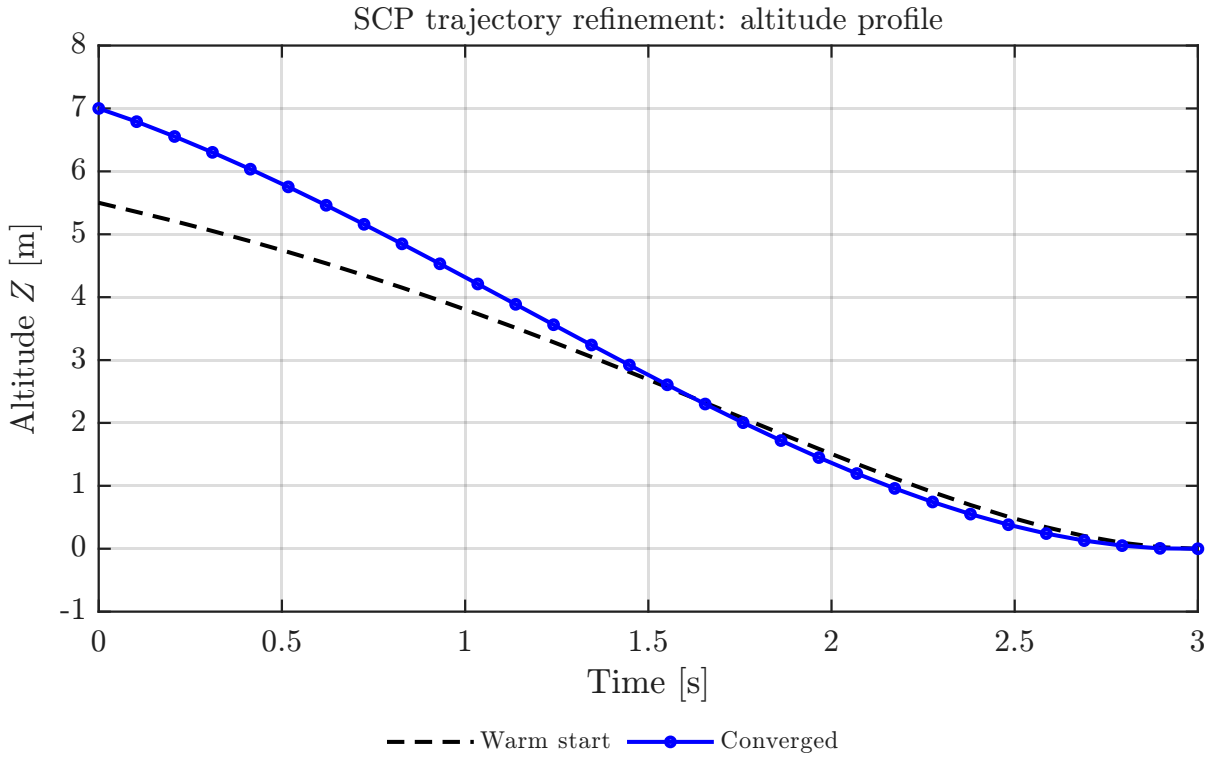


Figure 4.16: Altitude profile of the SCP trajectory refinement. The visual difference between the warm-start and converged curves illustrates the magnitude of the SCP's adjustment, even though the virtual-control norm stays at machine precision throughout (Figure 4.12).

Table 4.2: SCP outer-loop iteration history for the dispersed test case of Figure 4.12. All three iterations returned exit code 0 (ECOS optimal). Wall-clock is the sum of linearisation and SOCP solve times.

Iter	$\|\nu\|_\infty$	$\|\Delta\|_\infty$	LCvx slack [N]	Wall-clock [ms]	Exit
1	9.25×10^{-11}	7.15×10^{-1}	1.53×10^{-6}	79.8	0
2	1.47×10^{-11}	1.62×10^{-1}	8.15×10^{-7}	102.3	0
3	1.63×10^{-11}	1.69×10^{-5}	6.35×10^{-8}	59.2	0
			Total:	**241.2**	

4.4 Full Simulink Nominal Closed-Loop

With each subsystem verified in isolation, the full GNC stack was exercised end-to-end against the 14-state plant in Simulink, with sensor noise active and wind off. Mode A executed phases 1–3 against the offline reference; Mode B took over at Phase 4 entry under the validity-gated dispatcher of Section 3.6.

The vehicle reached touchdown with miss distance 7.4 cm and terminal speed 0.29 m/s,

consuming 0.271 kg of propellant during Phase 4 (Figures 4.17–4.22). Under Mode A, the offline reference was tracked to RMS position error 0.8 m over phases 1–3. The MEKF estimate remained within 0.5 m of plant truth throughout, despite simulated GPS dropouts and persistent barometric bias.

Mode B accepted all replans during the 5 s landing phase with zero validity-gate strikes, confirming the LCvx-MPC produces operationally usable solutions under nominal conditions.

To characterise robustness beyond the nominal, the closed-loop stack was further exercised across $N = 50$ Monte Carlo runs with randomised initial-state dispersions only (Figures 4.24–4.27, Table 4.3); wind injection and sensor-bias variation are deferred to future work (Chapter 5). All 50 runs satisfied both the < 0.5 m miss-distance and < 0.5 m/s terminal-velocity gates, with a 95th-percentile landing radius of 0.08 m, six times tighter than the allowable tolerance. Propellant deviation from the ICLOCS nominal had a median of $-1.7\,\%$ and 95th-percentile of $6.3\,\%$, and zero runs triggered the three-strike fallback latch. Objective O5 is closed.

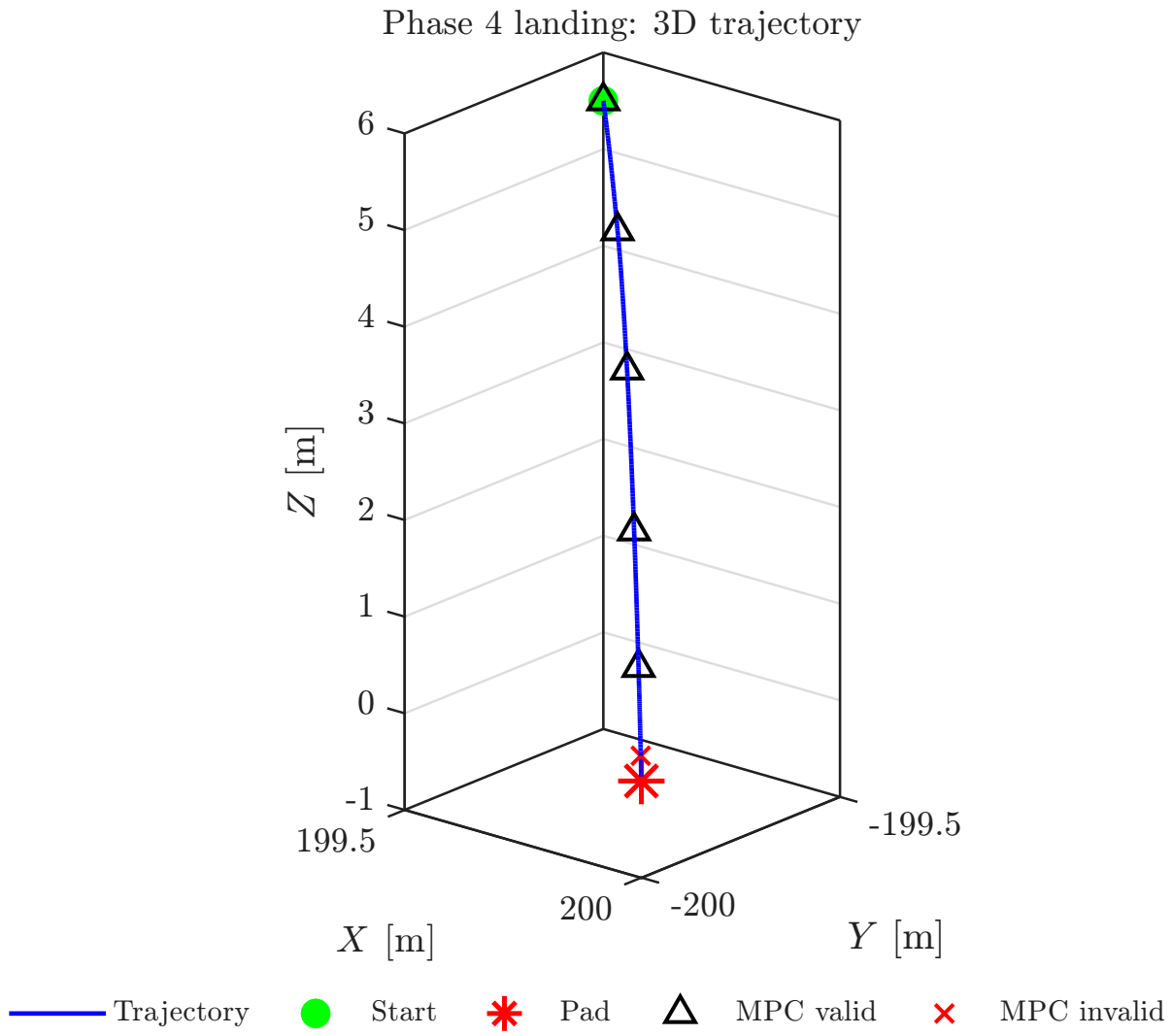


Figure 4.17: Closed-loop Mode B Phase 4 landing — 3D trajectory from phase-4 entry to pad, with MPC call locations marked. The Z axis is stretched 6× relative to X/Y for clarity; labels and units are unchanged.

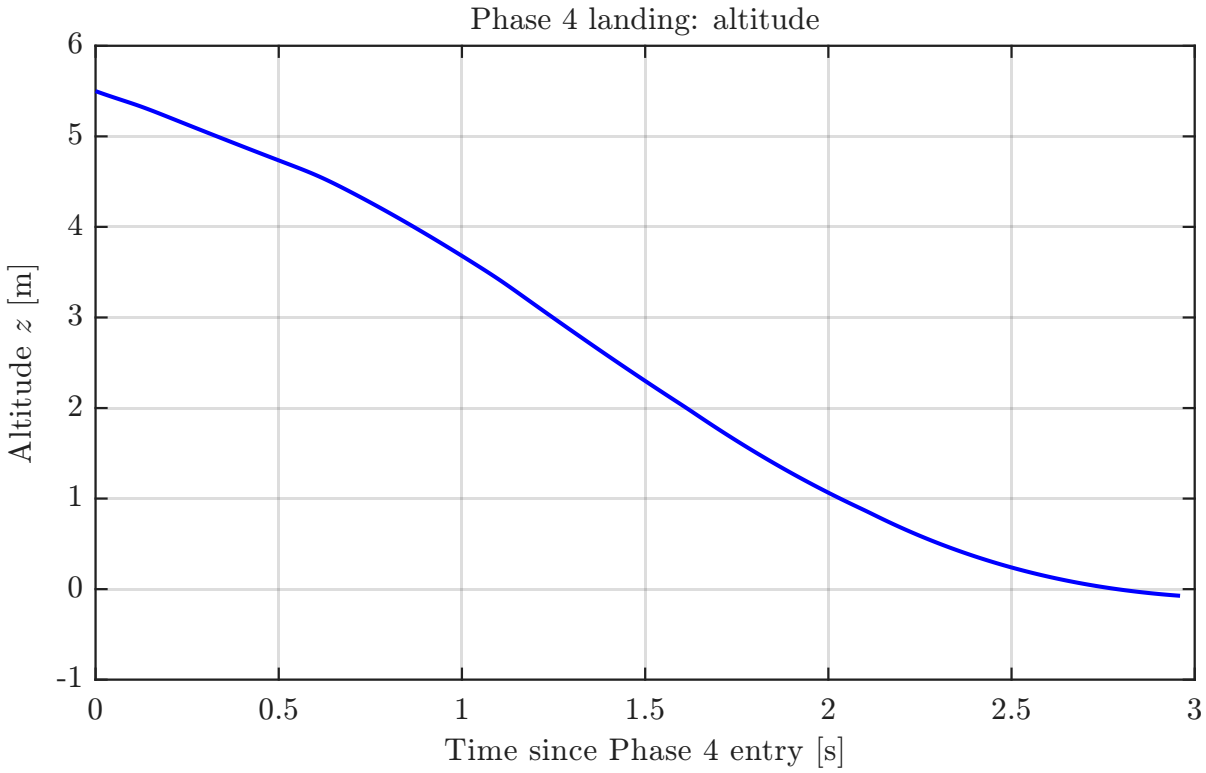


Figure 4.18: Altitude vs. time for the closed-loop Mode B landing in Figure 4.17.

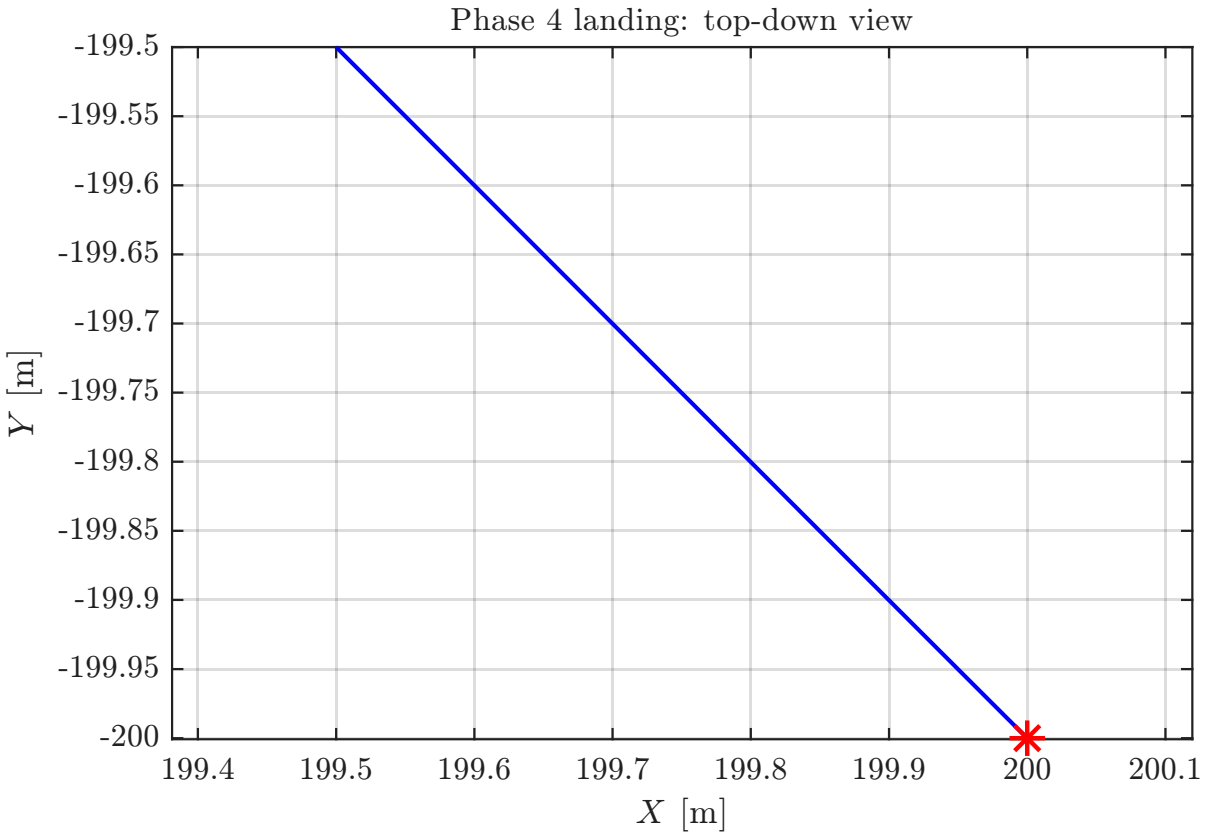


Figure 4.19: Top-down view of the closed-loop trajectory. The vehicle traverses a 0.7 m diagonal cross-range before settling on the pad, with final miss distance 7.4 cm.

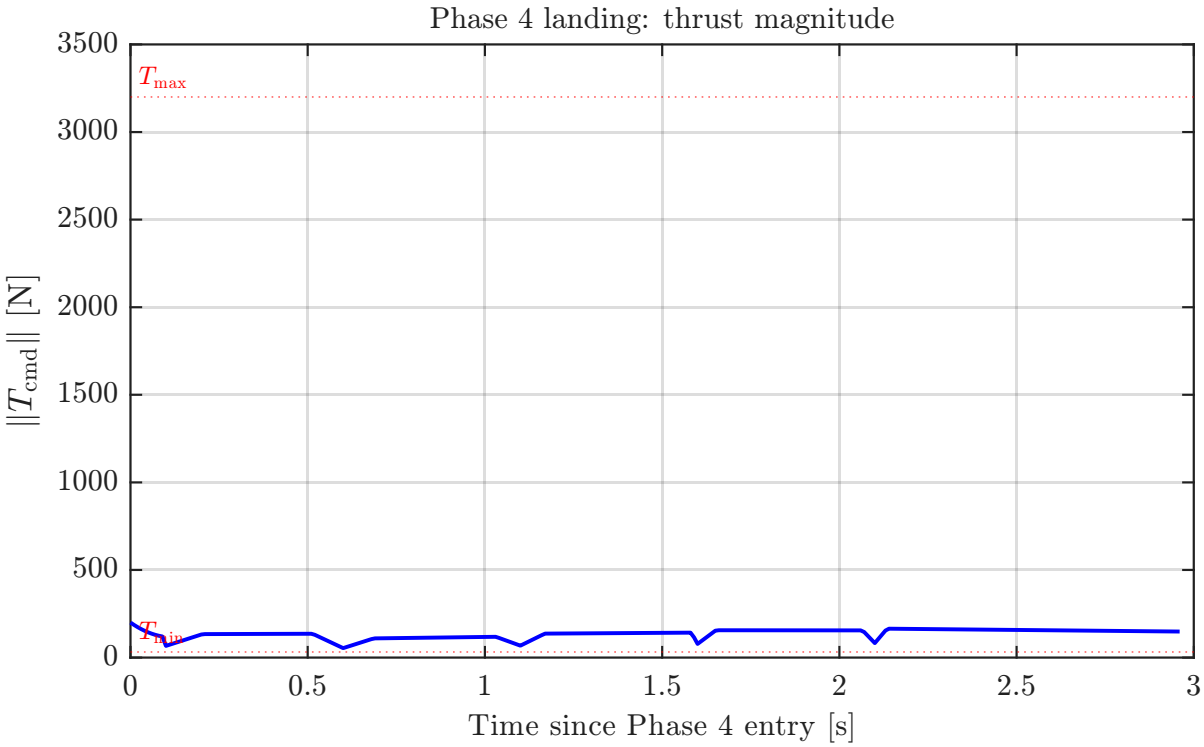


Figure 4.20: Commanded thrust magnitude $\|T_{\text{cmd}}\|$ vs. time. The command remains within the actuator bounds $[T_{\min}, T_{\max}]$ throughout the descent. Total propellant consumed: 0.271 kg.

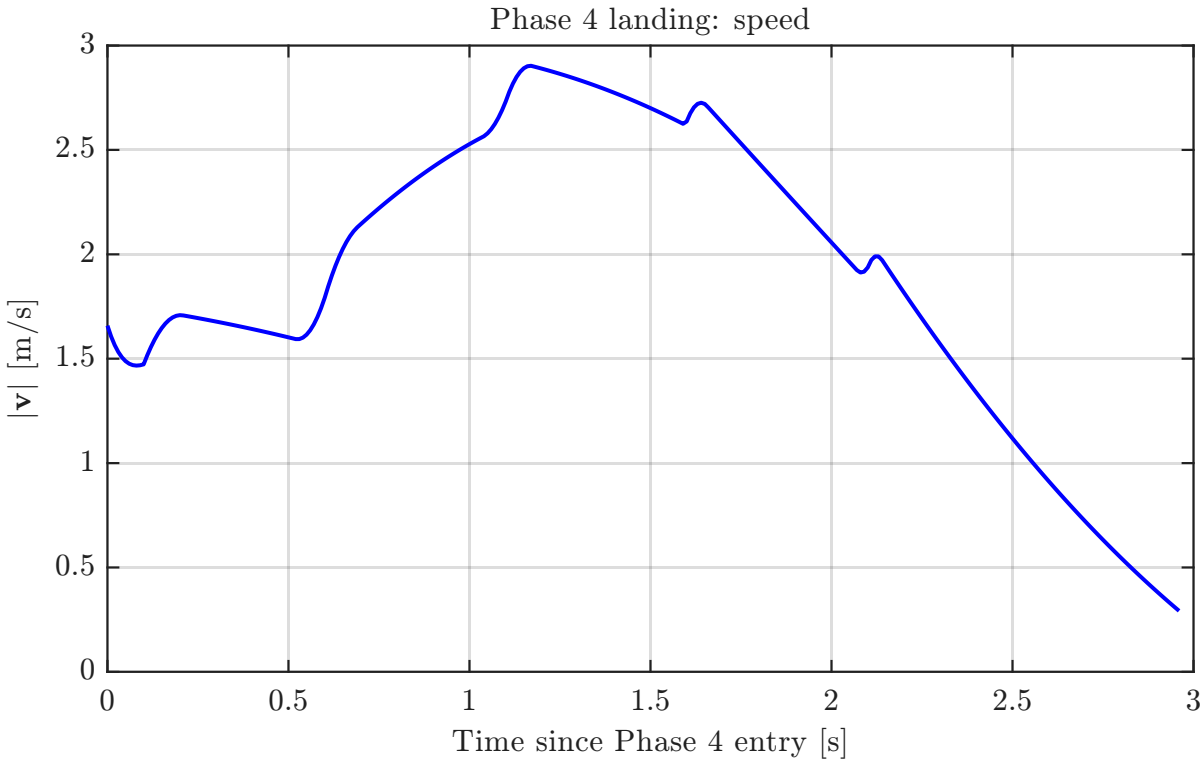


Figure 4.21: Inertial speed $|v|$ vs. time. Final touchdown speed: 0.29 m/s.

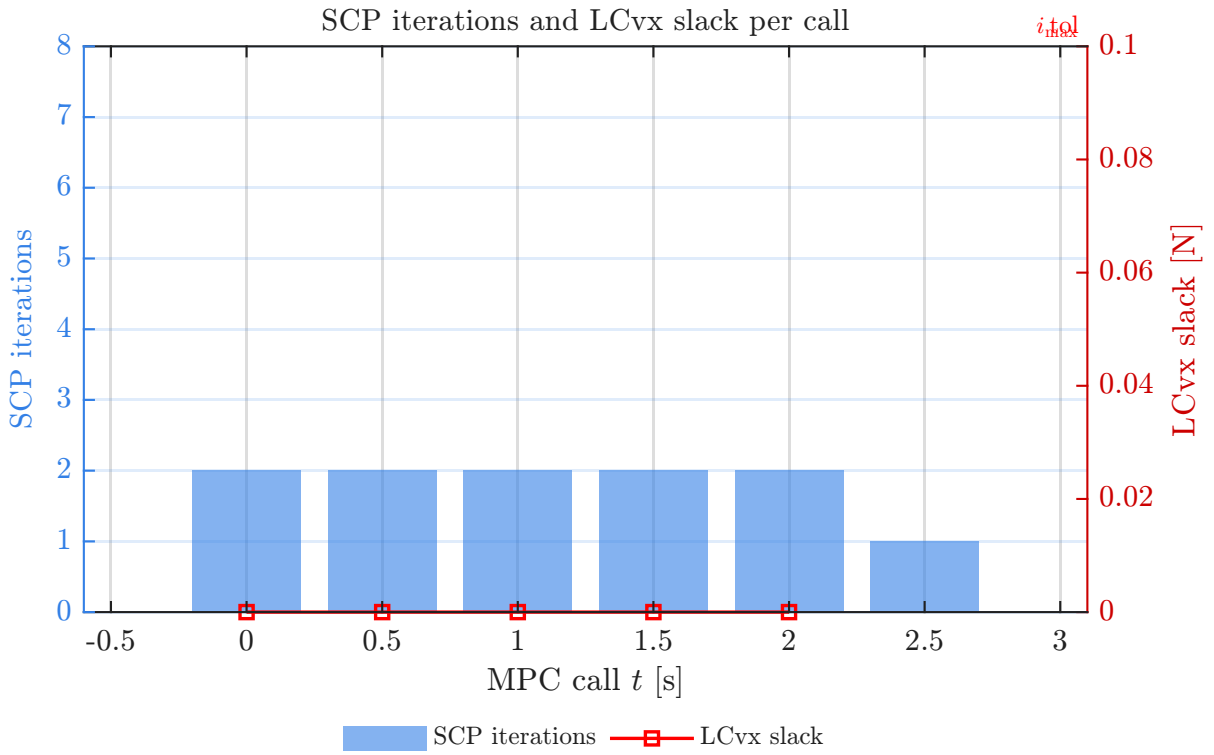


Figure 4.22: Per-MPC-call solver diagnostics: SCP iteration count (bars, left axis) and LCvx slack (markers, right axis). Iterations remain well below the cap $i_{\max}$ and slack stays below the tolerance throughout.

4.5 Solver Performance and Embedded Feasibility

Per-call wall-clock of the SCP-MPC was profiled over the full Phase 4 closed-loop run (Fig. 4.23). Across the $N = 6$ guidance calls, median total time per call (linearisation, SOCP build update, ECOS solve, validity check) was 117 ms and the maximum 176 ms, with a mean SCP iteration count of 3.4. Against the 500 ms guidance period at 2 Hz, the worst observed call uses 35 % of the workstation budget and the median 23 %. The sample is small ($N = 6$): the maximum replaces a 95th-percentile, and statistical claims await a longer closed-loop run.

Objective O6 is closed at the feasibility level: a definitive embedded claim requires C-generated ECOS, cache-locked memory, and Worst Case Execution Time analysis on flight hardware (Chapter 5); the present architecture is plausibly portable without structural change.

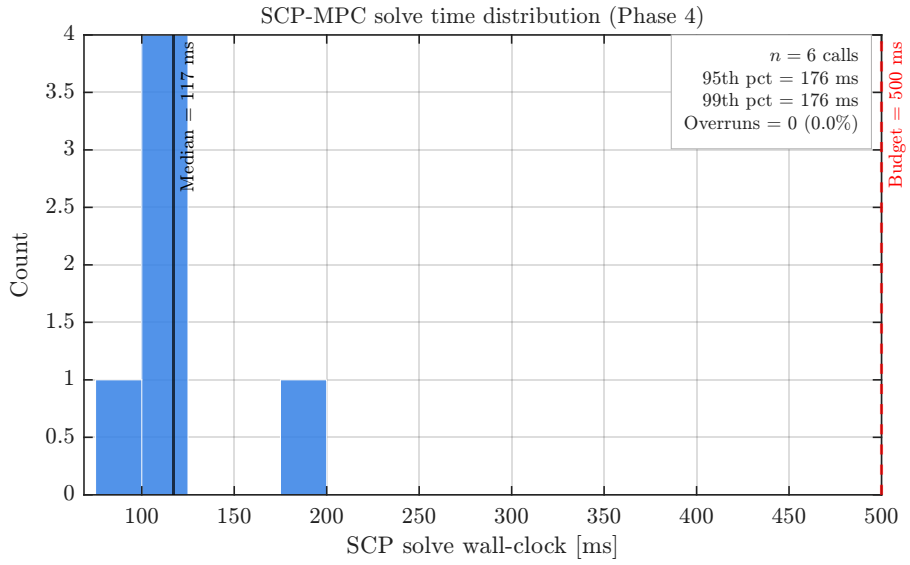


Figure 4.23: Distribution of SCP-MPC solve wall-clock times across the closed-loop Phase 4 landing ($N = 6$ calls). Median solve time: 117 ms; maximum: 176 ms. Real-time budget at 2 Hz guidance rate is 500 ms (red dashed). Zero overruns observed in the nominal closed loop. Sample size precludes meaningful percentile estimation; the maximum is reported instead.

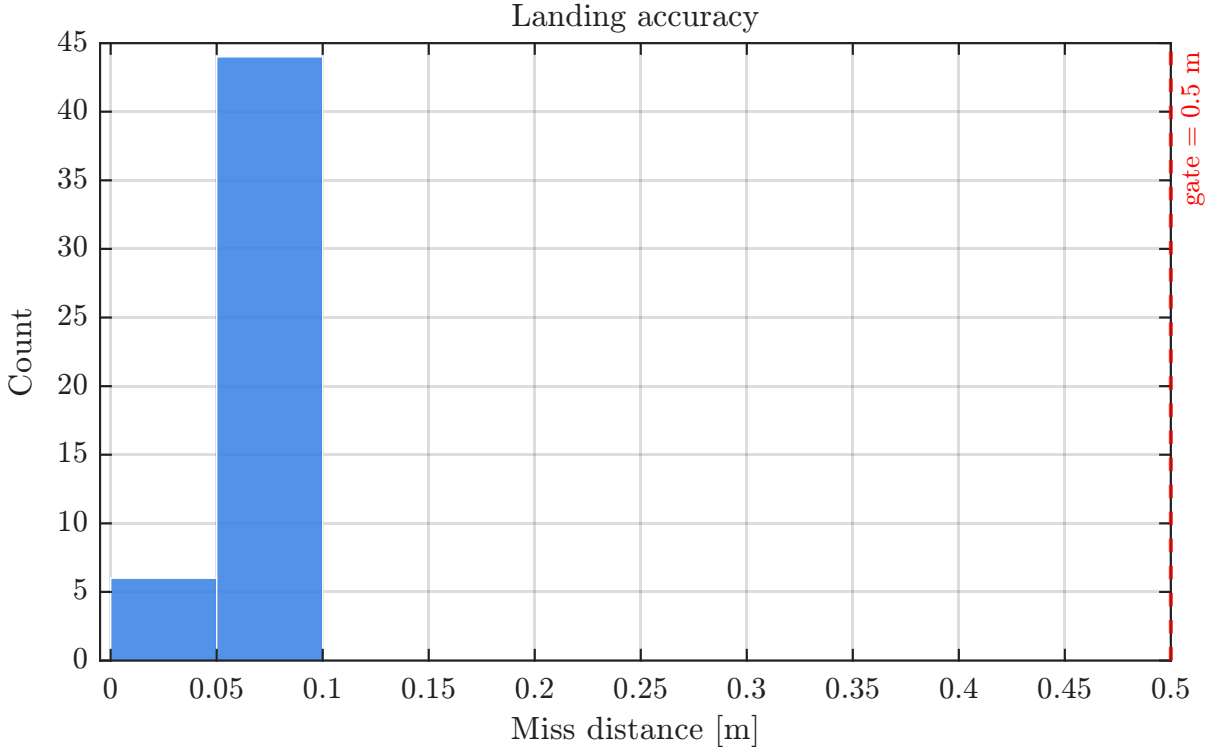


Figure 4.24: Miss-distance distribution across $N = 50$ Monte Carlo dispersed Phase 4 closed-loop runs (dispersion ranges in Table 4.3). All runs satisfy the < 0.5 m landing-accuracy gate.

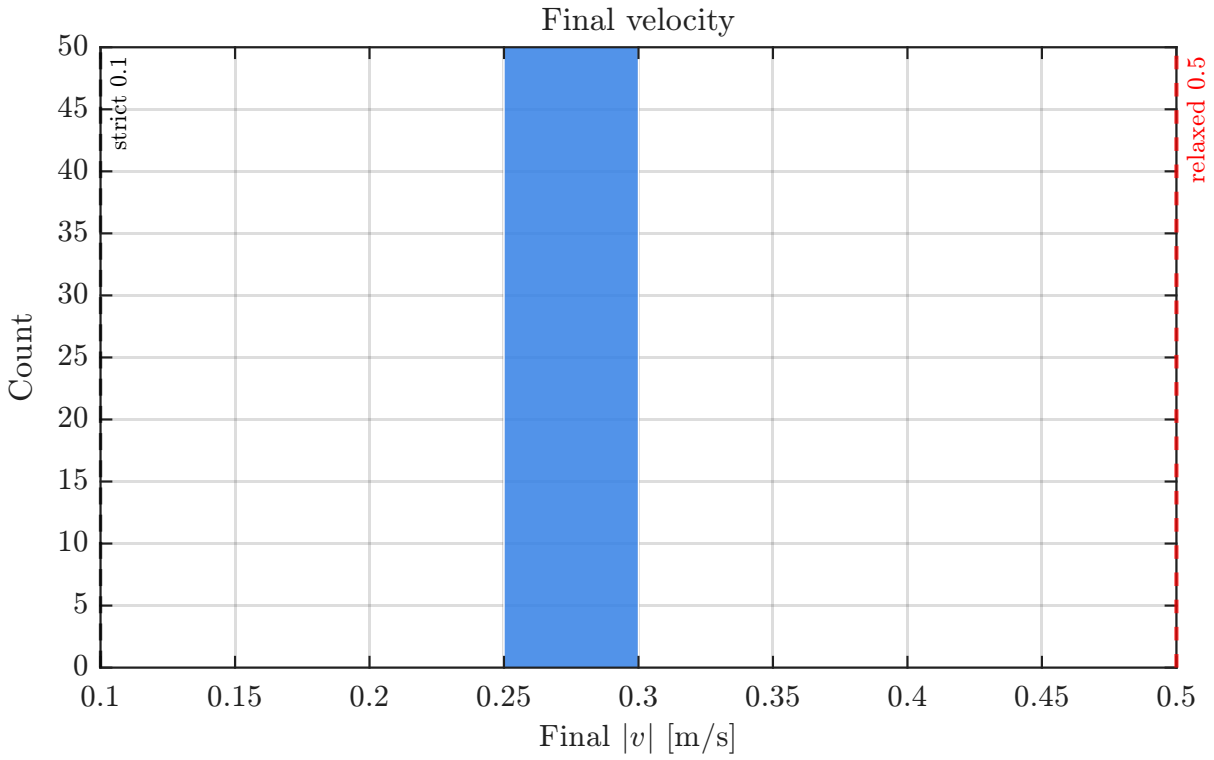


Figure 4.25: Final touchdown-velocity distribution for the same 50 dispersed runs as in Figure 4.24. All runs satisfy the relaxed < 0.5 m/s gate.

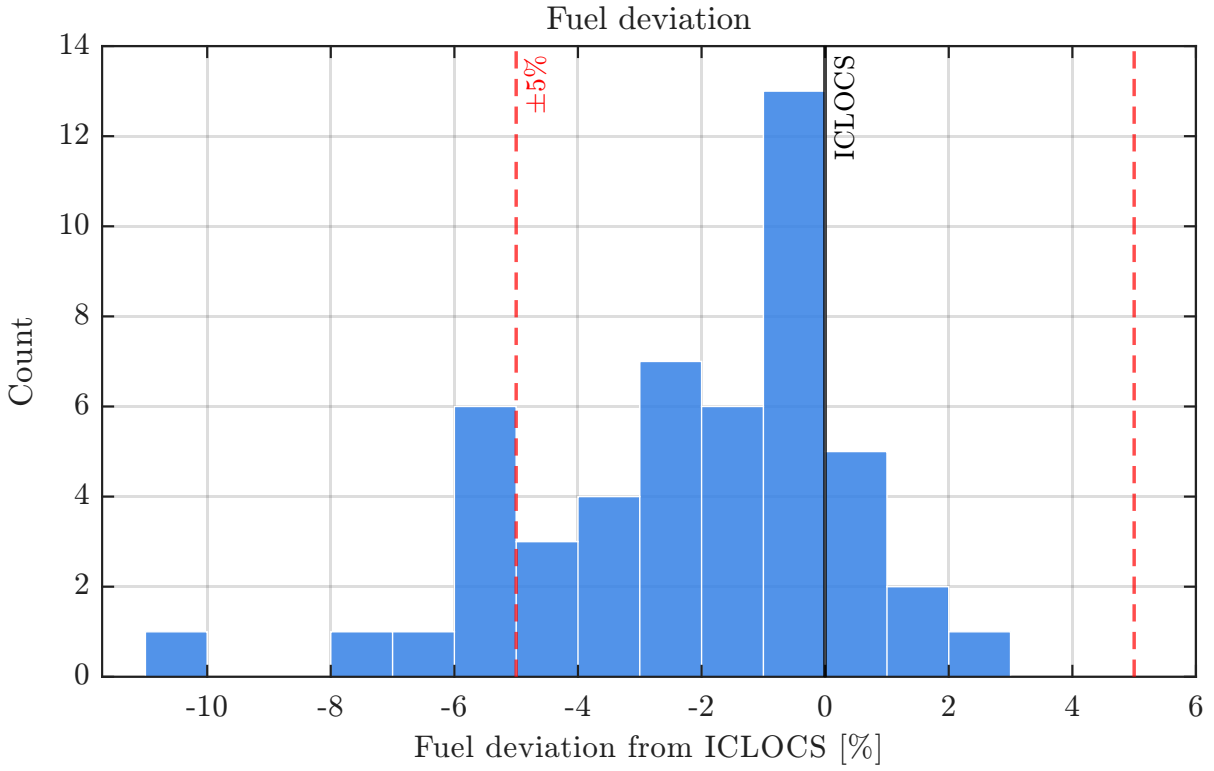


Figure 4.26: Fuel-burn deviation from the ICLOCS nominal reference across the 50 dispersed runs. Median -1.7%, 95th-percentile 6.3%; the bulk of the distribution lies within the $\pm 5\%$ band.

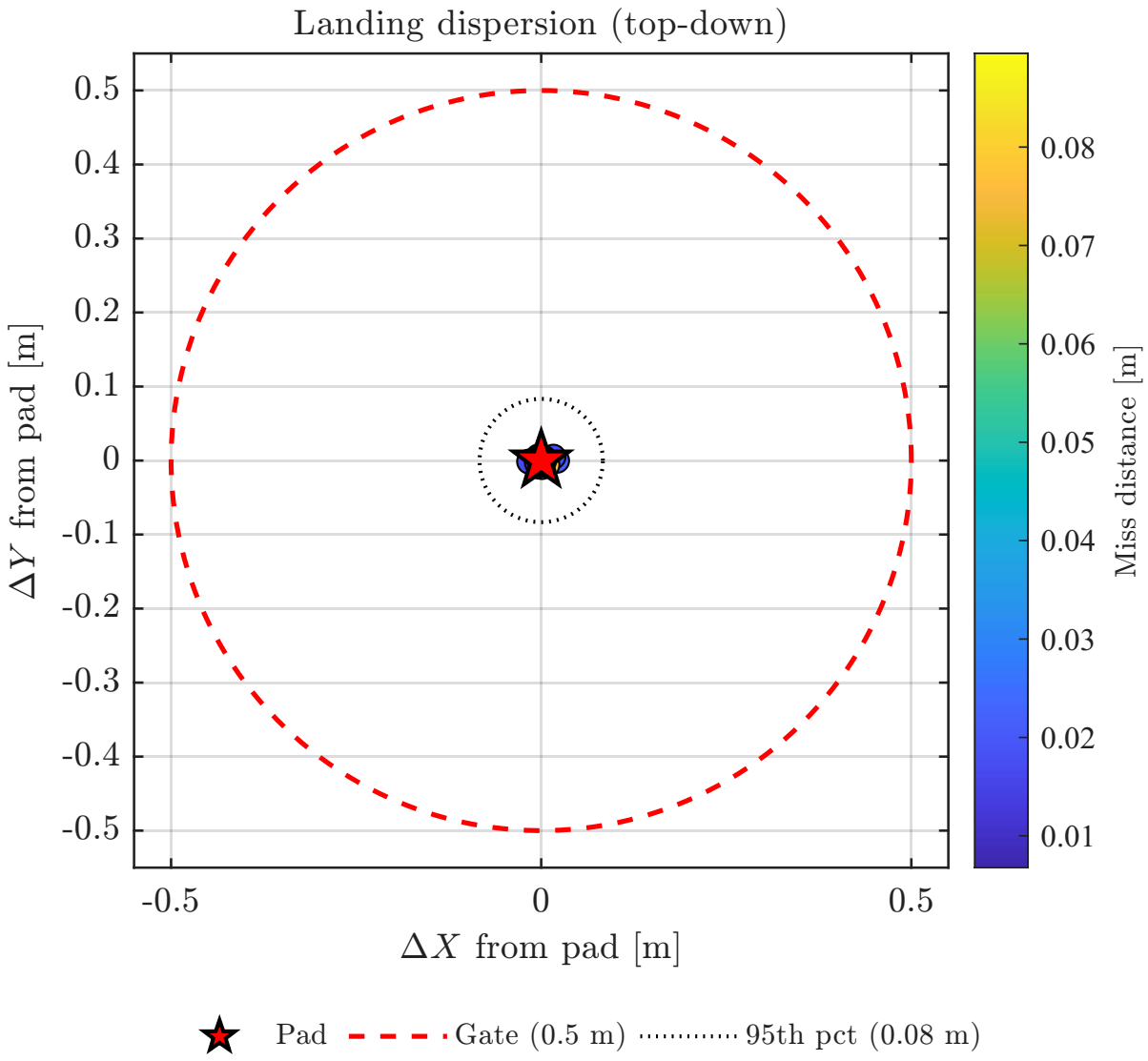


Figure 4.27: Top-down landing dispersion relative to the pad (red pentagram), coloured by miss distance. The 0.5 m landing-accuracy gate (dashed) dwarfs the observed 95th-percentile miss radius of 0.08 m. Zero fallback latches were triggered across the 50 runs.

Table 4.3: Monte Carlo dispersion ranges applied at Phase 4 entry (uniform, 50 seeds).

Quantity	**Nominal**	**Dispersion range**
Position r_x	0 m	± 5 m
Position r_y	0 m	± 5 m
Position r_z	5 m	± 1.5 m
Velocity v_x, v_y	0 m/s	± 0.8 m/s
Velocity v_z	0 m/s	± 0.5 m/s
Mass	13.19 kg	± 0.5 kg

Chapter 5

Conclusions

5.1 Summary of Contributions and Objective Closure

This thesis developed and verified a hybrid Guidance, Navigation, and Control (GNC) architecture for the Sunride *Desert Winds* simulation: an offline fuel-optimal four-phase reference, an online sequential convex programming (SCP) model predictive controller with lossless convexification (LCvx) for terminal landing, a 16-error-state Multiplicative Extended Kalman Filter (MEKF), a gain-scheduled Linear-Quadratic Integral (LQI) inner loop, and a validity-gated two-mode dispatcher. The six SMART objectives of Section 1.2 are closed as follows. **O1** (multi-start uniqueness): 50/50 runs at $\sigma_{\text{fuel}} = 0$ (§4.1). **O2** (fuel-vs-precision comparison): 15.8 % propellant and 17.6 % time savings (§4.1). **O3** (plant verification): seven benchmarks pass at 10^{-8} (§4.2). **O4** (SCP-MPC validity): five unit tests pass with mean SCP iterations 3.4 (§4.2, §4.3). **O5** (closed-loop integration): nominal Simulink landing succeeds with zero validity-gate strikes (§4.4). **O6** (embedded feasibility): median per-call utilisation of 23 % and maximum 35 % across $N = 6$ calls against the 500 ms guidance budget, with feasibility for embedded deployment conditional on the future-work items in §5.3 (§4.5).

5.2 Limitations

Four limitations bound the present results. First, although a 50-run Monte Carlo campaign with randomised initial-state dispersion was conducted (§4.4), it does not include wind injection or full sensor-bias variation, so atmospheric and sensor robustness remain unquantified beyond the nominal noise profile. Second, the Shinozuka wind model is implemented in the plant but was not injected in the reported closed-loop simulation. Third, the MEKF accelerometer and gyroscope bias covariances are diagnostically frozen at zero; full bias estimation is implemented in the filter equations but requires its own

verification campaign before being enabled. Fourth, solver wall-clock figures are workstation measurements; a definitive embedded claim requires C-generated ECOS, cache-locked memory, and worst-case execution-time analysis on flight-grade hardware.

5.3 Future Work

Each limitation of Section 5.2 points to an immediate next step. Extending the existing Monte Carlo campaign of §4.4 with wind injection (using the implemented Shinozuka model) and seeded sensor-bias variation is the highest-priority follow-on, to establish statistical landing accuracy and dispatcher robustness under realistic atmospheric and sensor conditions. Enabling the MEKF bias estimation requires only the unfreezing of the bias-covariance block, but a separate validation campaign against simulated bias profiles is needed to confirm convergence under realistic IMU drift. A hardware-in-the-loop bench using an embedded C-generated ECOS port would convert the workstation feasibility result into a flight-qualifiable timing claim. Finally, integration with the physical *Hopper* vehicle currently under development by SPL and Project Sunride is the natural culmination of this work, providing pre-flight GNC maturation against the hardware for which the simulation was specifically calibrated.

Bibliography

[1] P. V. M. Simplicío, A. Marcos, and S. Bennani, "Reusable launchers: development of a coupled flight mechanics, guidance and control benchmark," *J. Spacecraft Rockets*, 2019. [Online]. Available: `https://research-information.bris.ac.uk/ws/portalfiles/portal/208378543/Reusable_Launchers_development_of_a_coupled_flight_mechanics.pdf`

[2] H. W. Jones, "The recent large reduction in space launch cost," in *48th Int. Conf. Environmental Systems*, 2018.

[3] A. Gregorio, "Methodology and tool to support the conceptual design of a reusable VTVL TSTO vehicle," M.S. thesis, Dept. of Aerospace Engineering, Politecnico di Torino, Turin, Italy, Dec. 2024.

[4] B. Açıkmeşe, J. M. Carson III, and L. Blackmore, "Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem," *IEEE Trans. Control Syst. Technol.*, vol. 21, no. 6, pp. 2104–2113, 2013.

[5] B. Açıkmeşe and S. R. Ploen, "Convex programming approach to powered descent guidance for Mars landing," *J. Guid. Control Dyn.*, vol. 30, no. 5, pp. 1353–1366, 2007.

[6] M. Szmuk and B. Açıkmeşe, "Successive convexification for 6-DoF Mars rocket powered landing with free-final-time," in *AIAA Guidance, Navigation, and Control Conf.*, Feb 2018.[Online]. Available:`https://arxiv.org/abs/1802.03827`

[7] X. Chen, R. Zhang, and H. Li, "Trajectory dispersion control for precision landing guidance of reusable rockets," arXiv preprint arXiv:2504.11894, Apr. 2025. [Online]. Available: `https://arxiv.org/abs/2504.11894`

[8] Madden, M. M. (2018). Verifying implementation of the Dryden turbulence model and MIL-F-8785 gust gradient. In *Proceedings of the 2018 AIAA Modeling and Simulation Technologies Conference*, NASA Langley Research Center, Hampton, VA, USA. doi: 10.2514/6.2018-3580.

[9] Açıkmeşe, B., & Blackmore, L. (2011). Lossless convexification of a class

of optimal control problems with non-convex control constraints. *Automatica*, 47(2), 341–347. Available: https://www.sciencedirect.com/science/article/pii/S0005109810004516.

[10] D. P. Scharf *et al.*, "ADAPT demonstrations of onboard large-divert guidance with a VTVL rocket," in *IEEE Aerospace Conf.*, 2014. [Online]. Available: https://ieeexplore.ieee.org/document/6836462/authors

[11] D. P. Scharf, B. Açıkmeşe, D. Dueri, J. Benito, and J. Casoliva, "Implementation and experimental demonstration of onboard powered-descent guidance," *J. Guid. Control Dyn.*, vol. 40, no. 2, pp. 213–229, 2017, doi: 10.2514/1.G000399.

[12] M. Ferlin, B. Rmili, S. Le Martelot, A. Schindler, O. Boisneau, S. Dussy, S. Querry, B. Marciniak, P. Surmacz, M. Płochów, G. Pela, T. Gornicki, G. Poirey, and S. Besson, "FROG project: small demonstrator for big ambitions in RLV European objectives — status quo and results," in *Proc. 9th Eur. Conf. Aeronautics Space Sci. (EUCASS)*, 2022, doi: 10.13009/EUCASS2022-9718.

[13] H. Zhou, M. Wang, Q. Gan, X. Wang, and N. Cui, "Online multiphase reentry trajectory optimization for reusable rocket based on intelligent phase-transition," *Preprint (not peer reviewed).* [Online]. Available: https://ssrn.com/abstract=5407644

[14] dos Santos, P., & Oliveira, P. (2023). Thrust vector control and state estimation architecture for low-cost small-scale launchers. *Aerospace Science and Technology.* arXiv:2303.16983v2 [eess.SY].

[15] Linsën, R., Listov, P., de Lajarté, A., Schwan, R., & Jones, C. N. Optimal thrust vector control of an electric small-scale rocket prototype. Laboratoire d'Automatique, EPFL, Switzerland.

[16] J. Corke, "Trajectory optimisation of vertical take-off and vertical landing (VTVL) launch vehicles," BEng dissertation, Univ. of Sheffield, 2024.

[17] L. E. Briese, K. Schnepper, and P. Acquatella B., "Advanced modeling and trajectory optimization framework for reusable launch vehicles," in *IEEE Aerospace Conf.*, 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8396704

[18] Wang, C., & Song, Z. (2018). Trajectory optimization for reusable rocket landing. In *Proceedings of the 2018 IEEE CSAA Guidance, Navigation and Control Conference (CGNCC)*, Xiamen, China, pp. 1–6. doi: 10.1109/GNCC42960.2018.9019105.

[19] L. Blackmore, B. Açıkmeşe, and J. M. Carson III, "Lossless convexification of control-constrained nonlinear optimal control problems," in *Proc. American Con-*

trol Conf. (ACC), 2012. [Online]. Available: http://www.larsblackmore.com/BlackmoreEtAlACC2012.pdf

[20] D. Luo, K. Echigo, and B. Açıkmeşe, "Revisiting lossless convexification: Theoretical guarantees for discrete-time optimal control problems," *arXiv preprint arXiv:2410.09748*, Oct. 2024. [Online]. Available: https://arxiv.org/abs/2410.09748

[21] De Oliveira, A. (2023). *Guidance and Control System Design for Reusable Launch Vehicle Descent and Precise Landing*. Doctoral Dissertation, Doctoral Programme in Aerospace Engineering, Department of Aerospace Science and Technology, Politecnico di Milano, Italy. Cycle XXXVI. Supervisor: Prof. Michèle Lavagna. Industrial Supervisor: G. De Zaiacomo (Deimos Space). 220 pp.

[22] Y. Nie, O. Faqir, and E. C. Kerrigan, "ICLOCS2: try this optimal control problem solver before you try the rest," in *UKACC Int. Conf. Control*, IEEE, 2018. doi: 10.1109/CONTROL.2018.8516816.

[23] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2nd ed., SIAM, Philadelphia, PA, 2010.

[24] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Math. Program.*, vol. 106, no. 1, pp. 25–57, 2006, doi: 10.1007/s10107-004-0559-y.

[25] Bernacchia, D. (2018/2019). *Design of Thrust Vectoring attitude control system for Lunar Lander flying testbed*. Master's Thesis in Aerospace Engineering (LM-20), Alma Mater Studiorum – Università di Bologna, School of Engineering, Forlì Campus. Supervisor: Prof. Marco Zannoni. Co-supervisor: Prof. David Barnhart.

[26] Y. Mao, M. Szmuk, X. Xu, and B. Açıkmeşe, "Successive convexification: A superlinearly convergent algorithm for non-convex optimal control problems," *arXiv preprint arXiv:1804.06539*, Feb 2019. [Online]. Available: https://arxiv.org/abs/1804.06539

[27] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *Proc. IEEE Int. Symp. Computer-Aided Control Systems Design (CACSD)*, Taipei, Taiwan, 2004.

[28] A. Domahidi, E. Chu, and S. Boyd, "ECOS: an SOCP solver for embedded systems," in *Proc. European Control Conf. (ECC)*, Zürich, Switzerland, 2013, pp. 3071–3076.

[29] J. Pilipovsky, "Free final time adaptive mesh covariance steering via sequential convex programming," *arXiv preprint arXiv:2603.14868*, Mar. 2026. [Online]. Available: https://arxiv.org/abs/2603.14868

[30] A. V. Rao, "AAS 09-334 A SURVEY OF NUMERICAL METHODS FOR OPTIMAL CONTROL,", 2009. url=https://api.semanticscholar.org/CorpusID:13008493

[31] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, Cambridge, U.K., 2004, ISBN: 978-0-521-83378-3.

[32] J. L. Crassidis and J. L. Junkins, *Optimal Estimation of Dynamic Systems*, 2nd ed., CRC Press, Boca Raton, FL, 2011.

[33] F. L. Markley, "Attitude error representations for Kalman filtering," *J. Guid. Control Dyn.*, vol. 26, no. 2, pp. 311–317, 2003.

[34] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*, Prentice Hall, Englewood Cliffs, NJ, 1990.

[35] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed., Princeton University Press, Princeton, NJ, 2021.

[36] W. J. Rugh and J. S. Shamma, "Research on gain scheduling," *Automatica*, vol. 36, no. 10, pp. 1401–1425, 2000.

[37] UK Students for the Exploration and Development of Space (UKSEDS), "The UK's Student Space Charity," [Online]. Available: `https://ukseds.org/`

[38] European Rocketry Challenge (EuRoC), [Online]. Available: `https://euroc.pt/`

[39] Definity Education, [Online]. Available: `https://www.definityeducation.org/lander-challenge/`

[40] D. Luo, F. Spada, and B. Açıkmeşe, "Discrete lossless convexification for pointing constraints," *arXiv preprint arXiv:2501.06931*, Jun. 2025. [Online]. Available: `https://arxiv.org/abs/2501.06931`

[41] Y. Mao, M. Szmuk, and B. Açıkmeşe, "Successive convexification of non-convex optimal control problems with state constraints," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 4063–4069, 2017.[Online]. Available: `https://www.sciencedirect.com/science/article/pii/S2405896317312405` doi: `10.1016/j.ifacol.2017.08.790`.

[42] M. Sagliano, "Generalized hp pseudospectral-convex programming for powered descent and landing," *J. Guid. Control Dyn.*, vol. 44, no. 9, 2021.

[43] R. Yang, X. Liu, and Z. Song, "Rocket landing guidance based on linearization-free convexification," *J. Guid. Control Dyn.*, 2024.

[44] J. Choi and J.-H. Kim, "Powered descent guidance via first-order optimization with expansive projection," *IEEE Access*, vol. 12, 2024, doi: 10.1109/ACCESS.2024.3381620.

[45] F. Xiong, C. Li, Y. Zhao, and X. Hu, "Rocket landing guidance using convex optimization and proportional navigation considering performance-limited engine," *Acta Astronautica*, vol. 201, pp. 209–223, Dec. 2022, doi: 10.1016/j.actaastro.2022.09.013.

[46] P. Lysandrou, "6-DOF free-final-time successive convexification powered descent guidance algorithm implementation," Final Project for ASEN 6519: Aerospace Vehicle Guidance & Control, Univ. of Colorado Boulder, Boulder, CO. [Online]. Available: `https://www.lysandrou.com/projects/scvx.pdf`

[47] A. Hallmark and P. Zhao, "Improving INDI for input nonaffine systems via learning-based nonlinear control allocation," *arXiv preprint arXiv:2604.06337*, Apr. 2026. [Online]. Available: `https://arxiv.org/abs/2604.06337`

[48] J. Duchi, S. Boyd, and J. Mattingley, "Sequential convex programming," EE364B Lecture Notes, Stanford University, Spring 2018. [Online]. Available: `https://stanford.edu/class/ee364b/lectures/seq_notes.pdf`

[49] J. V. Burke, "Introduction to Nonlinear Programming," University of Washington, Math 408 Course Notes. [Online]. Available: `https://sites.math.washington.edu/~burke/crs/408/notes/nlp/nlp_intro.pdf`

[50] L. Blackmore, "Lossless Convexification," Personal webpage. [Online]. Available: `http://www.larsblackmore.com/losslessconvexification.html`

Appendix A

Reflective Statement

Skills developed

This project significantly improved my MATLAB proficiency and Simulink modelling, particularly in managing a code base spread across many interdependent files. Before this project, most of my MATLAB work had been confined to single scripts; the dissertation forced me to learn how to structure a multi-file project with verification tests, design documents, and clear interfaces between subsystems.

I also developed my control engineering knowledge well beyond the taught modules, gaining a working understanding of state-space systems and the practical machinery of estimator and controller design. My research skills improved markedly through the literature review: I learnt how to read original journal papers critically and identify which ideas were directly transferable to my problem and which were not. Perhaps most importantly, I developed the skill of turning mathematical formulations into working code, the step from a paper's equations to a verified implementation is a non-trivial one that I had not appreciated before this project. I also gained substantive knowledge of optimal control, direct collocation, sequential convex programming, and lossless convexification, and learnt how to combine multiple research sources into a single coherent solution rather than implementing each in isolation.

Key challenges and how I resolved them

Three challenges stand out.

First, balancing the project with the rest of my coursework was a persistent difficulty throughout the year. With other modules running in parallel, exam revision, and graduate applications to prepare, I had to learn to prioritise project tasks ruthlessly and accept

that some weeks could not give the project the time it needed. I resolved this by setting weekly milestones small enough to complete in fragmented sessions, rather than waiting for free days that never arrived.

Second, formulating the optimal control problem correctly so that the solver would converge reliably was harder than I expected. Managing all of the different constraints simultaneously, while still producing a trajectory that was both feasible and close to fuel-optimal, took several reformulations before the solution stabilised.

Third, designing the overall architecture was very difficult. Managing the many different files so that they integrated cleanly into a single end-to-end simulation, with each subsystem exchanging the correct signals at the correct rates, was a system-level challenge I had not encountered in any previous module.

What I would do differently

If I were starting again, I would plan the project more carefully and allocate more time to the architectural design in the early weeks of the year. Many of the integration problems I encountered later could have been avoided by deciding the interfaces between subsystems before implementing any of them, rather than discovering the inconsistencies through end-to-end testing once the layers were already coupled.

Contribution to my engineering development

This project taught me how to implement ideas from research papers into working code, a step that the taught syllabus does not really cover. I have become substantially more confident reading aerospace and control-theory literature, and more competent at writing software that another engineer could maintain. I also feel I now have a practical, rather than purely theoretical, grasp of GNC architecture design, specifically, which decisions matter most early in the design and which can safely be deferred. These are skills I expect to draw on directly in whatever direction my engineering career takes next.

Project management lessons

Regular meetings with my supervisor were valuable for keeping me accountable and honest about my progress. However, I have to admit that I worked slowly in the first half of the year and ended up rushing in the closing weeks, which I now recognise as my biggest project-management failure on this dissertation. Better time management, specifically, setting hard internal deadlines for each subsystem rather than treating the final submission as the only fixed point, is the lesson I will carry into future projects.

Project Gantt Chart

The Gantt chart below shows the planned versus actual hours across the project, organised by milestone and SMART objective. Both pages are rotated to read top-to-bottom on the portrait page.

Trajectory Optimisation & Online Replanning for Small-Scale VTVL - Project Gantt

Lucas White | Supervisor: Dr Yuanbo Nie | 6 Oct 2025 – 26 May 2026

Task	SMART Obj.	Allotted (h)	Actual (h)	W1 Oct 06-Oct 12	W2 Oct 13-Oct 19	W3 Oct 20-Oct 26	W4 Oct 27-Nov 02	W5 Nov 03-Nov 09	W6 Nov 10-Nov 16	W7 Nov 17-Nov 23	W8 Nov 24-Nov 30	W9 Dec 01-Dec 07	W10 Dec 08-Dec 14	W11 Dec 15-Dec 21	W12 Dec 22-Dec 28	W13 Dec 29-Jan 04	W14 Jan 05-Jan 11
MILESTONE 1 - Developing Schemes (10 Oct – 25 Nov 2025)																	
Literature review (VTVL background, OCP theory, convex opt., MEKF, GNC integration)	Motivation	35		6	7	7	6	5	3	1							
			42	4	7	8	8	6	5	3	1						
Dynamic modelling (14-state quaternion 6-DoF plant, drag, mass-varying inertia)	O3	15				3	4	4	4								
			18				3	5	5	4	1						
Actuation mechanism (TVC servo 2nd-order, throttle valve 1st-order, gimbal mapping)	—	6						2	2	2							
			5						2	3							
Control architecture (hybrid GNC layout, validity-gated 2-mode dispatcher design)	—	10							3	4	3						
			12							3	5	4					
MILESTONE 2 - Evaluation (28 Nov 2025 - 11 Feb 2026)																	
Mission specification (Desert Winds parameters, 4-waypoint mission, path constraints)	—	5										3	2				
			4									2	2				
Proof of concept (initial ICLOCS2 setup, baseline 4-phase solve, sanity checks)	O1	20										4	6	6			
			22										5	6			
Interim report writing(Understanding the Problem)	—	15											7	8			
			18										8	10			
MILESTONE 3 - Development (12 Feb - 3 Mar 2026)																	
Trajectory optimisation algorithm (Hermite–Simpson, fuel-optimal OCP, IPOPT)	O1	25															
			30														
Open-loop evaluation (50-run multi-start, fuel-vs-precision comparison)	O1, O2	10															
			12														
Closed-loop SCP-MPC with LCvx (YALMIP parametric SOCP, ECOS, validity gate, warm-start)	O4	35															
			45														
MILESTONE 4 - Analysis (5 Mar -1 May 2026)																	
Plant verification (7 analytical benchmarks at 1e-8 tolerance)	O3	10															
			8														
16-state MEKF implementation and verification (IMU/GPS/baro multi-rate fusion)	—	18															
			22														
LQI inner loop & actuator integration (mass-gain-scheduled, anti-windup)	—	10															
			12														
Full Simulink closed-loop integration (end-to-end stack, dispatcher in Simulink)	O5	22															
			32														
Simulation analysis (Monte Carlo dispersion, nominal closed-loop characterisation)	O5	12															
			15														
Solver performance & embedded feasibility profiling	O6	6															
			6														
Future hopper implementation analysis (design guidance for Sunride deployment)	—	6															
			5														
FINAL PHASE - Report writing, viva and submission (May 2026)																	
Final report writing (Intro → Conclusions, ~6k words)	—	45															
			55														
Figures, polish, references and Appendix A reflective statement	—	10															
			15														
Viva preparation (slide deck, anticipated questions, dry runs)	—	10															
			8														
Final submission (26 May)	—	2															
			2														
TOTAL - Planned (hours)		327		6	7	10	10	11	12	7	3	7	15	14	0	0	0
TOTAL - Actual (hours)			388	4	7	8	11	11	12	13	7	6	15	16	0	0	0

Key:	
	Planned hours that week (orange)
	Actual hours that week (green)
	Milestone heading
	No work scheduled (winter break + Jan exams)
	Key date (viva, submission)

W15	W16	W17	W18	W19	W20	W21	W22	W23	W24	W25	W26	W27	W28	W29	W30	W31	W32	W33	W34
Jan 12-Jan 18	Jan 19-Jan 25	Jan 26-Feb 01	Feb 02-Feb 08	Feb 09-Feb 15	Feb 16-Feb 22	Feb 23-Mar 01	Mar 02-Mar 08	Mar 09-Mar 15	Mar 16-Mar 22	Mar 23-Mar 29	Mar 30-Apr 05	Apr 06-Apr 12	Apr 13-Apr 19	Apr 20-Apr 26	Apr 27-May 03	May 04-May 10	May 11-May 17	May 18-May 24	May 25-May 31
			2	2															
			4	5	2														
			4	7	8	6													
			3	8	9	7	3												
					4	6													
						4	6	2											
				5	8	10	12												
						5	10	12	10	8									
							5	5											62
								4	4										
								4	7	7									
								5	8	6	3								
										5	5								
										5	5	2							
										4	8	6	4						
											4	8	10	8	2				
												4	6	2					
													4	8	3				
														3	3				
														3	3				
														2	4				
															2	3			
													5	10	12	10	5	3	
														4	12	16	14	6	3
																4	4	2	
																3	6	4	2
																	5	5	
																	3	5	
																			2
																			2
0	0	0	6	14	20	22	17	9	7	16	13	10	15	17	19	14	14	10	2
0	0	0	7	13	11	16	19	23	22	19	12	10	14	23	22	22	23	15	7

Appendix B

Use of AI Statement

Acknowledge, Describe, Evidence template: a completed version of this template is included as an appendix in accordance with the University of Sheffield's guidance on the use of generative AI (GenAI) technology.

Acknowledge

I confirm that no generative AI (GenAI) system was used in the preparation, drafting, or production of this assessment.

Describe

Not applicable. No generative AI tools were used in this assessment.

Evidence

Not applicable. No generative AI outputs were generated or incorporated.

Declaration

I confirm that no content created by generative AI technologies has been presented as my own work.

Signed:  Date: 23/05/26

Appendix C

Viva Poster

The presentation-style poster prepared for the viva, summarising the project background, methods, key results, and conclusions. It is rotated to read top-to-bottom on the portrait page.

Trajectory Optimisation and Online Replanning for a Small-Scale VTVL Launch Vehicle

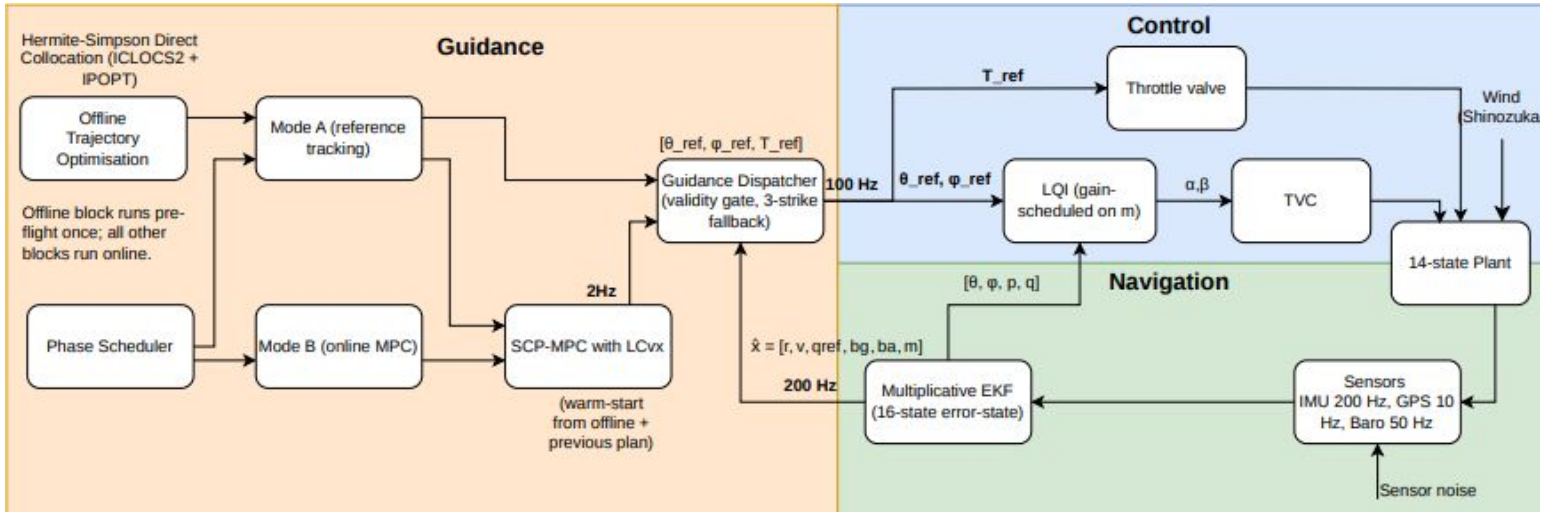
Lucas White (230157015) - Aerospace Engineering - May 2026 - Supervisor: Dr Yuanbo Nie

Background

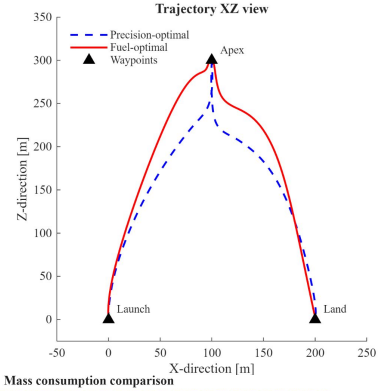
Reusable VTVL launchers have driven launch cost down ~10× ; the bottleneck has shifted from propulsion to onboard GNC
Convex powered-descent guidance has flight heritage at the >100 kg class (G-FOLD on Xombie)
Gap: Closed-loop integration of offline fuel-optimal reference with an online replanner is under-documented at the sub-25 kg testbed scale

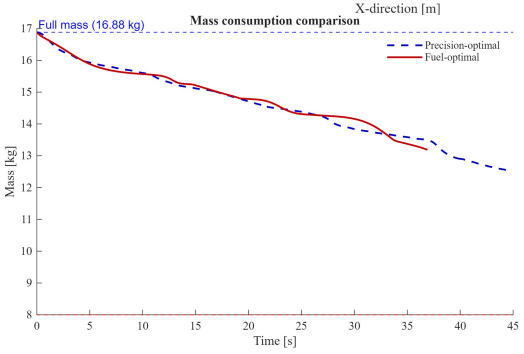
Aim: Develop and verify a hybrid GNC architecture for the 16.88 kg *Hopper* vehicle as pre-flight maturation for the Sunride Lander Challenge entry.

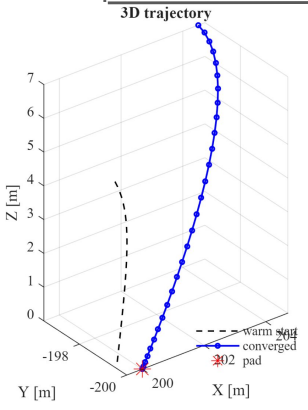
Methods

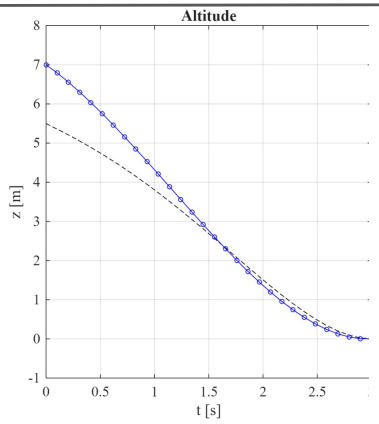


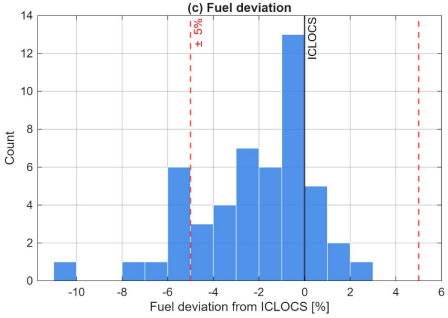
Results

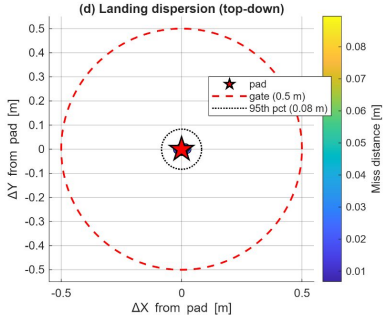












Propellant saving (fuel-opt vs precision)	Mission-time saving	Nominal miss distance	Terminal speed	Median SOCP solve (500 ms budget)
15.8%	17.6%	7.4 cm	0.29 m/s	148 ms

Conclusions: Hybrid GNC architecture verified end-to-end in Simulink: all 6 SMART objectives closed, zero validity-gate strikes across nominal and 50-run dispersed campaigns. 30% solver headroom (148 ms / 500 ms budget) leaves room for embedded port - direct enabler for the Sunride Lander Challenge entry.

Limitations: Software-only verification (no HIL); σ_wind = 0 in dispersion campaign; yaw monitored not controlled (gimbal constraint, not a defect).

Future work: Wind injection, MEKF bias-covariance unfreezing, HIL on Sunride Hopper.

(Açıkmeşe & Blackmore 2013, Scharf 2017, Szmuk & Açıkmeşe 2018, Corke 2024)

Appendix D

Project Aim, Objectives, and Research Questions

This appendix reproduces the original project aim, objectives, and research questions as defined at the start of the academic year. The dissertation's six SMART objectives stated in Section 1.2 are the technical refinement of the objectives listed below.

Project Aim

To design, verify, and evaluate a hybrid guidance, navigation, and control architecture for a small-scale VTVL rocket, coupling an offline fuel-optimal trajectory with an online convex model predictive controller to achieve precision landing, robustness, and computational efficiency on the Sunride *Hopper*.

Project Objectives

1. Formulate and implement a trajectory optimisation algorithm to generate fuel- or time-efficient flight profiles under operational constraints.
2. Evaluate the open-loop optimised solution against a precision-tracking baseline to quantify the propellant and mission-time gains under identical mission constraints.
3. Design and implement a closed-loop SCP-MPC with lossless convexification, arbitrated by a validity-gated dispatcher tracking the offline reference.
4. Assess control performance in terms of landing accuracy, dispatcher stability, computational efficiency, and propellant consumption under dispersion.
5. Analyse the feasibility of real-time onboard computation by profiling SCP wall-clock

and identifying trade-offs between optimality and embedded implementability.

6. Propose future improvements toward hardware-in-the-loop validation and integration with the next-generation Sunride *Hopper* flight vehicle.

Research Questions

1. How can fuel-optimal trajectory optimisation be applied to improve the efficiency and precision of guidance for small-scale VTVL launch vehicles?
2. What are the key trade-offs between optimality, robustness, and computational complexity when implementing convex MPC for online powered-descent guidance?
3. What methods can ensure feasibility of real-time onboard computation of sequential convex programming for sub-25 kg testbed-scale VTVL applications?
4. How can the insights gained from closed-loop simulation be adapted for integration into the experimental Sunride *Hopper* flight-test programme?

Appendix E

Risk Register

The project risk register, compiled at the outset and updated throughout the academic year, is reproduced overleaf. Pre-mitigation and residual scores are given on a Likelihood (L) $\times$ Severity (S) scale, with risk score $R = L \times S$. Two technical risks marked *operational solver failure* and *multi-rate Simulink integration* directly motivated the validity-gated dispatcher (Section 3.6) and the build-sequence verification campaign (Section 3.8) respectively.

Risk and Opportunities

Risk and Opportunities	Pre Mitigation			Mitigation Methods	Residual			(should be the best person to manage the
	L	S	R		L	S	R	
Incomplete dynamic model or incorrect assumptions: Inaccurate simulation results; controller may not behave as expected	3	4	12	Start with a validated simplified model (2D or vertical), verify against published data, and incrementally add complexity.	2	2	4	Lucas -Student
Optimisation solver fails to converge or produces infeasible trajectories:No viable open-loop solution to evaluate	3	3	9	Use problem scaling, warm starts, or simpler cost functions; test solver with relaxed constraints first.	1	2	2	Lucas -Student
Computational demands exceed available hardware: Real-time control or MPC infeasible for demo	4	4	16	Profile code early, simplify models, reduce horizon, or use a faster solver or compiled implementation.	2	2	4	Lucas -Student
Lack of reliable simulation results before deadline:Inability to evaluate control laws	3	2	6	Set clear milestones for simulation readiness; use baseline PID or analytical solutions for interim testing.	2	2	4	Lucas -Student
Control algorithm instability or divergence: System crashes in simulation or unsafe outputs	2	4	8	Perform linear stability analysis, add saturation and anti-windup logic, tune controllers gradually.	1	3	3	Lucas -Student
Sensor/noise models oversimplified: Unrealistic robustness assessment	3	3	9	Introduce representative noise/disturbance models and verify controller sensitivity.	2	2	4	Lucas -Student
Insufficient documentation of methods and results: Loss of marks and project credibility	4	3	12	Maintain a detailed logbook of assumptions, parameters, and test results throughout development.	2	2	4	Lucas -Student
Time constraints or task overload: Failure to meet key deliverables	4	5	20	Develop a phased plan (modelling → open-loop → closed-loop), prioritising a working prototype over full fidelity.	1	4	4	Lucas -Student
Software or toolchain errors: Project delays or data loss	4	3	12	Use version control (Git), back up regularly, and test code incrementally.	1	2	2	Lucas -Student
Online replanner returns mathematically optimal but operationally unusable solutions	3	4	12	five-flag validity gate (NaN check, LCvx slack tolerance, terminal error, plan continuity, solver exit code) with three-strike fallback latch to Mode A.	1	3	3	Lucas -Student
Misalignment with supervisor expectations: Rework or project scope changes late in timeline	4	3	12	Schedule regular progress reviews and sign off on milestones early.	1	2	2	Lucas -Student
Multi-rate Simulink integration errors between concurrent subsystems	4	3	12	Mitigation: build-sequence verification campaign exercising each layer in isolation before end-to-end integration.	2	2	4	Lucas -Student

Appendix F

MATLAB Source Code

The MATLAB implementation of the offline trajectory optimisation and the online SCP-MPC is reproduced below. The closed-loop integration, inner attitude loop, plant, and dispatcher are implemented in Simulink and shown in the block diagram of Figure 3.1.

F.1 Offline Trajectory Optimisation (ICLOCS2)

The four-phase fuel-optimal optimal control problem is transcribed by Hermite–Simpson collocation in ICLOCS2/IPOPT (Section 3.2). The driver script `main_ThreeDimRocketFourPhase.m` sequences the four phases, with each phase defined by a paired problem file, dynamics function, and settings file.

F.1.1 Driver Script

```
% main_ThreeDimRocket - Main script to solve the Optimal Control Problem
%
% VTVL Rocket (Four-phase Formulation)
%
% Copyright (C) 2019 Yuanbo Nie, Omar Faqir, and Eric Kerrigan. All Rights Reserved.
% The contribution of Paola Falugi, Eric Kerrigan and Eugene van Wyk for the work on ICLOCS Version 1 (2010) is kindly acknowledged.
% This code is published under the MIT License.
% Department of Aeronautics and Department of Electrical and Electronic Engineering,
% Imperial College London London  England, UK
% ICLOCS (Imperial College London Optimal Control) Version 2.5
% 1 Aug 2019
% iclocs@imperial.ac.uk
```

```
%
% Altered by L. White, University of Sheffield May 2026


%% Solve the problem
% main_ThreeDimRocket.m
% Multi-start version for fuel-optimal global optimum search
%
% Strategy: run the solver N_starts times from different initial guesses.
% With 12s per solve, N_starts = 50 takes ~10 minutes.
% If all successful solves converge to the same fuel cost (std dev < 0.05 kg),
% the global optimum is likely found.

clear all; close all; format compact;clc;

% Portable
script_dir = fileparts(mfilename('fullpath'));
cd(script_dir);

% Dissertation-matched plotting defaults
set(0, 'DefaultTextInterpreter',          'latex');
set(0, 'DefaultAxesTickLabelInterpreter', 'latex');
set(0, 'DefaultLegendInterpreter',        'latex');
set(0, 'DefaultAxesFontSize',     11);
set(0, 'DefaultTextFontSize',     11);
set(0, 'DefaultLegendFontSize',   10);
set(0, 'DefaultAxesLineWidth',    0.75);
set(0, 'DefaultLineLineWidth',    1.2);
set(0, 'DefaultLegendBox',        'off');
set(0, 'DefaultAxesBox',          'on');
set(0, 'DefaultFigureColor',      'white');

W_FULL = 6.20;
W_BIG  = 4.65;

phase_colors = [0      0.4470 0.7410;
    0.8500 0.3250 0.0980;
    0.4660 0.6740 0.1880;
    0.4940 0.1840 0.5560];
phase_names  = {'Ascent','Hover','Descent','Land'};


%% USER SETTINGS     adjust these
N_starts   = 50;     % number of solver attempts (increase for more confidence)
```

```
57 rng(42);                % fixed random seed - makes results reproducible
58                         % change this number to get a different random
                               sequence
59
60 %%  PROBLEM SETUP - done once outside the loop
61 %  (loading problem definition is slow; only do it once)
62 options.mp = settings_ThreeDimRocketFourPhase;
63 [problem, guess_original, options.phaseoptions] =
       ThreeDimRocketFourPhase;
64
65 %% Turn off IPOPT console output for all runs except the first
66 % (avoids hundreds of lines of solver output in your terminal)
67 options_quiet       = options;
68 options_quiet.mp.ipopt.print_level = 0;    % 0 = silent, 5 = verbose
69
70 %%  STORAGE
71 best_cost     = inf;          % track minimum fuel used [kg]
72 best_solution = [];           % store best solution struct
73 all_fuel      = nan(N_starts, 1);      % fuel used per attempt [kg]
74 all_feasible  = false(N_starts, 1);    % did the attempt converge?
75
76 % Initial mass is fixed      used to compute fuel consumed
77 m_initial = problem.phases{1}.states.x0(7);    % = ms + mp = 16.88 kg
78
79 %%  MULTI-START LOOP
80 fprintf('Starting multi-start optimisation: %d attempts\n', N_starts);
81 fprintf('Initial mass: %.3f kg\n\n', m_initial);
82
83 for k = 1:N_starts
84
85     fprintf('--- Attempt %d / %d ---  ', k, N_starts);
86
87     % DEBUG - check every phase has constraintErrorTol
88     for i = 1:length(problem.phases)
89         fprintf('Phase %d: constraintErrorTol exists = %d, settings =
                %s\n', ...
90             i, isfield(problem.phases{i}, 'constraintErrorTol'), ...
91             func2str(problem.phases{i}.settings));
92     end
93
94     if k == 1
95         % First attempt: original guess unchanged - gives a good
               baseline
96         guess_k   = guess_original;
97         opts_k    = options;          % verbose output for first solve
               only
98         fprintf('(original guess)\n');
```

```
    else
        % All subsequent attempts: randomly perturb the original guess
        % NOTE:  perturb from the ORIGINAL guess, not the last
        % solution. This ensures the search space is fully explored
           rather
        % than converging around one local minimum.
        guess_k  = perturbGuess(guess_original, problem);
        opts_k   = options_quiet;   % silent output
        fprintf('(perturbed guess)\n');
    end

    try
        [sol_k, ~] = solveMyProblem(problem, guess_k, opts_k);
        fprintf('    Solver returned, n_phases in solution = %d\n',
           length(sol_k.phaseSol));

        % ---- Compute fuel used [kg] ----
        % Total fuel = initial mass - final mass after phase 4.
        % Mass is state variable x(:,7), continuous across phases via
        % the linkage constraints, so we only need the Phase 4
           endpoint.
        n_phases  = length(sol_k.phaseSol);
        sol_last  = sol_k.phaseSol{n_phases};
        m_final   = speval(sol_last, 'X', 7, sol_last.tf);
        fuel_used = m_initial - m_final;

        all_fuel(k)     = fuel_used;
        all_feasible(k) = true;

        fprintf('    Fuel used: %.4f kg\n', fuel_used);

        % Keep the solution with minimum fuel
        if fuel_used < best_cost
            best_cost     = fuel_used;
            best_solution = sol_k;
            fprintf('    *** New best: %.4f kg ***\n', best_cost);
        end

    catch ME
        % Solver failed to converge     skip this attempt
        fprintf('    Solver failed: %s\n', getReport(ME, 'extended'));
    end

end


%%  RESULTS SUMMARY
```

```
143 feasible_fuels = all_fuel(all_feasible);
144
145 fprintf('MULTI-START RESULTS SUMMARY\n');
146 fprintf('-------------------------------------\n');
147 fprintf('Total attempts:       %d\n',   N_starts);
148 fprintf('Successful solves:    %d\n',   sum(all_feasible));
149 fprintf('Failed solves:        %d\n',   sum(~all_feasible));
150 fprintf('\nFuel consumption across successful solves:\n');
151 fprintf('  Best  (minimum):    %.4f kg\n', min(feasible_fuels));
152 fprintf('  Worst (maximum):    %.4f kg\n', max(feasible_fuels));
153 fprintf('  Mean:               %.4f kg\n', mean(feasible_fuels));
154 fprintf('  Std deviation:      %.4f kg\n', std(feasible_fuels));
155 fprintf('\nINTERPRETATION:\n');
156 fprintf('  Std dev < 0.05 kg  -> global optimum very likely found\n');
157 fprintf('  Std dev 0.05-0.2   -> increase N_starts for more
       confidence\n');
158 fprintf('  Std dev > 0.2      -> multiple local minima exist;
       increase N_starts\n');
159 fprintf('-------------------------------------------\n\n');
160
161
162 %%  USE BEST SOLUTION FOR FIGURE GENERATION
163
164 solution = best_solution;
165
166 % Saves the fuel-optimal result permanently for comparison
167 save('fuel_optimal_solution.mat', 'best_solution', 'best_cost',
       'all_fuel');
168 fprintf('Fuel-optimal solution saved to fuel_optimal_solution.mat\n');
169
170 %% Export reference trajectory for online MPC
171 % Converts ICLOCS 11-state Euler representation to 13-state
       quaternion format
172 % State mapping: [x,y,z,vx,vy,vz,m,theta_deg,phi_deg,p_rad/s,q_rad/s]
173 %              -> [x,y,z,vx,vy,vz,m,q0,q1,q2,q3,p_rad/s,q_rad/s]
174 % psi is pinned to 0 throughout (yaw uncontrolled in ICLOCS).
175
176 N_export = 1000;    % points per phase
177 reference.phase_transition_times = zeros(1,
       length(solution.phaseSol)+1);
178 reference.t     = [];
179 reference.x     = [];    % Nx13: quaternion-based
180 reference.u     = [];    % Nx3:  [T(N), alpha(deg), beta(deg)]
181 reference.phase = [];
182
183 for i = 1:length(solution.phaseSol)
184     sol_i = solution.phaseSol{i};
```

```
    tt    = linspace(sol_i.t0, sol_i.tf, N_export);

    % --- Extract 11 ICLOCS states ---
    x11 = zeros(N_export, 11);
    for j = 1:11
        x11(:,j) = speval(sol_i, 'X', j, tt)';
    end

    % --- Extract controls [T, alpha_deg, beta_deg] ---
    u3 = zeros(N_export, 3);
    for j = 1:3
        u3(:,j) = speval(sol_i, 'U', j, tt)';
    end

    % --- Convert Euler angles to quaternion (ZYX: yaw=0,
       pitch=theta, roll=phi) ---
    theta_rad = deg2rad(x11(:,8));   % pitch [rad]
    phi_rad   = deg2rad(x11(:,9));   % roll  [rad]
    psi_rad   = zeros(N_export, 1);  % yaw   [rad] - always 0 in
       ICLOCS

    % ZYX quaternion: q = qZ * qY * qX
    q0 = cos(psi_rad/2).*cos(theta_rad/2).*cos(phi_rad/2) ...
       + sin(psi_rad/2).*sin(theta_rad/2).*sin(phi_rad/2);
    q1 = cos(psi_rad/2).*cos(theta_rad/2).*sin(phi_rad/2) ...
       - sin(psi_rad/2).*sin(theta_rad/2).*cos(phi_rad/2);
    q2 = cos(psi_rad/2).*sin(theta_rad/2).*cos(phi_rad/2) ...
       + sin(psi_rad/2).*cos(theta_rad/2).*sin(phi_rad/2);
    q3 = sin(psi_rad/2).*cos(theta_rad/2).*cos(phi_rad/2) ...
       - cos(psi_rad/2).*sin(theta_rad/2).*sin(phi_rad/2);

    % Normalise (speval interpolation can drift slightly)
    qnorm = sqrt(q0.^2 + q1.^2 + q2.^2 + q3.^2);
    q0 = q0./qnorm;  q1 = q1./qnorm;
    q2 = q2./qnorm;  q3 = q3./qnorm;

    % --- Build 13-state matrix ---
    % [x, y, z, vx, vy, vz, m, q0, q1, q2, q3, p(rad/s),
       q_rate(rad/s)]
    % Note: ICLOCS states 10,11 (p,q) are in rad/s (SI torque/inertia
       dynamics)
    x13 = [x11(:,1:7), q0, q1, q2, q3, x11(:,10), x11(:,11)];

    % --- Accumulate ---
    reference.t     = [reference.t;     tt'];
    reference.x     = [reference.x;     x13];
    reference.u     = [reference.u;     u3];
```

```
228     reference.phase = [reference.phase; i*ones(N_export,1)];
229
230     reference.phase_transition_times(i)   = sol_i.t0;
231     reference.phase_transition_times(i+1) = sol_i.tf;
232 end
233
234 reference.fuel_used = best_cost;
235 reference.params.ms = 8;
236 reference.params.mp = 8.88;
237 reference.params.c  = 1488;
238 reference.params.n_phases = 4;
239
240 save('reference_trajectory.mat', 'reference');
241 fprintf('Reference trajectory exported: %d points, %.4f kg fuel\n',
    ...
242         length(reference.t), reference.fuel_used);
243
244 % Quick sanity check
245 assert(~any(isnan(reference.x(:))), 'NaN in exported states');
246 assert(~any(isnan(reference.u(:))), 'NaN in exported controls');
247 assert(all(abs(reference.t - sort(reference.t)) < 1e-9), 'Time not
    monotonic');
248 % Check monotonicity within each phase (tight tolerance)
249 % Phase boundaries can have tiny speval interpolation artefacts
    checked separately
250 for ph = 1:length(solution.phaseSol)
251     idx  = reference.phase == ph;
252     m_ph = reference.x(idx, 7);
253     assert(all(diff(m_ph) <= 1e-3), ...
254         sprintf('Mass increasing within phase %d (max rise = %.2e
            kg)', ...
255         ph, max(diff(m_ph))));
256 end
257
258 % Check phase-boundary jumps are not physically meaningful (< 10g is
    interpolation noise)
259 boundary_idx = find(diff(reference.phase) ~= 0);
260 for bi = boundary_idx'
261     m_jump = reference.x(bi+1, 7) - reference.x(bi, 7);
262     assert(m_jump <= 0.01, ...
263         sprintf('Physically large mass jump at phase boundary: +%.4f
            kg', m_jump));
264 end
265 fprintf('Mass monotonicity check passed.\n');
266 q_norms = sqrt(sum(reference.x(:,8:11).^2, 2));
267 assert(all(abs(q_norms - 1) < 1e-6), 'Quaternion norm error');
268 fprintf('Export sanity checks passed.\n');
```

```
269
270
271 %% Generate Figures
272 filepath = 'figures/test_16/';
273 if ~exist(filepath,'dir'), mkdir(filepath); end
274 n_phases = length(solution.phaseSol);
275 %% Figure: 3D trajectory (isometric view)
276 figure('Name','3D Trajectory', ...
277     'Units','inches','Position',[1 1 W_FULL 4.8], ...
278     'PaperPositionMode','auto');
279 hold on;
280
281 h_phase = gobjects(n_phases,1);
282 for i = 1:n_phases
283     sol = solution.phaseSol{i};
284     tt  = linspace(sol.t0, sol.tf, 500);
285     h_phase(i) = plot3(speval(sol,'X',1,tt), speval(sol,'X',2,tt), ...
286         speval(sol,'X',3,tt), '-', ...
287         'Color', phase_colors(i,:), 'LineWidth', 1.5);
288 end
289
290 xlabel('$X$ [m]'); ylabel('$Y$ [m]'); zlabel('$Z$ [m]');
291 title('Reference trajectory: 3D
    view','Interpreter','latex','FontSize',12);
292 grid on; box on; view(135,15);
293 xlim([-50 250]);
294 ylim([-250 50]);
295 zlim([0 350]);
296 axis equal;
297
298 ax = gca; ax.Position = [0.10, 0.18, 0.85, 0.78];
299
300 lgd = legend(h_phase, phase_names, ...
301     'Orientation','horizontal','Box','off','Interpreter','latex');
302 lgd.Units = 'normalized';
303 lgd.Position = [0.5 - lgd.Position(3)/2, 0.04, lgd.Position(3),
    lgd.Position(4)];
304
305 exportgraphics(gcf, fullfile(filepath,'fig_ref_trajectory_3d.pdf'),
    ...
306     'ContentType','vector');
307
308 %% Figure: Position vs time (X, Y, Z stacked)
309 figure('Name','Position vs Time', ...
310     'Units','inches','Position',[1 1 W_FULL 4.6], ...
311     'PaperPositionMode','auto');
312
```

```
tl = tiledlayout(3,1,'TileSpacing','compact','Padding','compact');

axis_labels = {'$X$ [m]', '$Y$ [m]', '$Z$ [m]'};
h_phase = gobjects(n_phases,1);

for k = 1:3
    nexttile; hold on;
    for i = 1:n_phases
        sol = solution.phaseSol{i};
        tt  = linspace(sol.t0, sol.tf, 500);
        h = plot(tt, speval(sol,'X',k,tt), '-', ...
            'Color', phase_colors(i,:), 'LineWidth', 1.5);
        if k == 1, h_phase(i) = h; end
    end
    ylabel(axis_labels{k});
    if k == 3
        xlabel('Time [s]');
        ylim([0 400]);
    else
        ylim([-300 300]);
    end
    grid on; box on;
end

title(tl,'Reference position vs
   time','Interpreter','latex','FontSize',12);

lgd = legend(h_phase, phase_names, ...
    'Orientation','horizontal','Box','off','Interpreter','latex');
lgd.Layout.Tile = 'south';

exportgraphics(gcf, fullfile(filepath,'fig_ref_position.pdf'), ...
    'ContentType','vector');
%% Figure: Velocity vs time (Vx, Vy, Vz stacked)
figure('Name','Velocity vs Time', ...
    'Units','inches','Position',[1 1 W_FULL 4.6], ...
    'PaperPositionMode','auto');

tl = tiledlayout(3,1,'TileSpacing','compact','Padding','compact');

axis_labels = {'$V_x$ [m/s]', '$V_y$ [m/s]', '$V_z$ [m/s]'};
h_phase = gobjects(n_phases,1);

for k = 1:3
    nexttile; hold on;
    for i = 1:n_phases
        sol = solution.phaseSol{i};
```

```
        tt  = linspace(sol.t0, sol.tf, 500);
        h = plot(tt, speval(sol,'X',k+3,tt), '-', ...
            'Color', phase_colors(i,:), 'LineWidth', 1.5);
        if k == 1, h_phase(i) = h; end
    end
    ylabel(axis_labels{k});
    if k == 3
        xlabel('Time [s]');
        ylim([-50 50]);
    else
        ylim([-15 15]);
    end
    grid on; box on;
end

title(tl,'Reference velocity vs
   time','Interpreter','latex','FontSize',12);

lgd = legend(h_phase, phase_names, ...
    'Orientation','horizontal','Box','off','Interpreter','latex');
lgd.Layout.Tile = 'south';

exportgraphics(gcf, fullfile(filepath,'fig_ref_velocity.pdf'), ...
    'ContentType','vector');
%% Figure: Mass vs time
figure('Name','Mass vs Time', ...
    'Units','inches','Position',[1 1 W_FULL 4.0], ...
    'PaperPositionMode','auto');
hold on;

h_phase = gobjects(n_phases,1);
for i = 1:n_phases
    sol = solution.phaseSol{i};
    tt  = linspace(sol.t0, sol.tf, 500);
    h_phase(i) = plot(tt, speval(sol,'X',7,tt), '-', ...
        'Color', phase_colors(i,:), 'LineWidth', 1.5);
end

yline(problem.phases{1}.states.xl(7), '--', 'No fuel line', ...
    'Color',[0.6 0 0], 'Interpreter','latex','FontSize',9, ...
    'LabelVerticalAlignment','bottom','HandleVisibility','off');
yline(problem.phases{1}.states.xu(7), '--', 'Max fuel line', ...
    'Color',[0.6 0 0], 'Interpreter','latex','FontSize',9, ...
    'LabelVerticalAlignment','top','HandleVisibility','off');

xlabel('Time [s]'); ylabel('Mass [kg]');
title('Reference mass vs time','Interpreter','latex','FontSize',12);
```

```
ylim([0 18]);
grid on; box on;

ax = gca; ax.Position = [0.10, 0.22, 0.85, 0.68];

lgd = legend(h_phase, phase_names, ...
    'Orientation','horizontal','Box','off','Interpreter','latex');
lgd.Units = 'normalized';
lgd.Position = [0.5 - lgd.Position(3)/2, 0.04, lgd.Position(3),
   lgd.Position(4)];

exportgraphics(gcf, fullfile(filepath,'fig_ref_mass.pdf'), ...
    'ContentType','vector');
%% Figure: Thrust vs time
figure('Name','Thrust vs Time', ...
    'Units','inches','Position',[1 1 W_FULL 4.0], ...
    'PaperPositionMode','auto');
hold on;

h_phase = gobjects(n_phases,1);
for i = 1:n_phases
    sol = solution.phaseSol{i};
    tt  = linspace(sol.t0, sol.tf, 500);
    h_phase(i) = plot(tt, speval(sol,'U',1,tt), '-', ...
        'Color', phase_colors(i,:), 'LineWidth', 1.5);
end

yline(problem.phases{1}.inputs.uu(1), '--', 'Max thrust', ...
    'Color',[0.6 0 0], 'LineWidth',0.8, ...
    'LabelVerticalAlignment','bottom',
       'LabelHorizontalAlignment','left', ...
    'Interpreter','latex', 'FontSize',9, 'HandleVisibility','off');

xlabel('Time [s]'); ylabel('Thrust [N]');
title('Reference thrust vs time','Interpreter','latex','FontSize',12);
grid on; box on;

ax = gca; ax.Position = [0.10, 0.22, 0.85, 0.68];

lgd = legend(h_phase, phase_names, ...
    'Orientation','horizontal','Box','off','Interpreter','latex');
lgd.Units = 'normalized';
lgd.Position = [0.5 - lgd.Position(3)/2, 0.04, lgd.Position(3),
   lgd.Position(4)];

exportgraphics(gcf, fullfile(filepath,'fig_ref_thrust.pdf'), ...
    'ContentType','vector');
```

```
449
450 %% Figure: Attitude angles vs time
451 figure('Name','Attitude vs Time', ...
452     'Units','inches','Position',[1 1 W_FULL 4.4], ...
453     'PaperPositionMode','auto');
454
455 tl = tiledlayout(2,1,'TileSpacing','compact','Padding','compact');
456
457 axis_labels = {'$\theta$ [deg]', '$\phi$ [deg]'};
458 state_idx   = [8, 9];
459 h_phase = gobjects(n_phases,1);
460
461 for k = 1:2
462     nexttile; hold on;
463     for i = 1:n_phases
464         sol = solution.phaseSol{i};
465         tt  = linspace(sol.t0, sol.tf, 500);
466         h = plot(tt, speval(sol,'X',state_idx(k),tt), '-', ...
467             'Color', phase_colors(i,:), 'LineWidth', 1.5);
468         if k == 1, h_phase(i) = h; end
469     end
470     yline(problem.phases{1}.states.xl(state_idx(k)), ':', ...
471         'Color',[0.6 0 0], 'HandleVisibility','off');
472     yline(problem.phases{1}.states.xu(state_idx(k)), ':', ...
473         'Color',[0.6 0 0], 'HandleVisibility','off');
474     ylabel(axis_labels{k});
475     if k == 2, xlabel('Time [s]'); end
476     ylim([-45 45]);
477     grid on; box on;
478 end
479
480 title(tl,'Reference attitude vs
    time','Interpreter','latex','FontSize',12);
481
482 lgd = legend(h_phase, phase_names, ...
483     'Orientation','horizontal','Box','off','Interpreter','latex');
484 lgd.Layout.Tile = 'south';
485
486 exportgraphics(gcf, fullfile(filepath,'fig_ref_attitude.pdf'), ...
487     'ContentType','vector');
488 %% Figure: Body rates vs time
489 figure('Name','Body Rates vs Time', ...
490     'Units','inches','Position',[1 1 W_FULL 4.4], ...
491     'PaperPositionMode','auto');
492
493 tl = tiledlayout(2,1,'TileSpacing','compact','Padding','compact');
494
```

```
axis_labels = {'$p$ [deg/s]', '$q$ [deg/s]'};
state_idx   = [10, 11];
h_phase = gobjects(n_phases,1);

for k = 1:2
    nexttile; hold on;
    for i = 1:n_phases
        sol = solution.phaseSol{i};
        tt  = linspace(sol.t0, sol.tf, 500);
        h = plot(tt, speval(sol,'X',state_idx(k),tt), '-', ...
            'Color', phase_colors(i,:), 'LineWidth', 1.5);
        if k == 1, h_phase(i) = h; end
    end
    yline(problem.phases{1}.states.xl(state_idx(k)), ':', ...
        'Color',[0.6 0 0], 'HandleVisibility','off');
    yline(problem.phases{1}.states.xu(state_idx(k)), ':', ...
        'Color',[0.6 0 0], 'HandleVisibility','off');
    ylabel(axis_labels{k});
    if k == 2, xlabel('Time [s]'); end
    grid on; box on;
end

title(tl,'Reference body rates vs
   time','Interpreter','latex','FontSize',12);

lgd = legend(h_phase, phase_names, ...
    'Orientation','horizontal','Box','off','Interpreter','latex');
lgd.Layout.Tile = 'south';

exportgraphics(gcf, fullfile(filepath,'fig_ref_rates.pdf'), ...
    'ContentType','vector');
%% Figure: Gimbal angles vs time
figure('Name','Gimbal vs Time', ...
    'Units','inches','Position',[1 1 W_FULL 4.4], ...
    'PaperPositionMode','auto');

tl = tiledlayout(2,1,'TileSpacing','compact','Padding','compact');

axis_labels = {'$\alpha$ [deg]', '$\beta$ [deg]'};
input_idx   = [2, 3];
h_phase = gobjects(n_phases,1);

for k = 1:2
    nexttile; hold on;
    for i = 1:n_phases
        sol = solution.phaseSol{i};
        tt  = linspace(sol.t0, sol.tf, 500);
```

```
        h = plot(tt, speval(sol,'U',input_idx(k),tt), '-', ...
            'Color', phase_colors(i,:), 'LineWidth', 1.5);
        if k == 1, h_phase(i) = h; end
    end
    yline(problem.phases{1}.inputs.ul(input_idx(k)), ':', ...
        'Color',[0.6 0 0], 'HandleVisibility','off');
    yline(problem.phases{1}.inputs.uu(input_idx(k)), ':', ...
        'Color',[0.6 0 0], 'HandleVisibility','off');
    ylabel(axis_labels{k});
    if k == 2, xlabel('Time [s]'); end
    grid on; box on;
end

title(tl,'Reference gimbal angles vs
   time','Interpreter','latex','FontSize',12);

lgd = legend(h_phase, phase_names, ...
    'Orientation','horizontal','Box','off','Interpreter','latex');
lgd.Layout.Tile = 'south';

exportgraphics(gcf, fullfile(filepath,'fig_ref_gimbal.pdf'), ...
    'ContentType','vector');

%% LOCAL FUNCTION: perturbGuess
%  Generates a physically sensible random perturbation of the
%  initial guess. Called for attempts 2 through N_starts.


function guess_out = perturbGuess(guess_in, problem)
% PERTURBGUESS  Create a randomly perturbed initial guess
%
% Perturbation strategy:
%   1. Phase transition times - varied within their allowed bounds
%   2. State trajectory endpoints - varied around the mission
   waypoints
%   3. Thrust input guesses - randomised within [T_min, 0.8*T_max]
%
% Gimbal angles (alpha, beta) are kept near zero as large angle
   guesses
% tend to push IPOPT toward infeasible regions.

guess_out = guess_in;   % copy structure, then modify fields
n_phases  = length(problem.phases);



% 1. PERTURB PHASE TRANSITION TIMES
```

```
% ------------------------------------------------------------------
%  t = [0,   t2,     t3,       t4,     t5]
%        |    |       |         |       |
%      ph1 end  ph2 end  ph3 end  ph4 end

%  Constraints:
%    t(1) = 0           (fixed)
%    t(2) in [4, 18]    (phase 1 ascent: minimum 4s, maximum 18s)
%    t(3) = t(2) + 5    (hover is exactly 5s      fixed by boundary
    constraint)
%    t(4) in [t(3)+4, 38]  (phase 3 descent: minimum 4s)
%    t(5) = t(4) + 5    (landing is exactly 5s      fixed by boundary
    constraint)


t2 = 4 + rand * 14;                       % phase 1 end: 4 to 18 s
t3 = t2 + 5;                               % hover always 5 s
max_t4 = min(38, t3 + 20);                % cap total mission at ~43 s
t4 = t3 + 4 + rand * (max_t4 - t3 - 4); % phase 3: at least 4 s
t5 = t4 + 5;                               % landing always 5 s

guess_out.mp.time = [0, t2, t3, t4, t5];

% Update each phase's individual t0 and tf to match
guess_out.phases{1}.t0 = 0;     guess_out.phases{1}.tf = t2;
guess_out.phases{2}.t0 = t2;    guess_out.phases{2}.tf = t3;
guess_out.phases{3}.t0 = t3;    guess_out.phases{3}.tf = t4;
guess_out.phases{4}.t0 = t4;    guess_out.phases{4}.tf = t5;


% 2. PERTURB STATE TRAJECTORY GUESSES
% ------------------------------------------------------------------
%  guess.phases{i}.states is a [2 x 11] matrix:
%    row 1 = state at phase start
%    row 2 = state at phase end
%
%  Vary the intermediate waypoints around the target positions.
%  Phase 1 start is fixed (launch from rest at origin)
%  Phase 4 end is fixed (landed, zero velocity)
% ------------------------------------------------------------------
m0    = problem.phases{1}.states.x0(7);    % initial mass 16.88 kg
m_min = problem.phases{1}.states.xl(7);    % dry mass 8 kg

% Perturbation scales  [X, Y, Z, Vx, Vy, Vz, m, theta, phi, p, q]
pos_s = 30;    % metres - spatial perturbation
vel_s = 3;     % m/s    - velocity perturbation
ang_s = 8;     % deg    - angle perturbation
```

```
rat_s = 2;     % deg/s - angular rate perturbation



% Apex waypoint (originally hover entry at 100, -100, 300)
% Now used as ascent end / descent start
ep_apex = [100 + pos_s*(rand-0.5), ...
    -100 + pos_s*(rand-0.5), ...
    300 + 10*(rand-0.5), ...
    vel_s*(rand-0.5), vel_s*(rand-0.5), vel_s*(rand-0.5), ...
    m0 * (0.65 + 0.25*rand), ...
    ang_s*(rand-0.5), ang_s*(rand-0.5), ...
    rat_s*(rand-0.5), rat_s*(rand-0.5)];

% Pre-landing waypoint (200, -200, 5)
ep_preland = [200 + pos_s*(rand-0.5), ...
    -200 + pos_s*(rand-0.5), ...
    5 + 2*rand, ...
    vel_s*(rand-0.5), vel_s*(rand-0.5), -1*rand, ...
    m0 * (0.40 + 0.25*rand), ...
    ang_s*(rand-0.5), ang_s*(rand-0.5), ...
    rat_s*(rand-0.5), rat_s*(rand-0.5)];

% Hover Disabled
% Phase 1 end / Phase 2 start      near hover target (100, -100, 300)
ep1 = [100 + pos_s*(rand-0.5), ...
      -100 + pos_s*(rand-0.5), ...
       300 + 10*(rand-0.5), ...          % Z: tighter, near 300 m
       vel_s*(rand-0.5), vel_s*(rand-0.5), vel_s*(rand-0.5), ...
       m0 * (0.65 + 0.25*rand), ...      % mass: 65-90% of initial
       ang_s*(rand-0.5), ang_s*(rand-0.5), ...
       rat_s*(rand-0.5), rat_s*(rand-0.5)];

% Phase 2 end / Phase 3 start      still near hover (small drift from
    ep1)
ep2 = ep1 + [5*(rand-0.5), 5*(rand-0.5), 0.3*(rand-0.5), ...
             1*(rand-0.5), 1*(rand-0.5), 1*(rand-0.5), ...
             -0.05*rand, ...             % slight mass decrease
                during hover
             2*(rand-0.5), 2*(rand-0.5), rat_s*(rand-0.5),
                rat_s*(rand-0.5)];

% Phase 3 end / Phase 4 start      near pre-landing target (200,
    -200, 5)
ep3 = [200 + pos_s*(rand-0.5), ...
      -200 + pos_s*(rand-0.5), ...
         5 + 2*rand, ...                 % Z slightly above 5 m
```

```
673         vel_s*(rand-0.5), vel_s*(rand-0.5), -1*rand, ... % slight
                downward velocity
674         m0 * (0.40 + 0.25*rand), ...       % mass: 40-65% of initial
675         ang_s*(rand-0.5), ang_s*(rand-0.5), ...
676         rat_s*(rand-0.5), rat_s*(rand-0.5)];
677
678
679 % original zero-state landing guess
680 guess_out.phases{1}.states(2,:) = ep1;
681 guess_out.phases{2}.states(1,:) = ep1;
682 guess_out.phases{2}.states(2,:) = ep2;
683 guess_out.phases{3}.states(1,:) = ep2;
684 guess_out.phases{3}.states(2,:) = ep3;
685 guess_out.phases{4}.states(1,:) = ep3;
686
687 % Clip all mass guesses to physically valid range [m_dry, m_initial]
688 for i = 1:n_phases
689     guess_out.phases{i}.states(:,7) = ...
690         max(m_min, min(m0, guess_out.phases{i}.states(:,7)));
691 end
692
693
694 % 3. PERTURB THRUST INPUT GUESSES
695 % ------------------------------------------------------------------
696 %  Thrust is the primary driver of fuel consumption, so varying it
697 %  widely is important for exploring different trajectory shapes.
698 %  Angles are kept small to avoid sending IPOPT into bad territory.
699 % ------------------------------------------------------------------
700 T_max     = problem.phases{1}.inputs.uu(1);     % 3200 N
701 T_min_val = problem.phases{1}.inputs.ul(1);     % 32 N  (1% of max)
702
703 for i = 1:n_phases
704     T_s = T_min_val + rand * (0.75 * T_max - T_min_val);
705     T_e = T_min_val + rand * (0.75 * T_max - T_min_val);
706     guess_out.phases{i}.inputs(1,1) = T_s;
707     guess_out.phases{i}.inputs(2,1) = T_e;
708
709     guess_out.phases{i}.inputs(1,2) = 2*(rand-0.5);   % alpha start
710     guess_out.phases{i}.inputs(2,2) = 2*(rand-0.5);   % alpha end
711     guess_out.phases{i}.inputs(1,3) = 2*(rand-0.5);   % beta start
712     guess_out.phases{i}.inputs(2,3) = 2*(rand-0.5);   % beta end
713 end
714
715 end
```

Listing F.1: `main_ThreeDimRocketFourPhase.m` — top-level driver sequencing the four ICLOCS2 phases and writing the locked reference trajectory to `reference_trajectory.mat` (Section 3.2).

F.2 Online Guidance: SCP-MPC with Lossless Convexification

The online layer compiles the SOCP sub-problem once via YALMIP and solves it at runtime with ECOS, wrapped in an SCP outer loop with trust-region adaptation (Section 3.5). The three routines below implement, in order, the parametric SOCP, the SCP outer loop, and the top-level MPC wrapper called by the dispatcher.

F.2.1 SOCP Builder

```
function optimizer_obj = build_socp(params)
% BUILD_SOCP  Compile the parametric SOCP for Mode B once at startup.
%
% Returns a YALMIP optimizer object that mpc_solve.m calls with numeric
% values per SCP iteration. Compiling once and re-using the handle is
% what keeps the per-call solve under 50 ms.
%
% --- Decision variables (non-dim) ---
%   x        7 x N       state trajectory [r; v; m]
%   u        3 x N       thrust vector T_vec
%   Gamma    1 x N       LCvx thrust-magnitude slack  (||T_vec|| <= Gamma)
%   sigma    1 x 1       free-final-time dilation (sigma = t_f / U_T)
%   nu       7 x (N-1)   virtual control (dynamics defect, soft)
%   s_delta  1 x N       trust-region slack per node (soft)
%
% --- Runtime parameters (in this order) ---
%   Abar          7 x 7 x (N-1)   discrete state Jacobian
%   Bbar          7 x 4 x (N-1)   discrete control Jacobian (start)
%   Cbar          7 x 4 x (N-1)   discrete control Jacobian (end, FOH)
%   Sigbar        7 x (N-1)       sigma column
%   zbar          7 x (N-1)       affine remainder
%   x0            7 x 1           initial condition (predicted state)
%   x_target      6 x 1           [r_target; v_target] (mass is FREE)
%   x_prev        7 x N           trust-region centre (state iterate)
%   u_prev        3 x N           trust-region centre (control iterate)
```

```
%   Gamma_prev    1 x N           trust-region centre (slack iterate)
%   sigma_low     1 x 1           lower bound on sigma
%   sigma_high    1 x 1           upper bound on sigma
%   w_t           1 x 1           time-penalty weight (0 landing, 1
    descent)
%
% --- Outputs (returned by optimizer_obj(...) call) ---
%   {x, u, Gamma, sigma, nu, s_delta}
%
% --- Constants baked in at compile time ----
%   T_min_nd, T_max_nd      thrust bounds (non-dim)
%   m_dry_nd                dry-mass floor (non-dim)
%   r_pad_nd                landing-pad position (non-dim)
%   gamma_gs                glideslope half-angle (rad)
%   delta_max               trust-region radius (non-dim)
%   w_nu, w_delta           virtual-control & trust-region weights
%
% --- Constraints ---
%   (1) Initial:         x(:,1) == x0
%   (2) Terminal:        x(1:6, N) == x_target            (mass free)
%   (3) Dynamics FOH:    x(:,k+1) == Abar*x_k + Bbar*u_aug_k +
    Cbar*u_aug_{k+1} + Sigbar*sigma + zbar + nu_k
%                        where u_aug = [T_vec; Gamma]
%   (4) LCvx SOC:        ||u(:,k)|| <= Gamma(k)
%   (5) LCvx box:        T_min_nd <= Gamma(k) <= T_max_nd
%   (6) Glideslope SOC: ||x(1:2,k) - r_pad(1:2)|| <= tan(gamma_gs) *
    (x(3,k) - r_pad(3))
%   (7) Subsurface:      x(3,k) >= r_pad(3)
%   (8) Mass floor:      x(7,k) >= m_dry_nd
%   (9) Sigma bounds:    sigma_low <= sigma <= sigma_high
%  (10) Trust region:    ||[x_k - x_prev_k; u_k - u_prev_k; Gamma_k -
    Gamma_prev_k]||
%                        <= delta_max + s_delta(k);   s_delta(k) >= 0
% --- Cost ---
%   J = -x(7,N)                    (maximise final mass = minimise
    propellant)
%       + w_nu * ||nu||_1
%       + w_delta * sum(s_delta)
%       + w_t * sigma
% Why minimise -m_f (and not sum(Gamma)) for fuel:
%   The true fuel cost is sigma * sum(Gamma) * dtau / c, which is
    BILINEAR
%   in sigma and Gamma (not convex). Minimising -m_f is convex (m_f
    is a
%   state variable) and exactly equivalent because the dynamics
    conserve
%   mass: m_f = m_0 - propellant_burned.
```

```
%
% CALLED BY: mpc_solve.m (once at startup, stored in
    params.socp_optimizer)
% CALLS:     none (uses YALMIP + ECOS)
% L. White, University of Sheffield, May 2026



    %  Step 1 - Extract sizes and bake-in constants from params

    N = params.scp.N;

    T_min_nd = params.Tmin / params.U_F;         % LCvx box lower
    T_max_nd = params.Tmax / params.U_F;         % LCvx box upper
    m_dry_nd = params.ms   / params.U_M;         % dry-mass floor
    r_pad_nd = params.r_pad / params.U_L;        % landing pad
       (non-dim)
    gamma_gs = params.gamma_gs;                  % radians
    tan_gs   = tan(gamma_gs);                    % bake the tan once

    delta_max = params.scp.delta_max;            % trust-region radius
    w_nu      = params.scp.w_nu;                 % virtual-control
       penalty
    w_delta   = params.scp.w_delta;              % trust-region slack
       penalty



    %  Step 2 - Clear YALMIP and declare decision variables
    yalmip('clear');

    x       = sdpvar(7, N, 'full');              % state trajectory
    u       = sdpvar(3, N, 'full');              % thrust vector T_vec
    Gamma   = sdpvar(1, N, 'full');              % LCvx slack
    sigma   = sdpvar(1, 1);                      % time dilation
    nu      = sdpvar(7, N-1, 'full');            % virtual control
    s_delta = sdpvar(1, N, 'full');              % trust-region slack


    %  Step 3 - Declare runtime-parameter sdpvars
    Abar       = sdpvar(7, 7, N-1, 'full');
    Bbar       = sdpvar(7, 4, N-1, 'full');
    Cbar       = sdpvar(7, 4, N-1, 'full');
    Sigbar     = sdpvar(7, N-1, 'full');
    zbar       = sdpvar(7, N-1, 'full');

    x0         = sdpvar(7, 1);
```

```
    x_target    = sdpvar(6, 1);                    % [r_target; v_target]
    x_prev      = sdpvar(7, N, 'full');
    u_prev      = sdpvar(3, N, 'full');
    Gamma_prev  = sdpvar(1, N, 'full');
    sigma_low   = sdpvar(1, 1);
    sigma_high  = sdpvar(1, 1);
    w_t         = sdpvar(1, 1);


    %  Step 4 - Build constraints
    C = [];

    % --- (1) Initial condition ---
    C = [C; x(:,1) == x0];

    % --- (2) Terminal: pin r and v, leave m free ---
    C = [C; x(1:6, N) == x_target];

    % --- (3) Dynamics (FOH on augmented control u_aug = [T_vec;
        Gamma]) --
    for k = 1:N-1
        u_aug_k   = [u(:,k);   Gamma(k)];
        u_aug_kp1 = [u(:,k+1); Gamma(k+1)];
        C = [C; x(:,k+1) == Abar(:,:,k) * x(:,k) ...
                          + Bbar(:,:,k) * u_aug_k ...
                          + Cbar(:,:,k) * u_aug_kp1 ...
                          + Sigbar(:,k) * sigma ...
                          + zbar(:,k) ...
                          + nu(:,k)];
    end

    % --- (4) LCvx SOC: ||T_vec_k|| <= Gamma_k ---
    %
    % --- (5) LCvx box: T_min_nd <= Gamma_k <= T_max_nd ---
    for k = 1:N
        C = [C; norm(u(:,k)) <= Gamma(k)];
        C = [C; T_min_nd <= Gamma(k) <= T_max_nd];
    end

    % --- (6) Glideslope SOC ---
    %   ||r_xy - r_pad_xy|| <= tan(gamma_gs) * (r_z - r_pad_z)
    for k = 1:N
        C = [C; norm(x(1:2,k) - r_pad_nd(1:2)) ...
                    <= tan_gs * (x(3,k) - r_pad_nd(3))];
    end

    % --- (7) Subsurface ---
```

```
154     for k = 1:N
155         C = [C; x(3,k) >= r_pad_nd(3)];
156     end
157
158     % --- (8) Mass floor ---
159     % Without the solver can plan with m -> 0, making T/m blow up
160     % at the next linearisation. Hard lower bound at dry mass is safe.
161     for k = 1:N
162         C = [C; x(7,k) >= m_dry_nd];
163     end
164
165     % --- (9) Sigma bounds ---
166     C = [C; sigma_low <= sigma <= sigma_high];
167
168     % --- (10) Trust region SOC (per node) ---
169     for k = 1:N
170         diff_vec = [x(:,k)     - x_prev(:,k);
171                     u(:,k)     - u_prev(:,k);
172                     Gamma(k)   - Gamma_prev(k)];
173         C = [C; norm(diff_vec) <= delta_max + s_delta(k)];
174         C = [C; s_delta(k) >= 0];
175     end
176     % --- (8.5) Monotonic vertical descent (anti-climb) ---
177
178     for k = 2:N
179         C = [C; x(6,k) <= 0];
180     end
181
182     % Thrust regularisation: norm() is SOC-native (YALMIP optimizer
        can
183     % reformulate at compile time), unlike sum(  .^2) which is
        quadratic
184     % and rejected by ECOS at runtime. Same minimiser set as the
        squared
185     % version, just a milder penalty rate at the same weight     bump
        the
186     % weight up correspondingly.
187     %
188     % Effect: prevents bang-coast-bang by penalising the L2 norm of
        the
189     %    trajectory. Bang profiles concentrate    at the bounds (high
        norm);
190     % smooth profiles spread    near the average (lower norm). The
        optimiser
191     % trades a small fuel-suboptimality for a much smoother
        trajectory.
192     %
```

```
193     % Same convex curvature trick fixes the cone-vertex degeneracy
            that
194     % was causing ECOS exit_code 4 on opening calls.
195     w_thrust_reg = 0.02;        % tune in [0.01, 0.2]      too high
            makes it lazy
196
197     J = -x(7, N) ...
198         + w_nu          * norm(nu(:), 1) ...
199         + w_delta       * sum(s_delta) ...
200         + w_t           * sigma ...
201         + w_thrust_reg * norm(Gamma);
202
203
204     %  Step 6 - Solver options
205     options = sdpsettings('solver',  'ecos', ...
206                           'verbose', 0,       ...
207                           'cachesolvers', 1);
208
209
210     %  Step 7 - Compile the optimizer
211     params_in = {Abar, Bbar, Cbar, Sigbar, zbar, ...
212                  x0,   x_target, x_prev,  u_prev, Gamma_prev, ...
213                  sigma_low, sigma_high, w_t};
214
215     vars_out  = {x, u, Gamma, sigma, nu, s_delta};
216
217     fprintf('build_socp: compiling SOCP (N=%d, %d intervals)...\n',
            N, N-1);
218     t_compile = tic;
219     optimizer_obj = optimizer(C, J, options, params_in, vars_out);
220     t_compile = toc(t_compile);
221     fprintf('build_socp: compile finished in %.2f s.\n', t_compile);
222 end
```

Listing F.2: `build_socp.m` — parametric SOCP construction: LCvx thrust constraint pair, FOH dynamics linearisation, trust-region cone, and SCP cost (Section 3.5, Table 3.2).

F.2.2 SCP Outer Loop

```
1 function [x_new, u_new, Gamma_new, sigma_new, info] = scp_iteration(
      ...
2     x_bar, u_bar, Gamma_bar, sigma_bar, ...
3     x0, x_target, sigma_low, sigma_high, w_t, ...
4     optimizer_obj, params)
5 % SCP_ITERATION  One outer-loop iteration of Successive Convex
      Programming.
```

```
%
% Step sequence:
%   1. Linearise the 3-DoF dynamics about (x_bar, u_bar, Gamma_bar, sigma_bar)
%      to produce the per-interval discretised Jacobians.
%   2. Pack them into the 3D-array form the compiled SOCP expects.
%   3. Call the compiled YALMIP optimizer with x_prev = x_bar etc.
%   4. Unpack the solution and compute convergence metrics:
%        - nu_norm    : inf-norm of virtual control across all intervals
%        - delta_norm : inf-norm of change in (x, u, Gamma) from iterate k
%        - lcvx_slack : max |Gamma - ||u_T||| in Newtons
%
% --- Inputs (all non-dim unless noted) ---
%   x_bar        - state iterate           7 x N
%   u_bar        - thrust vector iterate   3 x N
%   Gamma_bar    - LCvx slack iterate      1 x N
%   sigma_bar    - time dilation iterate   scalar
%   x0           - initial condition       7 x 1
%   x_target     - target [r; v]           6 x 1
%   sigma_low    - sigma lower bound       scalar  (= sigma_high for landing)
%   sigma_high   - sigma upper bound       scalar
%   w_t          - time penalty weight     scalar  (0 landing, 1 descent)
%   optimizer_obj - compiled YALMIP optimizer from build_socp.m
%   params       - vehicle params struct
%
% --- Outputs ---
%   x_new, u_new, Gamma_new, sigma_new - new iterate (non-dim)
%   info         - struct with the following fields:
%                    .exit_code   YALMIP solver exit code (0 = optimal)
%                    .nu_norm     max ||nu_k|| over intervals [non-dim]
%                    .delta_norm  max change in iterate (inf-norm) [non-dim]
%                    .lcvx_slack  max |Gamma - ||u_T|||             [N, SI]
%                    .s_delta_max max trust-region slack            [non-dim]
%                    .nu          full virtual-control matrix       7 x (N-1)
%                    .s_delta     full trust-region slack vector    1 x N
%                    .t_lin       linearisation wall-clock          [s]
%                    .t_solve     SOCP solve wall-clock             [s]
%
```

```
% CALLED BY: mpc_solve.m, test_modeB_step4.m
% CALLS:     linearise_dynamics.m, the compiled optimizer
%
% L. White, University of Sheffield, May 2026

    %% Step 1 - Linearise about current iterate
    t0 = tic;
    [Abar_c, Bbar_c, Cbar_c, Sigbar_c, zbar_c] = ...
        linearise_dynamics(x_bar, u_bar, Gamma_bar, sigma_bar,
           params);
    t_lin = toc(t0);

    %% Step 2 - Repack cells into 3D arrays for the optimizer
    Abar3D   = cat(3, Abar_c{:});      % 7 x 7 x (N-1)
    Bbar3D   = cat(3, Bbar_c{:});      % 7 x 4 x (N-1)
    Cbar3D   = cat(3, Cbar_c{:});      % 7 x 4 x (N-1)
    Sigbar2D = [Sigbar_c{:}];          % 7 x (N-1)
    zbar2D   = [zbar_c{:}];            % 7 x (N-1)

    %% Step 3 - Call the compiled optimizer
    % Parameter order MUST match build_socp.m
    param_values = {Abar3D, Bbar3D, Cbar3D, Sigbar2D, zbar2D, ...
                    x0, x_target, x_bar, u_bar, Gamma_bar, ...
                    sigma_low, sigma_high, w_t};

    t0 = tic;
    [sol, exit_code] = optimizer_obj(param_values);
    t_solve = toc(t0);

    if iscell(exit_code), exit_code = exit_code{1}; end

    %% Step 4 - Unpack solution
    % If the solver failed, return the previous iterate untouched and
    % flag a large nu_norm to force the outer loop to bail / fall back
    if exit_code ~= 0
        x_new     = x_bar;
        u_new     = u_bar;
        Gamma_new = Gamma_bar;
        sigma_new = sigma_bar;
        info = struct('exit_code', exit_code, ...
                      'nu_norm',    inf, ...
                      'delta_norm', inf, ...
                      'lcvx_slack', inf, ...
                      's_delta_max', inf, ...
                      'nu',         [], ...
                      's_delta',    [], ...
                      't_lin',      t_lin, ...
```

```
                    't_solve',    t_solve);
        return;
    end

    x_new       = sol{1};       % 7 x N
    u_new       = sol{2};       % 3 x N
    Gamma_new   = sol{3};       % 1 x N
    sigma_new   = sol{4};       % scalar
    nu_sol      = sol{5};       % 7 x (N-1)
    s_delta_sol = sol{6};       % 1 x N

    %% Step 5 - Convergence metrics
    % nu_norm: largest dynamics-defect among intervals. Should -> 0
       as the
    % iterate stops moving and the linearisation becomes
       self-consistent.
    nu_norm = max(vecnorm(nu_sol));       % scalar, inf-norm across k

    % delta_norm: largest per-node change in (x, u, Gamma) from
       iterate.
    % Use inf-norm of the stacked difference vector
    delta_per_node = zeros(1, params.scp.N);
    for k = 1:params.scp.N
        d = [x_new(:,k)   - x_bar(:,k);
             u_new(:,k)   - u_bar(:,k);
             Gamma_new(k) - Gamma_bar(k)];
        delta_per_node(k) = norm(d, inf);
    end
    delta_norm = max(delta_per_node);

    % LCvx slack in SI units (Newtons)
    lcvx_slack_nd = max(abs(Gamma_new - vecnorm(u_new)));
    lcvx_slack    = lcvx_slack_nd * params.U_F;     % convert to N

    s_delta_max = max(s_delta_sol);

    %% Step 6 - Pack info
    info = struct( ...
        'exit_code',   exit_code, ...
        'nu_norm',     nu_norm, ...
        'delta_norm',  delta_norm, ...
        'lcvx_slack',  lcvx_slack, ...
        's_delta_max', s_delta_max, ...
        'nu',          nu_sol, ...
        's_delta',     s_delta_sol, ...
        't_lin',       t_lin, ...
        't_solve',     t_solve);
```

```
end
```

Listing F.3: `scp_iteration.m` — SCP outer loop: linearise about the current iterate, solve the parametric SOCP via ECOS, and evaluate convergence on the virtual-control norm $\|\nu\|_\infty$, trust-region step $\|\Delta\|_\infty$, and fuel cost (Section 3.5).

F.2.3 MPC Wrapper

```
function [T_cmd, n_hat_cmd, sol_out, rejection_count_new, ...
    fallback_flag] = ...
     mpc_solve(x_hat, t_now, target_state, sigma_bounds_s, ...
               prev_sol, rejection_count, params)
% MPC_SOLVE  Top-level Mode B entry. One MPC call -> one command + trajectory.
%
% Called by the Guidance Subsystem at the guidance rate (1-2 Hz). Wires
% together the entire Mode B pipeline:
%
%   1. Slice the 7-state translational [r; v; m] from the 17-state MEKF estimate
%   2. State predictor: project forward by params.t_plan_delay
%   3. Warm start: from reference (first call in phase) or shift (within phase)
%   4. SCP outer loop until cost stabilises or i_max iterations hit
%   5. Re-dimensionalise the converged trajectory
%   6. Assemble sol_out struct (full schema, see below)
%   7. Extract first-step command via extract_command.m
%   8. Validity gate via validity_check.m -> sets fallback_flag if needed
%
% --- Inputs ---
%   x_hat             17 x 1   MEKF state estimate [SI]
%                                (1:3)=r [m], (4:6)=v [m/s], (7:10)=q,
%                                (11:13)=b_g, (14:16)=b_a, (17)=mass [kg]
%   t_now             scalar   current mission time [s]
%   target_state      6 x 1    [r_target(3); v_target(3)] in SI
%                                (mass left free for the SOCP to minimise)
%   sigma_bounds_s    1 x 2    [sigma_low, sigma_high] in seconds
%                                landing: [3, 3]   descent: [5, 30]
%   prev_sol          struct   previous accepted solution OR []
%   rejection_count   integer  consecutive Mode B rejections (this phase)
```

```
%   params            struct   vehicle params (must contain
    .socp_optimizer
%                              -- compiled at startup by build_socp.m)
%
% --- Outputs ---
%   T_cmd                 scalar   first-step thrust magnitude [N]
%   n_hat_cmd             3 x 1    first-step thrust direction
    (inertial)
%   sol_out               struct   full solution (see field list below)
%   rejection_count_new   integer  updated rejection counter
%   fallback_flag         bool     true if max_rejections reached
%                                  (caller should switch to Mode A)
%
% --- sol_out schema ---
%   .x_k              7 x N   (non-dim)  state trajectory
%   .u_k              3 x N   (non-dim)  thrust vector
%   .Gamma_k          1 x N   (non-dim)  LCvx slack
%   .sigma            scalar  (non-dim)  time dilation
%   .x_target         6 x 1   (non-dim)  target supplied to SCP
%   .t_grid_si        1 x N   [s]        dimensional time stamps
%   .T_k_si           1 x N   [N]        thrust magnitudes
%   .n_hat_k          3 x N   (unit)     thrust directions
%   .lcvx_slack_N     scalar  [N]        max LCvx slack across nodes
%   .nu_norm          scalar             max ||nu_k|| from final iter
%   .exit_code        scalar             YALMIP code from final iter
%   .iter             scalar             # SCP iterations actually used
%   .iter_history     struct array       per-iter diagnostics
%   .valid            bool               from validity_check
%   .fault_code       scalar             from validity_check
%   .t_solve_total    scalar  [s]        wall-clock for the whole call
%
% CALLED BY: Simulink guidance dispatcher (or test scripts)
% CALLS:     state_predictor, warm_start, scp_iteration,
%            extract_command, validity_check, nondim_state
% L. White, University of Sheffield, May 2026

    t_start = tic;

    %% Step 1 - slice the 7-state translational from the 17-state EKF
    x7_si = [x_hat(1:6); x_hat(17)];           % [r; v; m] in SI

    %% Step 2 - state predictor: project forward by t_plan_delay
    t_apply = t_now + params.t_plan_delay;
    x0_si = state_predictor(x7_si, t_now, t_apply, prev_sol, params);
    x0_nd = nondim_state(x0_si, params, 'to_nd');          % 7 x 1,
       non-dim

```

```
    %% Step 3 - determine the phase from t_apply (selects warm-start
       segment)
    phase_id = identify_phase(t_apply,
       params.reference.phase_transition_times);

    %% Step 4 - warm start
    %   - first call in this phase, OR a different phase than
       prev_sol, OR
    %     no prev_sol available  ->  warm-start from reference
    %   - else                   ->  shift (deferred to Step 8)
    need_reference_warmstart = isempty(prev_sol) ...
        || ~isfield(prev_sol, 'phase_id') ...
        || prev_sol.phase_id ~= phase_id;


    if need_reference_warmstart
        phase_end_t =
           params.reference.phase_transition_times(phase_id + 1);
        ws_start_t  = t_apply;
        ws_end_t    = min(phase_end_t, t_apply + sigma_bounds_s(2));

        if ws_end_t <= ws_start_t + 1e-3
            ws_end_t = ws_start_t + max(sigma_bounds_s(2), 0.5);
        end

        [x_bar, u_bar, Gamma_bar, sigma_bar] = warm_start( ...
            'reference', params, ws_start_t, ws_end_t, []);
    else
        % 'shift' mode (within-phase re-plans)
        [x_bar, u_bar, Gamma_bar, sigma_bar] = warm_start( ...
            'shift', params, [], [], prev_sol);
    end

    % Sigma warm-start: applies to BOTH reference and shift modes.
    % Use the previous converged sigma if available; otherwise
       cold-start
    % at the midpoint of the bounds. Clamp to the bounds either way.
    sigma_bounds_nd = sigma_bounds_s / params.U_T;
    if ~isempty(prev_sol) && isfield(prev_sol, 'sigma') &&
       ~isempty(prev_sol.sigma)
        sigma_bar = prev_sol.sigma;
    else
        sigma_bar = mean(sigma_bounds_s) / params.U_T;
    end
    sigma_bar = max(sigma_bounds_nd(1), min(sigma_bounds_nd(2),
       sigma_bar));

```

```
113     %% Step 5 - boundary conditions for the SOCP
114     % x0_nd is the predicted (post-100ms) initial condition.
115     % x_target is target_state non-dimensionalised.
116     r_target_nd = target_state(1:3) / params.U_L;
117     v_target_nd = target_state(4:6) / params.U_V;
118     x_target_nd = [r_target_nd; v_target_nd];                % 6 x 1
119
120     sigma_low_nd  = sigma_bounds_s(1) / params.U_T;
121     sigma_high_nd = sigma_bounds_s(2) / params.U_T;
122
123     % Time-penalty weight: 0 for landing (fixed t_f), 1 for descent
124     if abs(sigma_bounds_s(1) - sigma_bounds_s(2)) < 1e-6
125         w_t_val = params.scp.w_t_land;                       % fixed
            sigma
126     else
127         w_t_val = params.scp.w_t_desc;                       % free
            sigma
128     end
129
130     %% Step 6 - SCP outer loop (cost-based termination)
131     i_max    = params.scp.i_max;
132     nu_tol   = params.scp.nu_tol;
133     if isfield(params.scp, 'cost_tol')
134         cost_tol = params.scp.cost_tol;
135     else
136         cost_tol = 1e-4;                                     % 0.1 g
            fuel
137     end
138
139     iter_hist(i_max) = struct('nu_norm',[],'delta_norm',[], ...
140                               'lcvx_slack',[],'fuel_kg',[], ...
141                               't_lin',[],'t_solve',[],'exit_code',[]);
142     prev_fuel_kg = inf;
143     converged = false;
144     iters_used = 0;
145
146     for i = 1:i_max
147         [x_new, u_new, Gamma_new, sigma_new, info] = scp_iteration(
            ...
148             x_bar, u_bar, Gamma_bar, sigma_bar, ...
149             x0_nd, x_target_nd, sigma_low_nd, sigma_high_nd, w_t_val,
                ...
150             params.socp_optimizer, params);
151
152         fuel_kg = (1 - x_new(7,end)) * params.U_M;           % m0 -
            m_f
153
```

```
154         iter_hist(i).nu_norm    = info.nu_norm;
155         iter_hist(i).delta_norm = info.delta_norm;
156         iter_hist(i).lcvx_slack = info.lcvx_slack;
157         iter_hist(i).fuel_kg    = fuel_kg;
158         iter_hist(i).t_lin      = info.t_lin;
159         iter_hist(i).t_solve    = info.t_solve;
160         iter_hist(i).exit_code  = info.exit_code;
161
162         iters_used = i;
163
164         % Bail if the solver couldn't return a valid solution at all
165         if info.exit_code ~= 0
166             break;
167         end
168
169         % Update the iterate
170         x_bar     = x_new;
171         u_bar     = u_new;
172         Gamma_bar = Gamma_new;
173         sigma_bar = sigma_new;
174
175         % Cost-based termination (primary) gated on nu being small
176         if i > 1 && info.nu_norm < nu_tol
177             cost_change = abs(fuel_kg - prev_fuel_kg);
178             if cost_change < cost_tol
179                 converged = true;
180                 break;
181             end
182         end
183         prev_fuel_kg = fuel_kg;
184     end
185
186     iter_hist = iter_hist(1:iters_used);                    % trim
        unused
187
188     %% Step 7 - re-dimensionalise and assemble sol_out
189     % Build dimensional time grid: nodes equally spaced between
        t_apply
190     % and t_apply + sigma_si.
191     sigma_si = sigma_new * params.U_T;
192     t_grid_si = linspace(t_apply, t_apply + sigma_si, params.scp.N);
193
194     % Thrust magnitudes and unit directions per node (SI)
195     u_si = u_new * params.U_F;                              % 3 x
        N, [N]
196     T_k_si = vecnorm(u_si);                                 % 1 x
        N, [N]
```

```
197     n_hat_k = u_si ./ max(T_k_si, 1e-9);                    % 3 x
            N, unit
198
199     sol_out = struct( ...
200         'phase_id',       phase_id, ...
201         'x_k',            x_new, ...
202         'u_k',            u_new, ...
203         'Gamma_k',        Gamma_new, ...
204         'sigma',          sigma_new, ...
205         'x_target',       x_target_nd, ...
206         't_grid_si',      t_grid_si, ...
207         'T_k_si',         T_k_si, ...
208         'n_hat_k',        n_hat_k, ...
209         'lcvx_slack_N',   info.lcvx_slack, ...
210         'nu_norm',        info.nu_norm, ...
211         'exit_code',      info.exit_code, ...
212         'iter',           iters_used, ...
213         'iter_history',   iter_hist, ...
214         'converged',      converged, ...
215         'valid',          false, ...                         % set
                by check
216         'fault_code',     -1, ...                           % set
                by check
217         't_solve_total', 0);                                % set
                below
218
219     %% Step 8 - extract the first-step command
220     [T_cmd, n_hat_cmd, ~] = extract_command( ...
221         u_new(:,1), Gamma_new(1), params);
222
223     %% Step 9 - validity gate
224     [is_valid, fault_code, fallback_flag, rejection_count_new] = ...
225         validity_check(sol_out, rejection_count, params);
226
227     sol_out.valid         = is_valid;
228     sol_out.fault_code    = fault_code;
229     sol_out.t_solve_total = toc(t_start);
230 end
231
232
233 %%  LOCAL - phase identification from current time
234 function phase_id = identify_phase(t, transitions)
235 % Returns the integer phase index containing time t.
236 % Uses the half-open interval [t_k, t_{k+1}).  Clamps at edges.
237     if t <= transitions(1)
238         phase_id = 1;
239         return;
```

```
240     end
241     if t >= transitions(end)
242         phase_id = numel(transitions) - 1;
243         return;
244     end
245     phase_id = find(t >= transitions(1:end-1) & t <
            transitions(2:end), 1);
246 end
```

Listing F.4: `mpc_solve.m` — MPC top-level solve: warm-start seed selection, SCP outer-loop invocation, and five-flag validity-gate evaluation before returning the accepted plan to the dispatcher (Sections 3.5–3.6).

F.3 Navigation: Multiplicative EKF Initialisation

The 16-error-state MEKF is initialised by the routine below (Section 3.4). Prediction and update steps are implemented as Simulink MATLAB function blocks and are not reproduced here for brevity.

F.3.1 Initialisation Routine

```
1  function [x_nom, q_ref, P] = ekf_init_mekf(x_true_0, params)
2  % EKF_INIT_MEKF  Initialise error-state MEKF.
3
4  % Returns:
5  %   x_nom (13x1)  - nominal estimates [r(3); v(3); b_g(3); b_a(3);
       m(1)]
6  %   q_ref (4x1)   - reference attitude quaternion, Hamilton
       scalar-first
7  %   P (16x16)     - error-state covariance over [ r ;  v ;     ;
       b_g ;  b_a ;  m ]
8  % L. White, University of Sheffield, May 2026
9
10 % Nominal estimates
11 r_hat = x_true_0(1:3) + params.gps.pos_noise_std .* randn(3,1);
12 v_hat = zeros(3,1);
13 b_g   = zeros(3,1);
14 b_a   = zeros(3,1);
15 m_hat = params.m0 + params.ekf.mass_init_std * randn();
16 m_hat = max(m_hat, params.ms);
17 m_hat = min(m_hat, params.m0 * 1.10);
18
19 x_nom = [r_hat; v_hat; b_g; b_a; m_hat];   % (13x1)
20
```

```
% Reference attitude: upright on the pad
q_ref = [1; 0; 0; 0];                          % (4x1)

% Initial error-state covariance (16x16)
sigma_r   = params.gps.pos_noise_std;
sigma_v   = [0.5; 0.5; 0.5];
sigma_att = [0.5; 0.5; 0.5] * (pi/180);      % 3x1 rad
sigma_bg  = 3 * params.imu.gyro_bias_sigma;
sigma_ba  = params.imu.accel_bias_sigma;
sigma_m   = params.ekf.mass_init_std;

% Special-case for zeroed gyro/accel bias_sigma during diagnostic
    runs:
% replace zero with a small floor so P doesn't have zero diagonal
    entries
% (which would make the filter believe biases are exactly zero
    forever).
sigma_bg = max(sigma_bg, 1e-3 * ones(3,1));
sigma_ba = max(sigma_ba, 1e-2 * ones(3,1));

p_diag = [sigma_r.^2;        % indices 1-3    r
    sigma_v.^2;              % indices 4-6    v
    sigma_att.^2;            % indices 7-9
    sigma_bg.^2;             % indices 10-12  b_g
    sigma_ba.^2;             % indices 13-15  b_a
    sigma_m^2];              % index   16     m

P = diag(p_diag);


% DIAGNOSTIC: freeze biases at zero
P(10:12, 10:12) = zeros(3,3);   %  b_g  block
P(13:15, 13:15) = zeros(3,3);   %  b_a  block

% Also zero any cross-correlations (should already be 0 in
    block-diagonal init, but be safe)
P(10:15, :)     = 0;
P(:, 10:15)     = 0;
P(10:15, 10:15) = 0;

end
```

Listing F.5: `ekf_init_mekf.m` — MEKF initialisation: nominal state vector, error-state covariance, and process/measurement noise priors (Section 3.4).

Appendix G

Simulink Model

The complete top-level Simulink model used for the end-to-end closed-loop simulation (Section 3.8). The diagram is too wide to capture in a single screenshot at legible resolution, so it is presented as two adjacent halves; together they form the full top-level layout. Both are rotated to read top-to-bottom on the portrait page.

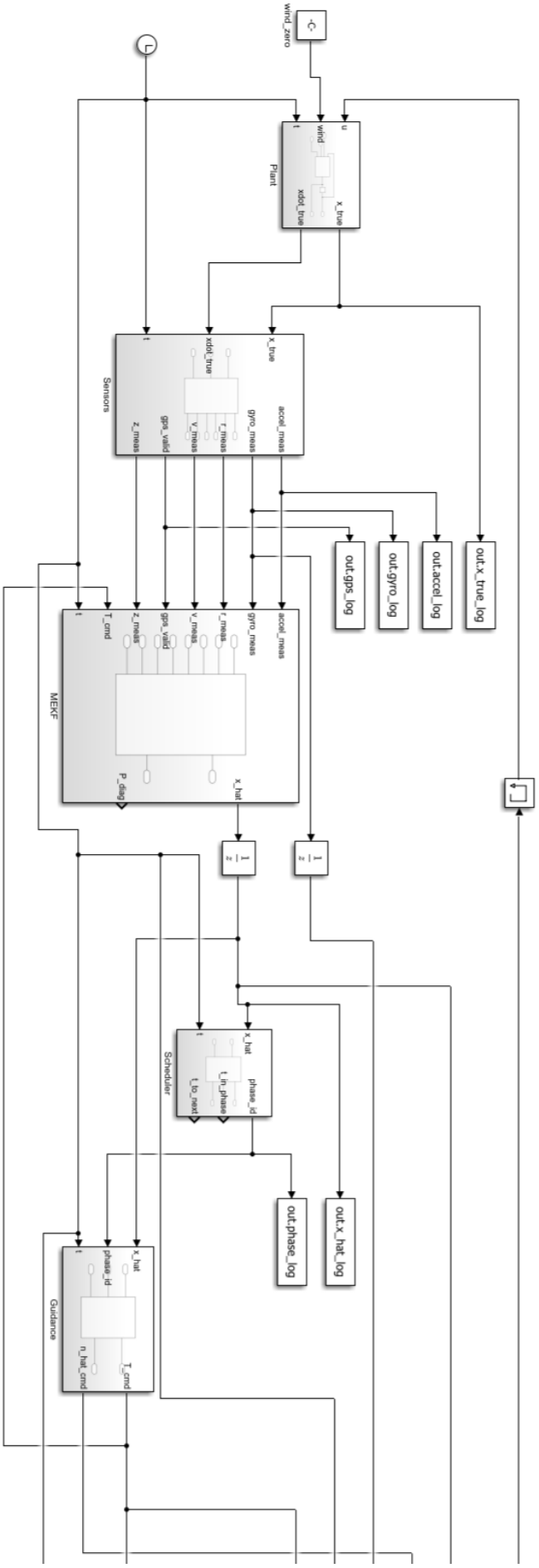


Figure G.1: Top-level Simulink closed-loop model, left half. Continues in Figure G.2.

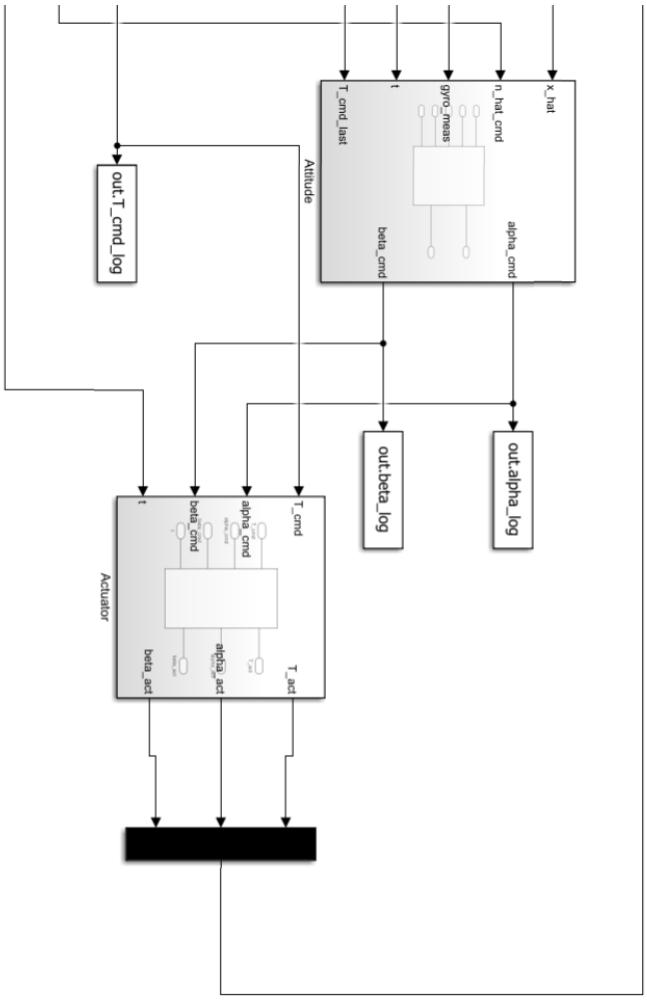


Figure G.2: Top-level Simulink closed-loop model, right half. Continues from Figure G.1.

Appendix H

Ethics Approval Letter

Not applicable, this project is a computational simulation study with no human participants, animals, or hazardous materials.